ỦY BAN NHÂN DÂN TP HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



Họ và tên sinh viên: Đào Duy Hưng

Mã số sinh viên: 3122410158

LAB03 THỎA MÃN RÀNG BUỘC

Môn học : **Trí tuệ nhân tạo nâng cao**

Giảng viên : **ThS. Đỗ Như Tài**

TP. Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1: BIỂU DIỄN CSP VÀ SINH ĐỒ THỊ CẦN TÔ MÀU 2
1.1. Giới thiệu bài toán thỏa mãn ràng buộc 2
1.1.1. Khái niệm Constraint Satisfaction Problem (CSP) 2
1.1.2. Mục tiêu bài thực hành 2
1.1.3. Giới thiệu bài toán tô màu đồ thị 3
1.2. Mô hình hóa bài toán tô màu đồ thị 3
1.2.1. Tập biến X và các đỉnh của đồ thị 3
1.2.2. Miền giá trị D và tập màu cho phép 4
1.2.3. Tập ràng buộc C giữa các đỉnh kề nhau 4
1.2.4. Nguyên tắc biểu diễn ràng buộc với cạnh i < j 5
1.3. Sinh và biểu diễn đồ thị bằng Python 5
1.3.1. Khởi tạo các điểm ngẫu nhiên 5
1.3.2. Xây dựng đồ thị bằng phép tam giác hóa Delaunay 5
1.3.3. Xác định danh sách đỉnh, cạnh và neighbors 6
1.3.4. Trực quan hóa đồ thị được sinh ra 7
1.4. Khảo sát miền giá trị và tính hợp lệ của phép tô màu 8
1.4.1. Khảo sát bài toán với miền bốn màu 8
1.4.2. Khảo sát bài toán với miền ba màu 8
1.4.3. Phân biệt phép gán đầy đủ và phép gán hợp lệ 9
1.5. Kết quả thực nghiệm 9
1.5.1. Thống kê số đỉnh, số cạnh và quan hệ láng giềng 9
1.5.2. Kết quả tô màu trên đồ thị Delaunay 10
1.5.3. So sánh khả năng thỏa mãn ràng buộc với ba và bốn màu 10
1.6. Nhận xét 11
CHƯƠNG 2: KIỂM TRA TÍNH ĐẦY ĐỦ VÀ NHẤT QUÁN 13
2.1. Cơ sở lý thuyết về phép gán trong CSP 13
2.1.1. Phép gán rỗng và phép gán từng phần 13
2.1.2. Phép gán đầy đủ 14
2.1.3. Điều kiện xác định nghiệm CSP hợp lệ 14
2.2. Cấu trúc dữ liệu biểu diễn CSP 15
2.2.1. Thành phần variables 15
2.2.2. Thành phần domains 15
2.2.3. Thành phần constraints 16
2.3. Các hàm kiểm tra phép gán 16
2.3.1. Hàm complete() kiểm tra tính đầy đủ 16

2.3.2. Hàm consistent() kiểm tra tính nhất quán....17
2.3.3. Kiểm tra giá trị thuộc miền và xác định nghiệm hợp lệ....17
2.4. Xây dựng và thực hiện các ca kiểm thử....18
2.4.1. Kiểm thử phép gán rỗng và từng phần....18
2.4.2. Kiểm thử phép gán đầy đủ nhưng vi phạm ràng buộc....18
2.4.3. Kiểm thử phép gán đầy đủ và hợp lệ....18
2.4.4. Kiểm thử phép gán sử dụng giá trị ngoài miền....19
2.5. Kết quả thực nghiệm và phân tích....19
2.5.1. Bảng tổng hợp các ca kiểm thử....19
2.5.2. So sánh kết quả complete() và consistent()....20
2.5.3. Phân tích sự khác biệt giữa tính đầy đủ và tính nhất quán....21
2.6. Nhận xét....21
CHƯƠNG 3: BACKTRACKING SEARCH VÀ THỨ TỰ CHỌN BIẾN....23
3.1. Cơ sở lý thuyết thuật toán Backtracking Search....23
3.1.1. Nguyên lý tìm kiếm quay lui....23
3.1.2. Cơ chế mở rộng và quay lui khi vi phạm ràng buộc....23
3.1.3. Vai trò của thứ tự chọn biến và giá trị....23
3.2. Thuật toán Backtracking cơ bản....24
3.2.1. Triển khai hàm backtrack_search()....24
3.2.2. Cơ chế lựa chọn biến chưa được gán....24
3.2.3. Theo dõi COUNT và trạng thái tìm kiếm....24
3.3. Các chiến lược cải thiện thứ tự tìm kiếm....25
3.3.1. Minimum Remaining Values (MRV)....25
3.3.2. Least Constraining Value (LCV)....25
3.3.3. Kết hợp MRV và LCV trong Backtracking Search....25
3.4. Thiết lập và kết quả thực nghiệm....26
3.4.1. Điều kiện thực nghiệm trên cùng đồ thị....26
3.4.2. Kết quả Backtracking với miền bốn màu....26
3.4.3. Kết quả Backtracking với miền ba màu....26
3.4.4. Bảng so sánh COUNT, số lần thử màu và thời gian chạy....26
3.4.5. Trực quan hóa nghiệm tô màu và hiệu suất tìm kiếm....27
3.5. Đánh giá hiệu quả của MRV và LCV....28
3.5.1. Ảnh hưởng đến số nút được kiểm tra....28
3.5.2. Ảnh hưởng đến số lần thử gán giá trị....28
3.5.3. Sự đánh đổi giữa chi phí heuristic và thời gian thực thi....29
3.6. Nhận xét....29

CHƯƠNG 4: FORWARD CHECKING VÀ MIN-CONFLICTS..... 30
4.1. Giới thiệu các phương pháp cải thiện tìm kiếm CSP..... 30
4.1.1. Hạn chế của Backtracking cơ bản..... 30
4.1.2. Phương pháp suy luận ràng buộc và tìm kiếm cục bộ..... 31
4.2. Thuật toán Forward Checking..... 31
4.2.1. Nguyên lý loại bỏ giá trị không hợp lệ khỏi miền..... 31
4.2.2. Cập nhật miền các đỉnh láng giềng sau khi gán màu..... 32
4.2.3. Cơ chế khôi phục miền khi quay lui..... 32
4.2.4. Kết hợp Forward Checking với MRV và LCV..... 32
4.3. Thuật toán Min-Conflicts..... 33
4.3.1. Nguyên lý tìm kiếm cục bộ trong CSP..... 33
4.3.2. Khởi tạo phép gán đầy đủ..... 33
4.3.3. Lựa chọn biến đang vi phạm ràng buộc..... 34
4.3.4. Lựa chọn giá trị nhằm giảm số ràng buộc vi phạm..... 34
4.3.5. Điều kiện dừng và cơ chế khởi tạo lại..... 34
4.4. Thiết lập thực nghiệm..... 35
4.4.1. Sinh đồ thị Delaunay với các kích thước khác nhau..... 35
4.4.2. Khảo sát miền ba màu và bốn màu..... 35
4.4.3. Thiết lập random seed và số lần thực nghiệm..... 35
4.4.4. Các chỉ số COUNT, số bước, tỷ lệ thành công và thời gian..... 36
4.5. Kết quả thực nghiệm và so sánh..... 36
4.5.1. Kết quả trên đồ thị ban đầu..... 36
4.5.2. Kết quả trên nhiều đồ thị ngẫu nhiên..... 37
4.5.3. So sánh tỷ lệ tìm được nghiệm hợp lệ..... 38
4.5.4. So sánh COUNT, số bước và thời gian thực thi..... 39
4.5.5. Phân biệt trường hợp vô nghiệm và hết ngân sách tìm kiếm..... 41
4.6. Nhận xét..... 41
CHƯƠNG 5: BIỂU DIỄN BÀI TOÁN N QUÂN HẬU..... 43
5.1. Giới thiệu bài toán n-Queens..... 43
5.1.1. Phát biểu bài toán và mục tiêu tìm kiếm..... 43
5.1.2. Quy tắc tấn công của quân hậu..... 43
5.1.3. Mô hình hóa n-Queens dưới góc độ CSP..... 44
5.2. Biểu diễn trạng thái bàn cờ..... 44
5.2.1. Vector trạng thái gồm n phần tử..... 44
5.2.2. Quy ước mỗi cột có đúng một quân hậu..... 45
5.2.3. Không gian trạng thái của bài toán..... 45

5.3. Các hàm xử lý bài toán n-Queens..... 46
5.3.1. Hàm random_board() khởi tạo bàn cờ ngẫu nhiên..... 46
5.3.2. Hàm show_board() trực quan hóa bàn cờ..... 46
5.3.3. Hàm conflicts() đếm các cặp quân hậu xung đột..... 47
5.4. Hàm mục tiêu và điều kiện nghiệm hợp lệ..... 47
5.4.1. Xung đột giữa các quân hậu cùng hàng..... 47
5.4.2. Xung đột giữa các quân hậu trên đường chéo..... 47
5.4.3. Hàm mục tiêu h(board)..... 48
5.4.4. Điều kiện nghiệm hợp lệ h(board) = 0..... 48
5.5. Kết quả thực nghiệm..... 49
5.5.1. Khảo sát các bàn cờ ngẫu nhiên với n = 4..... 49
5.5.2. Khảo sát các bàn cờ ngẫu nhiên với n = 8..... 49
5.5.3. Kiểm chứng cấu hình mẫu [1, 3, 0, 2]..... 49
5.5.4. Đối chiếu hàm conflicts() với phương pháp kiểm tra từng cặp hậu..... 51
5.6. Nhận xét..... 52
CHƯƠNG 6: CÁC BIẾN THỂ HILL CLIMBING..... 53
6.1. Cơ sở lý thuyết về Hill Climbing..... 53
6.1.1. Nguyên lý tối ưu hóa cục bộ..... 53
6.1.2. Hàm mục tiêu và không gian trạng thái lân cận..... 53
6.1.3. Hiện tượng Local Optimum và Plateau..... 54
6.2. Triển khai các thuật toán Hill Climbing..... 55
6.2.1. Steepest-ascent Hill Climbing – Task 1..... 55
6.2.2. Inefficient Stochastic Hill Climbing – Task 2..... 55
6.2.3. Efficient Stochastic Hill Climbing – Task 3..... 56
6.3. Cơ chế lựa chọn trạng thái và điều kiện dừng..... 56
6.3.1. Lựa chọn trạng thái cải thiện tốt nhất..... 56
6.3.2. Lựa chọn ngẫu nhiên trong tập trạng thái cải thiện..... 56
6.3.3. Sinh và đánh giá từng trạng thái lân cận ngẫu nhiên..... 57
6.3.4. Điều kiện SOLVED, LOCAL_MINIMUM, TRY_LIMIT và STEP_LIMIT 57
6.4. Thiết lập và kết quả thực nghiệm..... 58
6.4.1. Thiết lập random seed và số lần chạy..... 58
6.4.2. Kết quả thực nghiệm trên n = 4..... 58
6.4.3. Kết quả thực nghiệm trên n = 8..... 59
6.4.4. Bảng so sánh tỷ lệ thành công và số xung đột cuối..... 59
6.4.5. So sánh số bước và số lần đánh giá hàm conflicts()..... 60
6.5. Phân tích hiệu quả các thuật toán..... 60

6.5.1. Trực quan hóa quá trình hội tụ..... 60
6.5.2. Sự đánh đổi giữa chất lượng nghiệm và chi phí tính toán..... 62
6.5.3. Nguyên nhân dừng sớm tại cực tiểu cục bộ..... 62
6.6. Nhận xét..... 63
CHƯƠNG 7: RANDOM RESTART VÀ SIMULATED ANNEALING..... 64
7.1. Hạn chế của Hill Climbing truyền thống..... 64
7.1.1. Sự phụ thuộc vào trạng thái khởi tạo..... 64
7.1.2. Khả năng mắc kẹt tại cực tiểu cục bộ..... 64
7.1.3. Động cơ sử dụng Random Restart và Simulated Annealing..... 65
7.2. Thuật toán Random Restart..... 65
7.2.1. Nguyên lý khởi tạo lại trạng thái ngẫu nhiên..... 65
7.2.2. Triển khai Random Restart cho ba biến thể Hill Climbing..... 66
7.2.3. Giới hạn số lần khởi động lại..... 66
7.2.4. Lưu trữ nghiệm tốt nhất và thống kê chi phí tìm kiếm..... 67
7.3. Thuật toán Simulated Annealing..... 67
7.3.1. Nguyên lý hoạt động của Simulated Annealing..... 67
7.3.2. Quy tắc chấp nhận trạng thái tốt hơn và xấu hơn..... 68
7.3.3. Xác suất chấp nhận bước xấu phụ thuộc nhiệt độ..... 68
7.3.4. Lịch làm nguội Exponential..... 68
7.3.5. Lịch làm nguội Linear..... 69
7.3.6. Điều kiện dừng và lưu nghiệm tốt nhất..... 69
7.4. Thiết lập thực nghiệm..... 70
7.4.1. Điều kiện chạy trên n = 4 và n = 8..... 70
7.4.2. Thiết lập random seed và ngân sách tìm kiếm..... 70
7.4.3. Các chỉ số đánh giá thuật toán..... 71
7.5. Kết quả thực nghiệm..... 71
7.5.1. Kết quả Random Restart trên n = 4 và n = 8..... 71
7.5.2. Kết quả Simulated Annealing với hai lịch làm nguội..... 72
7.5.3. So sánh tỷ lệ đạt nghiệm h = 0..... 73
7.5.4. So sánh số lần đánh giá và thời gian thực thi..... 73
7.5.5. Trực quan hóa quá trình hội tụ của Simulated Annealing..... 74
7.6. Nhận xét..... 75
CHƯƠNG 8: ĐÁNH GIÁ TỔNG HỢP VÀ BÀI NÂNG CAO..... 77
8.1. Mục tiêu và phạm vi thực nghiệm tổng hợp..... 77
8.1.1. Các thuật toán được lựa chọn để so sánh..... 77
8.1.2. Thiết lập dữ liệu và random seed..... 77

8.1.3. Số lần chạy và ngân sách thực nghiệm....78
8.1.4. Các chỉ số đánh giá hiệu suất....78
8.2. Kết quả thực nghiệm tổng hợp trên n-Queens....79
8.2.1. Kết quả Task 6 trên n = 4....79
8.2.2. Kết quả Task 6 trên n = 8....80
8.2.3. Bảng so sánh tỷ lệ thành công và h cuối trung bình....80
8.2.4. So sánh số bước và số lần đánh giá hàm mục tiêu....81
8.2.5. So sánh thời gian thực thi giữa các thuật toán....82
8.3. Phân tích quá trình hội tụ....83
8.3.1. Trực quan hóa đường hội tụ trên cùng trạng thái khởi tạo....83
8.3.2. Khả năng cải thiện trong giai đoạn đầu....83
8.3.3. Hiện tượng Plateau và Local Optimum....84
8.3.4. Ảnh hưởng của Random Restart và Simulated Annealing đến hội tụ....84
8.4. Khảo sát khả năng mở rộng....85
8.4.1. Thiết lập thực nghiệm với n = 4, 8, 12, 16, 20....85
8.4.2. Kết quả thời gian thực thi theo kích thước bàn cờ....85
8.4.3. Biểu đồ log-log thời gian chạy....86
8.4.4. Phân tích độ dốc thực nghiệm và chi phí cài đặt....87
8.4.5. Đánh giá khả năng mở rộng của các thuật toán....87
8.5. Phần thực nghiệm nâng cao về toán tử lân cận....88
8.5.1. Các toán tử di chuyển được khảo sát....88
8.5.2. Ảnh hưởng của toán tử đến không gian trạng thái lân cận....88
8.5.3. Thực nghiệm trên n = 8 và n = 12....89
8.5.4. So sánh tỷ lệ thành công và h cuối trung bình....89
8.5.5. Trực quan hóa quá trình hội tụ của các toán tử....90
8.6. Đánh giá tổng hợp các phương pháp tìm kiếm....91
8.6.1. Tính hợp lệ và chất lượng nghiệm....91
8.6.2. Tác động của Local Optimum, Plateau và Ridge....92
8.6.3. Hiệu quả của các biến thể Hill Climbing....92
8.6.4. Vai trò của Random Restart và Simulated Annealing....92
8.6.5. Sự đánh đổi giữa tỷ lệ thành công và chi phí tính toán....93
8.6.6. Hạn chế của bộ thực nghiệm và điều kiện áp dụng....93
8.7. Kết luận bài thực hành tổng hợp....93

LỜI MỞ ĐẦU

Trong bối cảnh trí tuệ nhân tạo ngày càng được ứng dụng rộng rãi trong học tập, nghiên cứu và thực tiễn, việc hiểu rõ các nguyên lý, phương pháp và cách xây dựng hệ thống trí tuệ nhân tạo là nền tảng cần thiết đối với sinh viên theo định hướng công nghệ. Môn học Trí tuệ nhân tạo nâng cao giúp người học tiếp cận các kiến thức và vấn đề cốt lõi của lĩnh vực AI, đồng thời rèn luyện khả năng phân tích, tư duy logic và vận dụng kiến thức vào những bài toán thực tế.

Assignment này được thực hiện nhằm củng cố và hệ thống hóa những kiến thức đã được học thông qua việc nghiên cứu, phân tích và trả lời các câu hỏi theo từng chủ đề. Thay vì chỉ tiếp cận kiến thức ở mức lý thuyết, quá trình thực hiện bài tập giúp tôi hiểu rõ hơn cách các khái niệm AI được hình thành, cách chúng liên kết với nhau và cách có thể áp dụng vào các bài toán cụ thể.

Trong quá trình thực hiện, tôi tập trung vào việc tự tìm hiểu nội dung, đối chiếu với tài liệu môn học, phân tích vấn đề và trình bày kết quả theo hướng rõ ràng, có lập luận. Qua đó, tôi không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện khả năng nghiên cứu tài liệu, giải quyết vấn đề và đánh giá một hệ thống AI dưới nhiều góc độ khác nhau.

Thông qua assignment này và các bài tập tiếp theo, tôi kỳ vọng có thể xây dựng nền tảng kiến thức vững chắc hơn về Trí tuệ nhân tạo, đồng thời từng bước nâng cao khả năng vận dụng kiến thức vào các bài toán thực tế. Đây cũng là cơ sở để tôi tiếp tục phát triển các kỹ năng chuyên môn và nghiên cứu sâu hơn trong lĩnh vực AI trong quá trình học tập và định hướng nghề nghiệp sa

CHƯƠNG 1: BIỂU DIỄN CSP VÀ SINH ĐỒ THỊ CẦN TÔ MÀU

1.1. Giới thiệu bài toán thỏa mãn ràng buộc

1.1.1. Khái niệm Constraint Satisfaction Problem (CSP)

Bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problem – CSP) là một mô hình giải quyết vấn đề trong trí tuệ nhân tạo, trong đó lời giải được hình thành bằng cách gán giá trị cho tập hợp các biến sao cho mọi ràng buộc của bài toán đều được thỏa mãn. Khác với những bài toán tìm đường chú trọng quá trình di chuyển từ trạng thái ban đầu đến trạng thái đích, CSP tập trung vào việc xác định một phép gán thỏa mãn đồng thời các điều kiện đã cho.

Một bài toán CSP được biểu diễn bằng bộ ba CSP = (X, D, C), trong đó X là tập biến cần gán giá trị, D là tập miền giá trị tương ứng của các biến và C là tập ràng buộc xác định những tổ hợp giá trị được chấp nhận. Một phép gán chỉ được xem là nghiệm CSP khi tất cả biến đều có giá trị thuộc miền cho phép và không có ràng buộc nào bị vi phạm.

1.1.2. Mục tiêu bài thực hành

Bài thực hành sử dụng notebook CSP_graph_coloring_example.ipynb để khảo sát cách xây dựng và biểu diễn CSP thông qua bài toán tô màu đồ thị. Nội dung tập trung vào việc sinh đồ thị từ các điểm hai chiều bằng phép tam giác hóa Delaunay, xác định quan hệ láng giềng giữa các đỉnh, xây dựng ba thành phần X, D, C và kiểm tra sự khác biệt khi sử dụng miền ba màu hoặc bốn màu.

Bên cạnh đó, bài thực hành làm rõ ý nghĩa của phép gán đầy đủ và nghiệm hợp lệ. Kết quả được quan sát trực tiếp trên đồ thị, đồng thời đối chiếu với dữ liệu xuất từ notebook nhằm kiểm chứng tính đúng đắn của mô hình.

1.1.3. Giới thiệu bài toán tô màu đồ thị

Cho đồ thị vô hướng G = (V, E), trong đó V là tập các đỉnh và E là tập các cạnh nối giữa những cặp đỉnh kề nhau. Bài toán tô màu đồ thị yêu cầu gán cho mỗi đỉnh một màu trong bảng màu cho phép sao cho hai đỉnh có chung cạnh không nhận cùng một màu. Những đỉnh không kề nhau có thể sử dụng trùng màu mà không vi phạm điều kiện bài toán.

Trong thí nghiệm, đồ thị có 10 đỉnh được sinh từ tọa độ hai chiều. Vị trí hình học được dùng để xác định các cạnh, còn màu sắc mới là giá trị cần tìm trong bài toán CSP. Vì vậy, khoảng cách hoặc tọa độ giữa các đỉnh không trực tiếp quyết định một phép tô màu có hợp lệ hay không; điều quyết định là quan hệ kề được tạo ra từ đồ thị.

1.2. Mô hình hóa bài toán tô màu đồ thị

1.2.1. Tập biến X và các đỉnh của đồ thị

Mỗi đỉnh tương ứng với một biến của CSP. Với n = 10, tập biến có dạng:

$$X = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

Trong mã Python của notebook, các tên biến được lưu dưới dạng chuỗi ký tự thông qua variables = [str(var) for var in range(n)]. Chẳng hạn, biến '4' đại diện cho màu cần gán ở đỉnh số 4. Cách đặt tên này thuận tiện khi xây dựng từ điển lưu phép gán và danh sách các ràng buộc nhị phân.

1.2.2. Miền giá trị D và tập màu cho phép

Miền giá trị của mỗi biến là tập các màu mà đỉnh được phép sử dụng. Notebook khảo sát hai cấu hình trên cùng một đồ thị:

$$D_i^{(4)} = \{\text{red, blue, green, orange}\};$$

$$D_i^{(3)} = \{\text{red, blue, green}\}, \text{ với mọi đỉnh } i \text{ thuộc } X.$$

Trong chương trình, thông tin miền được lưu ở csp['domains'], ánh xạ mỗi tên đỉnh tới danh sách màu. Khi chuyển từ bốn màu sang ba màu, chỉ miền giá trị thay đổi; tập đỉnh và các cạnh của đồ thị được giữ nguyên để bảo đảm tính so sánh.

Với 10 đỉnh, số phép gán đầy đủ về mặt tổ hợp là $4^{10} = 1.048.576$ nếu có bốn màu và $3^{10} = 59.049$ nếu có ba màu. Đây là số phép gán có thể tạo ra trước khi xét ràng buộc, không phải số nghiệm hợp lệ của CSP.

1.2.3. Tập ràng buộc C giữa các đỉnh kề nhau

Với mỗi cạnh (i, j) thuộc E, bài toán yêu cầu hai đầu cạnh phải mang màu khác nhau:

$$C_i\square: \text{color}(i) \neq \text{color}(j), \text{ với } (i, j) \in E.$$

Tập ràng buộc được biểu diễn bởi các cặp tên biến trong csp['constraints']. Đây là các ràng buộc nhị phân vì mỗi điều kiện liên quan tới đúng hai biến. Đối với đồ thị thực nghiệm, có 21 cạnh không hướng nên tương ứng 21 ràng buộc khác màu. Việc thỏa mãn toàn bộ các ràng buộc này là điều kiện cốt lõi của nghiệm tô màu hợp lệ.

1.2.4. Nguyên tắc biểu diễn ràng buộc với cạnh i < j

Trong đồ thị vô hướng, nếu đỉnh i kề đỉnh j thì đỉnh j cũng kề đỉnh i. Do đó, khi duyệt các danh sách neighbors, một cạnh có thể được gặp hai lần dưới dạng (i, j) và (j, i). Nếu đưa cả hai vào tập ràng buộc, chương trình sẽ lưu hai lần cùng một điều kiện khác màu.

Notebook sử dụng điều kiện if i < j khi tạo ràng buộc. Nhờ đó, với mỗi cạnh không hướng, chỉ cặp có chỉ số nhỏ đứng trước được giữ lại. Điều kiện này không biến đồ thị thành đồ thị có hướng mà chỉ tránh đếm trùng. Kết quả kiểm tra cho thấy tổng bậc các đỉnh bằng 42, đúng bằng hai lần số cạnh: $42 = 2 \times 21$.

1.3. Sinh và biểu diễn đồ thị bằng Python

1.3.1. Khởi tạo các điểm ngẫu nhiên

Notebook dùng NumPy để khởi tạo 10 điểm trong không gian hai chiều, với các tọa độ nguyên nằm trong khoảng từ 0 đến 99. Tham số np.random.seed(1111) được thiết lập trước khi sinh dữ liệu, sau đó các điểm được sắp xếp theo hoành độ bằng np.argsort(points[:, 0]).

Việc cố định seed giúp phép sinh điểm có thể tái lập trong cùng môi trường và giữ thống nhất dữ liệu khi khảo sát nhiều miền màu. Mỗi điểm được đánh số từ 0 đến 9 sau thao tác sắp xếp, qua đó xác định tên đỉnh được dùng xuyên suốt các bước xây dựng đồ thị và CSP.

1.3.2. Xây dựng đồ thị bằng phép tam giác hóa Delaunay

Từ mảng tọa độ points, chương trình gọi Delaunay(points) của thư viện scipy.spatial để xây dựng phép tam giác hóa. Các cạnh thuộc các tam giác được sử dụng làm quan hệ kề giữa các điểm, tạo thành đồ thị vô hướng có thể trực quan hóa trên mặt phẳng.

Hàm plt.triplot() biểu diễn những cạnh này, trong khi các điểm và nhãn được hiển thị bằng Matplotlib. Kết quả tạo được đồ thị 10 đỉnh và 21 cạnh. Đồ thị là dữ liệu đầu vào chung cho cả khảo sát ba màu lẫn bốn màu, do đó sự khác biệt về khả năng tìm nghiệm xuất phát từ miền giá trị chứ không phải từ việc thay đổi cấu trúc đồ thị.

1.3.3. Xác định danh sách đỉnh, cạnh và neighbors

Quan hệ láng giềng được lấy từ thuộc tính tri.vertex_neighbor_vertices, gồm hai mảng indptr và indices. Danh sách láng giềng của đỉnh i được truy xuất theo đoạn indices[indptr[i]:indptr[i+1]], rồi sắp xếp để thuận tiện cho việc kiểm tra và xuất bảng.

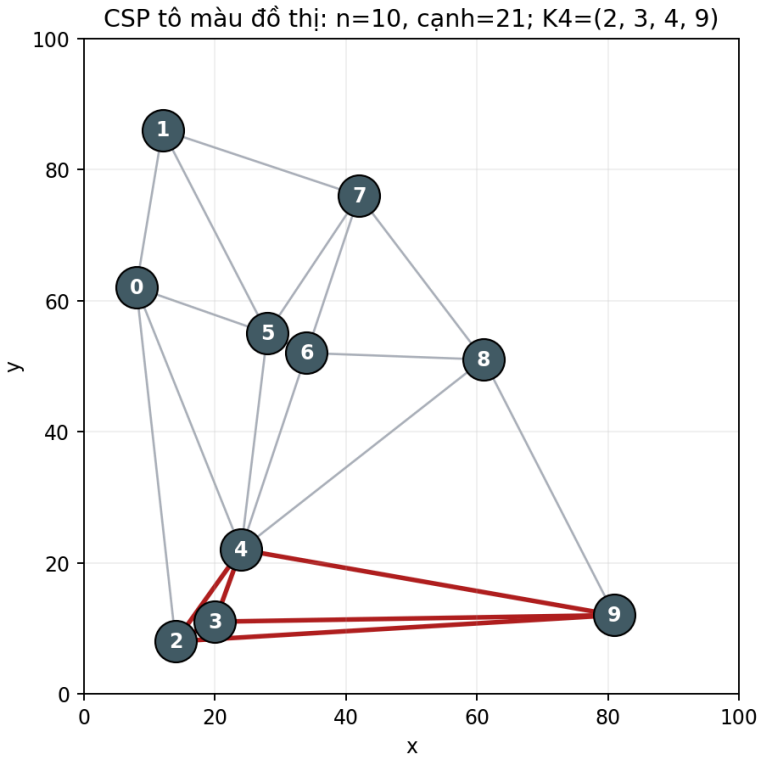
Bảng 1.1 trình bày danh sách kề được notebook xuất ra. Bậc của một đỉnh là số đỉnh láng giềng trực tiếp, đồng thời cũng là số cạnh đi qua đỉnh đó.

Bảng 1.1. Danh sách láng giềng và bậc của các đỉnh trên đồ thị Delaunay

Đỉnh	Các đỉnh kề (neighbors)	Bậc
0	1, 2, 4, 5	4
1	0, 5, 7	3
2	0, 3, 4, 9	4
3	2, 4, 9	3
4	0, 2, 3, 5, 6, 8, 9	7
5	0, 1, 4, 6, 7	5
6	4, 5, 7, 8	4
7	1, 5, 6, 8	4
8	4, 6, 7, 9	4
9	2, 3, 4, 8	4

Đỉnh 4 có bậc lớn nhất bằng 7, còn các đỉnh 1 và 3 có bậc nhỏ nhất bằng 3. Tổng bậc bằng 42, bậc trung bình là 42/10 = 4,2. Kiểm thử trong notebook cũng xác nhận rằng quan hệ láng giềng đối xứng và danh sách cạnh không chứa phần tử trùng lặp.

1.3.4. Trực quan hóa đồ thị được sinh ra



Hình 1.1. Đồ thị Delaunay gồm 10 đỉnh, 21 cạnh; các cạnh tô đỏ làm nổi bật đồ thị con đầy đủ K_4 với bốn đỉnh 2, 3, 4 và 9.

Hình 1.1 cho thấy cấu trúc kết nối của các đỉnh sau phép tam giác hóa. Các cạnh màu xám biểu diễn quan hệ kề thông thường. Sáu cạnh màu đỏ nối bốn đỉnh 2, 3, 4 và 9 cho thấy mỗi đỉnh trong nhóm đều kề với ba đỉnh còn lại. Nhóm này tạo thành đồ thị đầy đủ K_4 và có vai trò quyết định trong việc đánh giá khả năng tô màu bằng ba màu ở phần thực nghiệm tiếp theo.

1.4. Khảo sát miền giá trị và tính hợp lệ của phép tô màu

1.4.1. Khảo sát bài toán với miền bốn màu

Với miền D_i = {red, blue, green, orange}, chương trình dùng hàm Backtracking cơ bản có sẵn trong notebook làm công cụ kiểm chứng. Kết quả thu được một phép gán đầy đủ cho cả 10 đỉnh, sử dụng bốn màu và không vi phạm bất kỳ cạnh nào.

Một nghiệm cụ thể được ghi nhận như sau: đỉnh 0 màu đỏ; đỉnh 1 màu xanh dương; đỉnh 2 màu xanh dương; đỉnh 3 màu đỏ; đỉnh 4 màu xanh lá; đỉnh 5 màu cam; đỉnh 6 màu đỏ; đỉnh 7 màu xanh lá; đỉnh 8 màu xanh dương; đỉnh 9 màu cam. Nghiệm này chỉ là một trong những cách tô hợp lệ, không hàm ý mỗi đỉnh bắt buộc phải nhận màu tương ứng trong mọi nghiệm.

1.4.2. Khảo sát bài toán với miền ba màu

Khi miền được giảm còn D_i = {red, blue, green}, Backtracking trả về None, tức không tìm được phép tô hợp lệ. Trong trường hợp này, có thể khẳng định bài toán thực sự vô nghiệm bằng lập luận cấu trúc đồ thị, chứ không chỉ dựa vào kết quả chạy thuật toán.

Cụ thể, bốn đỉnh {2, 3, 4, 9} hình thành K_4 với đủ sáu cạnh nối từng cặp đỉnh. Do tất cả bốn đỉnh đều kề nhau, chúng phải mang bốn màu phân biệt. Nếu chỉ có ba màu, theo nguyên lý đếm, ít nhất hai trong bốn đỉnh sẽ trùng màu và vi phạm ràng buộc. Vì đồ thị toàn phần chứa K_4, không thể tồn tại nghiệm ba màu cho đồ thị đang xét. Kết hợp với nghiệm bốn màu đã tìm được, có thể kết luận số màu tối thiểu của đồ thị thực nghiệm bằng 4.

Kết luận này chỉ áp dụng cho đồ thị cụ thể được tạo bằng seed 1111, không có nghĩa mọi đồ thị phẳng đều cần bốn màu.

1.4.3. Phân biệt phép gán đầy đủ và phép gán hợp lệ

Một phép gán đầy đủ là phép gán đã chỉ định giá trị cho toàn bộ biến trong X. Tuy nhiên, điều kiện đầy đủ không bảo đảm rằng các ràng buộc của CSP đều được thỏa mãn. Chẳng hạn, phép gán cả 10 đỉnh đều màu đỏ là đầy đủ nhưng vi phạm cả 21 cạnh của đồ thị vì mỗi cạnh đều nối hai đỉnh cùng màu.

Ngược lại, phép gán chỉ có đỉnh 0 nhận màu đỏ là phép gán từng phần; dù không tạo xung đột giữa những biến đã gán, nó chưa phải nghiệm vì còn chín đỉnh chưa nhận màu. Do đó, một nghiệm CSP phải đáp ứng đồng thời ba điều kiện: mọi biến đều được gán, giá trị gán thuộc miền cho phép và tất cả ràng buộc giữa các đỉnh kề nhau đều được thỏa mãn.

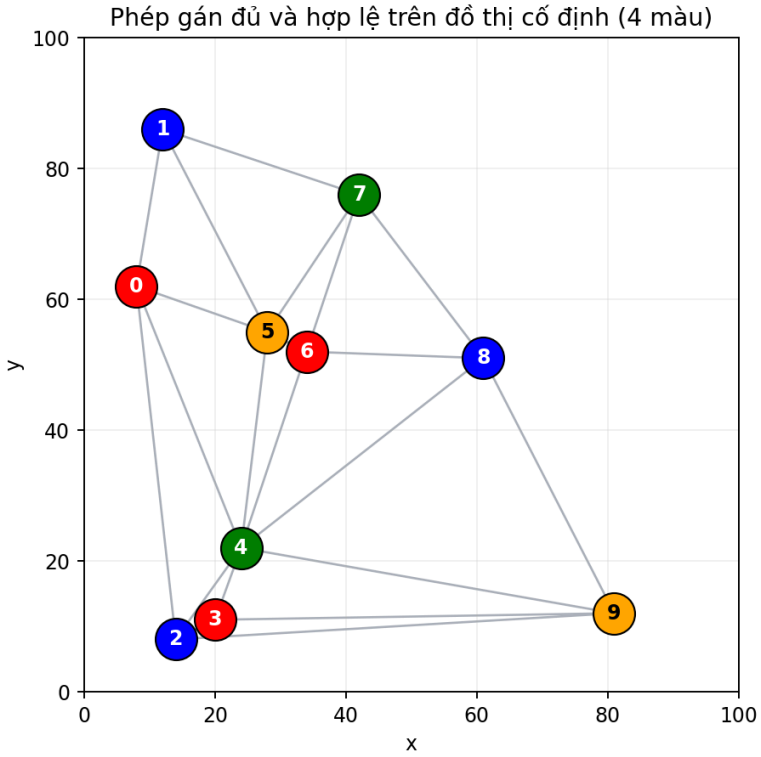
1.5. Kết quả thực nghiệm

1.5.1. Thống kê số đỉnh, số cạnh và quan hệ láng giềng

Kết quả ghi nhận từ các file Bai1_CSP_Edges.csv và Bai1_CSP_Neighbors.csv xác nhận có 10 đỉnh, 21 cạnh không hướng, tổng bậc bằng 42 và bậc trung bình bằng 4,2. Đỉnh có nhiều láng giềng nhất là đỉnh 4 với bảy đỉnh kề. Tập ràng buộc của mô hình CSP tương ứng chính xác với 21 cạnh, nhờ quy tắc chỉ lưu cặp (i, j) khi $i < j$.

Các thống kê này được sử dụng làm cơ sở kiểm tra cấu trúc đồ thị trước khi đánh giá thuật toán tô màu. Việc đối chiếu số cạnh với tổng bậc giúp phát hiện lỗi khi tạo quan hệ kề hoặc vô tình nhân đôi ràng buộc.

1.5.2. Kết quả tô màu trên đồ thị Delaunay



Hình 1.2. Phép tô màu đầy đủ và hợp lệ trên đồ thị Delaunay bằng bốn màu.

Hình 1.2 trực quan hóa nghiệm được tìm bởi Backtracking gốc. Các đỉnh cùng màu không có cạnh nối trực tiếp với nhau, trong khi những đỉnh có chung cạnh luôn mang màu khác nhau. Riêng bốn đỉnh thuộc K_4 được tô bằng bốn màu phân biệt, phù hợp với ràng buộc của đồ thị. Kết quả xác nhận nghiệm không chỉ đầy đủ về số đỉnh được gán mà còn hợp lệ theo tất cả 21 cạnh.

1.5.3. So sánh khả năng thỏa mãn ràng buộc với ba và bốn màu

Bảng 1.2 tổng hợp số liệu từ `Bai1_CSP_Domain_Comparison.csv`. Trong thí nghiệm này, Backtracking gốc được sử dụng nhằm kiểm chứng nghiệm, còn việc nghiên cứu và tối ưu thứ tự chọn biến, chọn giá trị sẽ được triển khai ở những bài thực hành sau.

Bảng 1.2. So sánh kết quả tô màu đồ thị với miền ba màu và bốn màu

Chỉ tiêu	Miền 4 màu	Miền 3 màu
Số đỉnh	10	10
Số cạnh / ràng buộc nhị phân	21	21
Số phép gán đầy đủ tiềm năng	1.048.576	59.049
Tìm được nghiệm hợp lệ	Có	Không
Số nút kiểm tra COUNT	11	70
Thời gian của lượt chạy ghi nhận (ms)	0,1609	0,5809

Đối với miền bốn màu, Backtracking tìm được nghiệm với COUNT = 11; trong khi miền ba màu cần COUNT = 70 trước khi xác định không có nghiệm. COUNT được định nghĩa trong notebook là số lần gọi đệ quy của hàm tìm kiếm, không đồng nghĩa số phép gán màu đã thử. Thời gian 0,1609 ms và 0,5809 ms là giá trị thu được từ một lượt đo nên chỉ có ý nghĩa minh họa, không đủ để kết luận chênh lệch hiệu năng trung bình.

Điểm đáng chú ý là miền ba màu có số phép gán đầy đủ tiềm năng nhỏ hơn miền bốn màu nhưng lại không tồn tại nghiệm trong đồ thị này. Như vậy, việc giảm kích thước miền giá trị không đồng nghĩa với việc làm cho bài toán CSP dễ giải hơn. Tính khả thi phụ thuộc vào sự tương thích giữa miền màu với cấu trúc ràng buộc của đồ thị.

1.6. Nhận xét

Qua Bài 1, bài toán tô màu đồ thị đã được mô hình hóa rõ ràng theo ba thành phần X, D và C của CSP. Phép tam giác hóa Delaunay giúp sinh cấu trúc đồ thị từ dữ liệu tọa độ hai chiều, còn danh sách neighbors và tập cạnh cho phép chuyển cấu trúc đó thành những ràng buộc nhị phân. Cách chỉ tạo ràng buộc với $i < j$ là phù hợp với đồ thị vô hướng và tránh việc lưu trùng cùng một điều kiện khác màu.

Kết quả trên đồ thị 10 đỉnh, 21 cạnh cho thấy miền bốn màu có ít nhất một nghiệm đầy đủ và hợp lệ, trong khi miền ba màu vô nghiệm do tồn tại đồ thị con đầy đủ K_4 gồm các đỉnh 2, 3, 4 và 9. Thí nghiệm vì thế minh họa rằng số màu cần thiết không chỉ phụ thuộc vào số

đỉnh mà còn vào cách các đỉnh liên kết với nhau. Sự tồn tại của một nhóm bốn đỉnh đôi một kề nhau đủ để đặt ra giới hạn dưới bốn màu cho toàn bộ đồ thị.

Bài thực hành cũng làm rõ rằng tính đầy đủ của phép gán khác với tính hợp lệ. Một phép gán đủ mười màu vẫn có thể vi phạm các cạnh; ngược lại, một phép gán từng phần chưa vi phạm ràng buộc cũng chưa phải nghiệm. Do đó, việc kiểm tra lời giải CSP phải kết hợp tính đầy đủ, tính thuộc miền và tính nhất quán trên toàn bộ ràng buộc.

Nhìn chung, việc khảo sát và kiểm chứng mô hình ở Bài 1 tạo nền tảng để tiếp tục nghiên cứu các hàm kiểm tra phép gán, Backtracking, chiến lược chọn biến, Forward Checking và Min-Conflicts trong những bài thực hành tiếp theo. Các kết quả COUNT và thời gian của Backtracking ở chương này chỉ được sử dụng để xác nhận khả năng tìm nghiệm, chưa phải đánh giá hiệu suất thuật toán một cách hệ thống.

CHƯƠNG 2: KIỂM TRA TÍNH ĐẦY ĐỦ VÀ NHẤT QUÁN

2.1. Cơ sở lý thuyết về phép gán trong CSP

2.1.1. Phép gán rỗng và phép gán từng phần

Trong bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problem – CSP), phép gán (assignment) là thao tác lựa chọn một giá trị từ miền cho phép cho một hoặc nhiều biến của bài toán. Đối với bài toán tô màu đồ thị, mỗi biến tương ứng với một đỉnh, còn giá trị được gán chính là màu của đỉnh đó. Phép gán được biểu diễn dưới dạng từ điển Python, với khóa là tên đỉnh và giá trị là màu đã lựa chọn. Một đỉnh chưa xuất hiện trong từ điển được hiểu là chưa được gán màu.

Phép gán rỗng, ký hiệu A = {}, chưa chứa giá trị của bất kỳ biến nào. Phép gán từng phần chỉ xác định màu cho một tập con của các đỉnh. Chẳng hạn, A = {'0': 'red', '1': 'blue'} là phép gán từng phần trên đồ thị gồm 10 đỉnh. Việc phép gán chưa bao phủ toàn bộ tập biến không đồng nghĩa với việc phép gán đó sai; ở giai đoạn xây dựng nghiệm, một phép gán từng phần vẫn được xem là nhất quán khi chưa xuất hiện xung đột giữa những biến đã được gán.

Đối với phép gán rỗng, hàm kiểm tra nhất quán trong notebook trả về True vì chưa có ràng buộc nào có cả hai biến được gán để có thể bị vi phạm. Đây là tính nhất quán theo tập ràng buộc đã được kích hoạt, không phải khẳng định phép gán rỗng là một nghiệm hoàn chỉnh.

2.1.2. Phép gán đầy đủ

Phép gán đầy đủ (complete assignment) là phép gán trong đó mỗi biến thuộc tập X đều có một giá trị. Với đồ thị đang khảo sát, tập X gồm 10 đỉnh được đánh số từ 0 đến 9, do đó phép gán đầy đủ phải bao gồm đủ 10 tên đỉnh trong từ điển. Có thể mô tả điều kiện này như sau:

$$\text{Complete}(A) \Leftrightarrow \forall x \in X, x \in \text{keys}(A).$$

Điều kiện đầy đủ chỉ phản ánh số lượng biến đã được gán, không đánh giá các đỉnh có nhận màu hợp lệ hoặc có vi phạm ràng buộc khác màu hay không. Ví dụ, nếu gán toàn bộ 10 đỉnh bằng màu đỏ, phép gán vẫn đầy đủ, mặc dù tất cả 21 cạnh của đồ thị đều nối hai đỉnh có cùng màu. Vì vậy, khái niệm phép gán đầy đủ cần được phân biệt với khái niệm nghiệm CSP.

2.1.3. Điều kiện xác định nghiệm CSP hợp lệ

Một nghiệm hợp lệ phải đồng thời thỏa mãn ba yêu cầu: tất cả biến đều được gán, giá trị được gán thuộc miền giá trị tương ứng và không có ràng buộc nào bị vi phạm. Với phép gán A và CSP = (X, D, C), điều kiện nghiệm được biểu diễn:

$$\text{Solution}(A) = \text{Complete}(A) \wedge \text{Consistent}(A) \wedge \text{DomainValid}(A).$$

Trong bài toán tô màu đồ thị, ràng buộc của mỗi cạnh (i, j) yêu cầu color(i) ≠ color(j). Khi một phép gán đáp ứng cả ba điều kiện trên, đồ thị được tô màu đầy đủ, không có hai đỉnh kề nhau trùng màu và không sử dụng màu ngoài tập được phép. Nội dung thực hành sẽ kiểm chứng từng điều kiện bằng các ca kiểm thử riêng biệt để tránh đồng nhất tính đầy đủ với tính hợp lệ.

2.2. Cấu trúc dữ liệu biểu diễn CSP

2.2.1. Thành phần variables

Notebook lưu bài toán CSP trong một từ điển có ba khóa: variables, domains và constraints. Thành phần csp['variables'] là danh sách tên các biến, tương ứng với các đỉnh của đồ thị Delaunay được kế thừa từ Bài 1. Với n = 10, danh sách biến có dạng ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']. Các tên đỉnh được lưu dưới dạng chuỗi để sử dụng làm khóa trong các từ điển phép gán và miền giá trị.

Số lượng biến là cơ sở xác định tính đầy đủ. Chương trình kiểm tra lần lượt từng biến trong danh sách này để xác nhận biến đã có trong assignment hay chưa. Bài thực hành giữ nguyên tập biến khi thay đổi miền ba màu và bốn màu, nhờ đó các ca kiểm thử được đối chiếu trên cùng cấu trúc đồ thị.

2.2.2. Thành phần domains

Thành phần csp['domains'] là từ điển ánh xạ mỗi tên biến tới danh sách những màu được phép sử dụng. Trong cấu hình bốn màu, miền của mỗi đỉnh là D_i = {red, blue, green, orange}. Đối với cấu hình ba màu, miền được giới hạn còn D_i = {red, blue, green}. Các cấu hình này được lưu riêng ở csp4 và csp3, trong khi tập biến và tập ràng buộc không thay đổi.

Việc kiểm tra miền có vai trò độc lập với việc kiểm tra quan hệ khác màu. Chẳng hạn, màu purple không thuộc miền bốn màu, nhưng vẫn có thể khác màu tất cả các đỉnh kề. Nếu chỉ kiểm tra sự khác nhau giữa các đầu cạnh, chương trình có thể bỏ sót lỗi sử dụng màu không được phép. Đây là trường hợp được kiểm thử cụ thể ở TC07.

2.2.3. Thành phần constraints

Thành phần csp['constraints'] lưu các cặp đỉnh có cạnh nối trực tiếp; mỗi cặp biểu diễn một ràng buộc nhị phân yêu cầu hai đỉnh mang màu khác nhau. Với đồ thị Delaunay ở Bài 1, chương trình ghi nhận 21 cạnh không hướng, tương ứng 21 ràng buộc. Khi tạo tập ràng buộc, điều kiện i < j được áp dụng để mỗi cạnh chỉ được lưu một lần, tránh lặp lại cùng quan hệ kề theo thứ tự đảo ngược.

Khi kiểm tra một phép gán từng phần, chỉ những ràng buộc có cả hai biến đã được gán mới có thể được đánh giá trực tiếp. Vì thế, các cạnh nối tới đỉnh chưa được tô màu chưa được xem là vi phạm; kết luận nhất quán ở thời điểm đó chỉ phản ánh những ràng buộc đã có đủ thông tin kiểm tra.

2.3. Các hàm kiểm tra phép gán

2.3.1. Hàm complete() kiểm tra tính đầy đủ

Trong notebook gốc, hàm complete(assignment, csp) duyệt toàn bộ danh sách csp['variables'], kiểm tra mỗi biến có xuất hiện trong tập khóa của assignment hay không, rồi kết hợp các kết quả bằng hàm all(). Nếu tất cả biến đều xuất hiện, hàm trả về True; ngược lại, hàm trả về False. Ví dụ complete({}, csp) = False, còn phép gán đủ cả 10 đỉnh đều màu đỏ có complete() = True.

Hàm complete() không truy cập đến csp['constraints'] hoặc so sánh giá trị màu giữa các đỉnh, do đó không thể được sử dụng độc lập để xác nhận nghiệm hợp lệ. Đây là hành vi đúng theo mục đích thiết kế của hàm: kiểm tra sự hiện diện của giá trị gán đối với toàn bộ biến.

2.3.2. Hàm consistent() kiểm tra tính nhất quán

Hàm consistent(assignment, csp) duyệt các cặp biến trong csp['constraints']. Với mỗi cạnh (u, v), hàm chỉ thực hiện so sánh khi cả u và v đã có trong assignment. Nếu phát hiện assignment[u] bằng assignment[v], hàm trả về False ngay tại ràng buộc vi phạm đầu tiên. Nếu duyệt hết mà không phát hiện xung đột, hàm trả về True.

Theo định nghĩa này, phép gán rỗng và phép gán chỉ một đỉnh luôn nhất quán đối với những ràng buộc khác màu của đồ thị. Ngược lại, khi hai đỉnh kề nhau cùng nhận màu red, kết quả consistent() = False. Hàm gốc chỉ kiểm tra ràng buộc trên cạnh, chưa kiểm tra giá trị màu có thuộc miền đã khai báo. Vì vậy, consistent() = True chưa đủ để khẳng định nghiệm hợp lệ.

2.3.3. Kiểm tra giá trị thuộc miền và xác định nghiệm hợp lệ

Để kiểm tra đầy đủ các điều kiện, phần thực hành bổ sung hàm domain_valid(assignment, model) nhằm xác nhận mọi khóa của assignment thuộc tập variables và mọi giá trị gán thuộc miền domains tương ứng. Hàm violating_edges() được sử dụng để thống kê những cạnh có cả hai đầu đã được gán cùng màu, qua đó hỗ trợ giải thích trực quan kết quả kiểm thử.

Hàm is_csp_solution(assignment, model) kết hợp kết quả của complete(), consistent() và domain_valid(). Cách xây dựng này giữ nguyên định nghĩa của hai hàm gốc trong notebook, đồng thời bổ sung kiểm tra miền riêng biệt. Ví dụ, khi thay một màu hợp lệ bằng purple trong phép tô bốn màu, trạng thái vẫn đầy đủ và không tạo cạnh trùng màu, nhưng domain_valid() = False, kéo theo is_csp_solution() = False.

2.4. Xây dựng và thực hiện các ca kiểm thử

2.4.1. Kiểm thử phép gán rỗng và từng phần

Các ca TC01, TC02 và TC03 lần lượt khảo sát phép gán rỗng, phép gán một đỉnh và phép gán hai đỉnh kề nhau bằng hai màu khác nhau. Cả ba trường hợp đều có complete() = False do chưa tô đủ 10 đỉnh, nhưng consistent() = True vì không xuất hiện cạnh có hai đầu cùng màu. Những trường hợp này chứng minh rằng nhất quán là một tính chất có thể được duy trì ngay từ giai đoạn xây dựng phép gán từng phần.

Với TC03, hai đỉnh được lựa chọn là một cặp thật sự xuất hiện trong constraints của đồ thị cố định, sau đó gán hai màu red và blue. Nhờ đó, phép thử xác nhận trực tiếp hành vi kiểm tra ràng buộc nhị phân thay vì chỉ kiểm tra hai đỉnh không liên quan.

2.4.2. Kiểm thử phép gán đầy đủ nhưng vi phạm ràng buộc

TC04 sử dụng hai đỉnh kề nhau cùng màu red. Khi cả hai đầu cạnh đã được gán giá trị giống nhau, consistent() trả về False dù phép gán vẫn chưa đầy đủ; số cạnh vi phạm bằng 1. TC05 mở rộng thành phép gán tất cả 10 đỉnh cùng màu red. Trong trường hợp này complete() = True, nhưng consistent() = False vì toàn bộ 21 cạnh đều vi phạm ràng buộc khác màu.

Kết quả TC05 thể hiện rõ rằng việc gán đủ biến chỉ là điều kiện cần để xác định nghiệm. Không thể suy luận tính nhất quán từ số lượng biến đã được gán, vì sự hợp lệ còn phụ thuộc quan hệ giữa giá trị tại các đỉnh kề nhau.

2.4.3. Kiểm thử phép gán đầy đủ và hợp lệ

TC06 sử dụng phép tô bốn màu hợp lệ đã được tìm và kiểm chứng trong Bài 1. Cả 10 đỉnh đều được gán màu thuộc miền, đồng thời tất cả 21 cạnh đều thỏa ràng buộc khác

màu. Vì vậy, complete(), consistent(), domain_valid() và is_csp_solution() đều trả về True; số cạnh vi phạm bằng 0.

Đối với TC08, chương trình chuyển các màu orange trong một phép tô bốn màu thành red rồi kiểm tra kết quả theo miền ba màu. Phép gán sau chuyển đổi vẫn đầy đủ và các màu đều nằm trong miền mới, nhưng có 3 cạnh vi phạm nên không phải nghiệm. Kết quả phù hợp với nhận xét của Bài 1 rằng đồ thị cố định chứa đồ thị con đầy đủ K_4 gồm các đỉnh 2, 3, 4 và 9, vì vậy không thể tô hợp lệ bằng ba màu.

2.4.4. Kiểm thử phép gán sử dụng giá trị ngoài miền

TC07 được thiết kế để kiểm tra giới hạn của hàm consistent() gốc. Bắt đầu từ nghiệm tô bốn màu hợp lệ, chương trình thay màu của một đỉnh bằng purple, một giá trị không thuộc miền {red, blue, green, orange}. Do purple khác với mọi màu hợp lệ đang sử dụng tại các đỉnh kề, không xuất hiện cạnh vi phạm và consistent() vẫn trả về True. Phép gán cũng đầy đủ vì toàn bộ 10 biến đều có giá trị.

Tuy nhiên, domain_valid() = False và is_csp_solution() = False. Ca kiểm thử này cho thấy ngay cả khi complete() và consistent() đều trả về True, vẫn cần kiểm tra thêm miền giá trị trước khi kết luận một phép gán là nghiệm CSP.

2.5. Kết quả thực nghiệm và phân tích

2.5.1. Bảng tổng hợp các ca kiểm thử

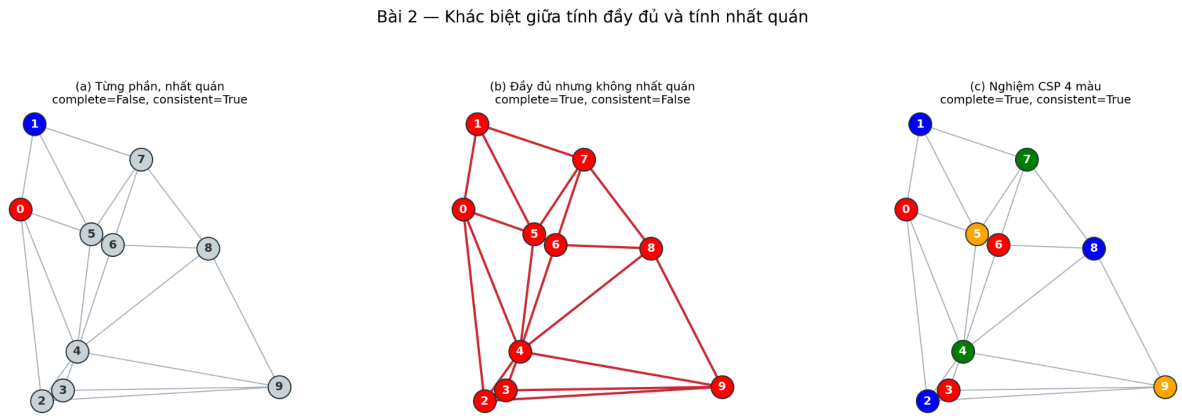
Bài thực hành được thực hiện trên đồ thị Delaunay cố định có 10 đỉnh và 21 cạnh, với cấu hình seed = 1111. Tám ca kiểm thử được xây dựng cùng kết quả mong đợi cho complete(), consistent() và nghiệm CSP. Dữ liệu xuất ra tệp Bai2_CSP_Completeness_Consistency_Tests.csv; cả 8 ca đều cho kết quả PASS khi đối chiếu giá trị thực tế với kết quả kỳ vọng.

Bảng 2.1. Tổng hợp kết quả kiểm tra tính đầy đủ, nhất quán và nghiệm CSP

Ca	Trường hợp kiểm thử	Đầy đủ	Nhất quán	Trong miền	Nghiệm	Cạnh vi phạm
TC01	Phép gán rỗng	False	True	True	False	0
TC02	Một đỉnh được gán	False	True	True	False	0
TC03	Hai đỉnh kề khác màu	False	True	True	False	0
TC04	Hai đỉnh kề cùng màu	False	False	True	False	1
TC05	Đầy đủ nhưng tất cả màu đỏ	True	False	True	False	21
TC06	Đầy đủ và hợp lệ (4 màu)	True	True	True	True	0
TC07	Đầy đủ, màu ngoài miền	True	True	False	False	0
TC08	Đầy đủ 3 màu nhưng vi phạm cạnh	True	False	True	False	3

Các kết quả cho thấy những giá trị True hoặc False của từng phép kiểm tra có thể kết hợp theo nhiều cách khác nhau. Trong tám ca khảo sát, chỉ TC06 đồng thời đáp ứng ba điều kiện và được xác định là nghiệm CSP hợp lệ.

2.5.2. So sánh kết quả complete() và consistent()



Hình 2.1. So sánh phép gán từng phần nhất quán, phép gán đầy đủ nhưng không nhất quán và nghiệm CSP bốn màu trên cùng đồ thị Delaunay.

Hình 2.1 minh họa ba trạng thái tiêu biểu. Ở hình (a), chỉ hai đỉnh được tô màu khác nhau nên phép gán vẫn thiếu các biến còn lại, nhưng chưa có xung đột. Ở hình (b), tất cả đỉnh

đều được tô đỏ, thỏa điều kiện đầy đủ nhưng tạo xung đột trên toàn bộ 21 cạnh, được biểu diễn bằng các đường màu đỏ. Ở hình (c), mỗi đỉnh nhận một trong bốn màu cho phép và hai đầu của mọi cạnh đều khác màu, do đó phép gán vừa đầy đủ vừa nhất quán.

Đáng chú ý, ba hình sử dụng cùng cấu trúc đồ thị, vì vậy sự khác biệt về tính hợp lệ đến từ cách gán màu chứ không phải do thay đổi tập đỉnh hoặc tập cạnh. Nhờ cách trực quan hóa này, kết quả của các hàm kiểm tra trở nên dễ đối chiếu với trạng thái thực tế của đồ thị.

2.5.3. Phân tích sự khác biệt giữa tính đầy đủ và tính nhất quán

Qua các trường hợp khảo sát, complete() và consistent() giải quyết hai câu hỏi độc lập. Hàm complete() xác nhận phạm vi gán biến, còn consistent() đánh giá sự tương thích giữa các giá trị đã gán theo ràng buộc cạnh. Vì thế, một phép gán có thể nhất quán nhưng chưa đầy đủ như TC01–TC03; ngược lại, một phép gán có thể đầy đủ nhưng không nhất quán như TC05 và TC08.

Trường hợp TC07 tiếp tục cho thấy tính đầy đủ kết hợp với tính nhất quán trên cạnh vẫn chưa phải điều kiện đủ khi mã nguồn không tích hợp kiểm tra miền trong consistent(). Do đó, việc phân tách ba tiêu chí trong hàm is_csp_solution() giúp hệ thống kiểm tra kết quả một cách chính xác hơn. Đồng thời, đối với bài toán có miền ba màu, việc trả về kết quả không hợp lệ không phải lỗi của hàm kiểm tra; đồ thị cụ thể có K_4 nên không tồn tại nghiệm ba màu, bất kể thứ tự gán màu.

2.6. Nhận xét

Bài thực hành đã làm rõ sự khác biệt giữa phép gán từng phần, phép gán đầy đủ và nghiệm của bài toán thỏa mãn ràng buộc. Thông qua cấu trúc dữ liệu variables, domains và constraints trong notebook, chương trình có thể kiểm tra riêng số biến đã được gán, các ràng buộc khác màu và sự phù hợp của giá trị với miền cho phép. Cả tám ca kiểm thử đều

đạt kết quả mong đợi, trong đó TC05 và TC07 là hai ví dụ thể hiện rõ giới hạn của việc chỉ xét một hoặc hai điều kiện.

Một kết quả quan trọng là hàm consistent() của notebook gốc chỉ kiểm tra các cạnh có đủ hai đầu được gán; hàm không tự kiểm tra giá trị ngoài miền. Việc bổ sung domain_valid() và kết hợp ba điều kiện trong is_csp_solution() là cần thiết để đánh giá chính xác nghiệm hoàn chỉnh mà vẫn giữ nguyên chức năng của các hàm gốc. Ngoài ra, hình ảnh trực quan xác nhận cùng một cấu trúc đồ thị có thể xuất hiện các phép gán với trạng thái đầy đủ và nhất quán khác nhau.

Kết quả này là nền tảng cho những bài thực hành tiếp theo về Backtracking Search, MRV, LCV và Forward Checking. Trong các thuật toán đó, việc kiểm tra nhất quán sớm giúp loại bỏ những phép gán không khả thi ngay trong quá trình tìm kiếm, còn kiểm tra đầy đủ và miền giá trị bảo đảm nghiệm được trả về thật sự đáp ứng mô hình CSP đã xây dựng.

CHƯƠNG 3: BACKTRACKING SEARCH VÀ THỨ TỰ CHỌN BIẾN

3.1. Cơ sở lý thuyết thuật toán Backtracking Search

3.1.1. Nguyên lý tìm kiếm quay lui

Backtracking Search là thuật toán tìm kiếm có hệ thống nhằm xây dựng từng bước phép gán thỏa mãn ràng buộc cho bài toán CSP. Đối với tô màu đồ thị, mỗi biến biểu diễn một đỉnh, miền giá trị là tập màu cho phép và mỗi cạnh yêu cầu hai đầu mút nhận màu khác nhau. Thuật toán bắt đầu với phép gán rỗng, chọn một biến chưa được gán, lần lượt thử giá trị trong miền rồi kiểm tra tính nhất quán. Nếu phép gán đầy đủ, thuật toán trả về nghiệm; nếu không còn giá trị phù hợp tại một nhánh, thuật toán quay lui.

3.1.2. Cơ chế mở rộng và quay lui khi vi phạm ràng buộc

Mỗi lời gọi đệ quy tương ứng với một phép gán từng phần đã qua bước kiểm tra nhất quán. Khi thử màu khiến một đỉnh có màu giống đỉnh kề đã gán, chương trình bác bỏ phép gán thử ngay mà không đệ quy sâu hơn. Nếu đã thử hết màu mà chưa tìm thấy nghiệm, biến vừa gán được xóa khỏi phép gán hiện hành để xét nhánh khác. Cơ chế này tránh duyệt tiếp những nhánh chắc chắn vi phạm ràng buộc.

3.1.3. Vai trò của thứ tự chọn biến và giá trị

Thứ tự chọn biến quyết định ràng buộc nào được xử lý sớm; thứ tự chọn giá trị quyết định cách bảo toàn lựa chọn cho các biến chưa gán. MRV ưu tiên đỉnh có ít màu hợp lệ còn lại, còn LCV ưu tiên màu loại trừ ít khả năng lựa chọn của các đỉnh kề chưa gán. Những

heuristic này không thay đổi tập nghiệm mà thay đổi tiến trình khám phá không gian tìm kiếm.

3.2. Thuật toán Backtracking cơ bản

3.2.1. Triển khai hàm backtrack_search()

Notebook giữ nguyên hàm backtrack_search(csp) gốc. Hàm khởi tạo COUNT bằng 0, gọi backtrack({}, csp) và trả về phép gán tìm được hoặc None. Trong hàm backtrack(), mỗi lời gọi tăng COUNT một đơn vị; thuật toán kiểm tra complete(), chọn biến chưa gán, thử các màu thuộc domains của biến, dùng consistent() để quyết định có thể tiếp tục đệ quy hay không. Nếu một nhánh thất bại, phép gán thử được xóa trước khi thử màu kế tiếp.

3.2.2. Cơ chế lựa chọn biến chưa được gán

Ở phiên bản cơ bản, select_unassigned_var() lấy biến đầu tiên chưa xuất hiện trong assignment theo thứ tự csp["variables"]. Các giá trị được thử theo đúng thứ tự ban đầu của csp["domains"][var]. Đây là chiến lược Baseline để đối chứng với hai phiên bản có heuristic.

3.2.3. Theo dõi COUNT và trạng thái tìm kiếm

Trong thí nghiệm, COUNT đếm số lời gọi đệ quy, bao gồm cả lời gọi ở nút gốc, không phải tổng số màu từng được thử. Chỉ số attempts đếm các thao tác thử gán màu, kể cả phép gán không nhất quán. Ngoài hai chỉ số này, notebook ghi số lần quay lui, độ sâu lớn nhất và thời gian chạy để đánh giá chi phí tìm kiếm từ nhiều góc độ. Vì định nghĩa khác nhau nên COUNT không được đồng nhất với attempts.

3.3. Các chiến lược cải thiện thứ tự tìm kiếm

3.3.1. Minimum Remaining Values (MRV)

Với một biến v chưa gán và phép gán hiện tại A, miền khả dụng D_valid(v,A) gồm các màu thuộc D(v) mà chưa được một đỉnh kề đã gán sử dụng. MRV chọn biến v làm cho |D_valid(v,A)| nhỏ nhất. Nếu nhiều biến hòa nhau, notebook giữ thứ tự biến gốc. Cách chọn "biến khó nhất trước" giúp bộc lộ sớm nhánh không thể mở rộng, nhưng phải tính lại số màu khả dụng của các biến chưa gán.

3.3.2. Least Constraining Value (LCV)

Sau khi xác định biến v, LCV xếp các màu đang hợp lệ theo số lựa chọn mà chúng sẽ loại bỏ tại những đỉnh kề chưa gán. Màu gây mất ít lựa chọn nhất được ưu tiên. Trong hiện thực notebook, order_domain_lcv() chỉ xét các màu đã hợp lệ với phép gán hiện tại, tức bỏ qua màu chắc chắn gây xung đột trước khi sắp xếp. Do đó chỉ số attempts chịu tác động đồng thời của việc lọc giá trị và thứ tự LCV.

3.3.3. Kết hợp MRV và LCV trong Backtracking Search

Phiên bản kết hợp sử dụng MRV để chọn biến, sau đó LCV để chọn giá trị thử. Các hàm backtrack_search_ordered(), select_unassigned_var_mrv() và order_domain_lcv() được triển khai độc lập, không ghi đè các hàm gốc. Việc tính miền hợp lệ chỉ dựa trên các đỉnh kề đã gán; chưa có Forward Checking hay lan truyền miền, vốn thuộc nội dung của Bài 4.

3.4. Thiết lập và kết quả thực nghiệm

3.4.1. Điều kiện thực nghiệm trên cùng đồ thị

Dữ liệu là cùng đồ thị Delaunay từ Bài 1 với n = 10 đỉnh, 21 cạnh, seed = 1111. Thực nghiệm khảo sát hai miền gồm bốn màu và ba màu, đồng thời so sánh Baseline, MRV, MRV + LCV. Mỗi cặp chiến lược–miền được đo thời gian 20 lần (tổng cộng 120 lượt đo), báo cáo giá trị trung vị và trung bình. Thời gian được đo bằng perf_counter() đối với hàm tìm kiếm, không bao gồm xuất bảng và hình.

3.4.2. Kết quả Backtracking với miền bốn màu

Ở miền bốn màu, cả ba thuật toán đều tìm được phép gán hợp lệ cho toàn bộ 10 đỉnh mà không cần quay lui. COUNT đều bằng 11 và độ sâu lớn nhất đều bằng 10. Số lần thử màu lần lượt là 23 đối với Baseline, 24 đối với MRV và 10 đối với MRV + LCV. Kết quả chứng tỏ một chiến lược có thể giảm thử gán mà không giảm số nút đệ quy.

3.4.3. Kết quả Backtracking với miền ba màu

Cả ba thuật toán đều trả về None khi sử dụng ba màu. Đây là kết quả đúng về mặt toán học vì đồ thị chứa K_4 gồm bốn đỉnh 2, 3, 4, 9, trong đó mọi cặp đỉnh đều kề nhau và phải mang màu khác nhau. Baseline kiểm tra 70 nút, MRV và MRV + LCV kiểm tra 34 nút. Số lần thử màu tương ứng là 210, 102 và 33. Với bài toán vô nghiệm, heuristic giúp phát hiện mâu thuẫn sớm hơn, mặc dù vẫn phải tìm kiếm đủ để kết luận.

3.4.4. Bảng so sánh COUNT, số lần thử màu và thời gian chạy

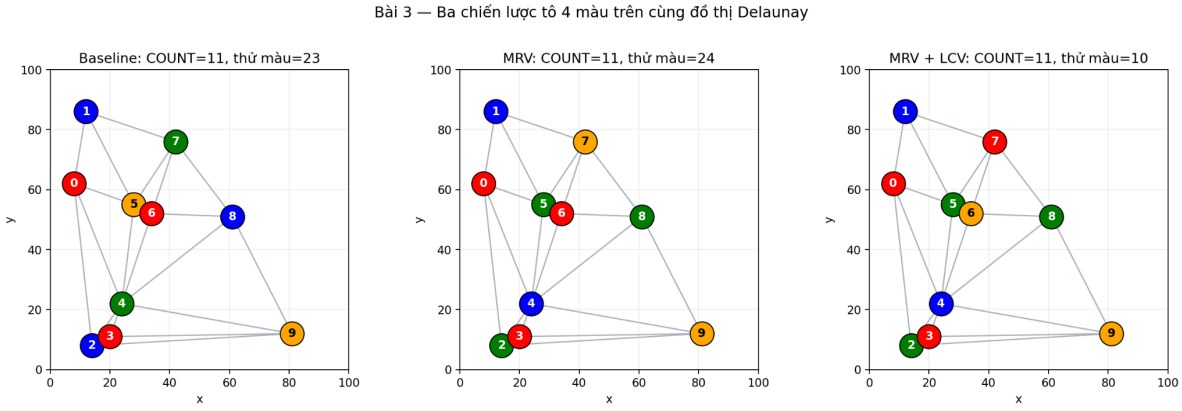
Bảng 3.1. Tổng hợp kết quả tìm kiếm và trung vị thời gian chạy (ms) của 20 lượt đo trên mỗi cấu hình.

Miền	Chiến lược	Nghiệm	COUNT	Attempts	Quay lui	Độ sâu	Trung vị (ms)
4 màu	Baseline	Có	11	23	0	10	0.0712

4 màu	MRV	Có	11	24	0	10	0.0887
4 màu	MRV + LCV	Có	11	10	0	10	0.1132
3 màu	Baseline	Không	70	210	70	7	0.4831
3 màu	MRV	Không	34	102	34	6	0.2984
3 màu	MRV + LCV	Không	34	33	34	6	0.4171

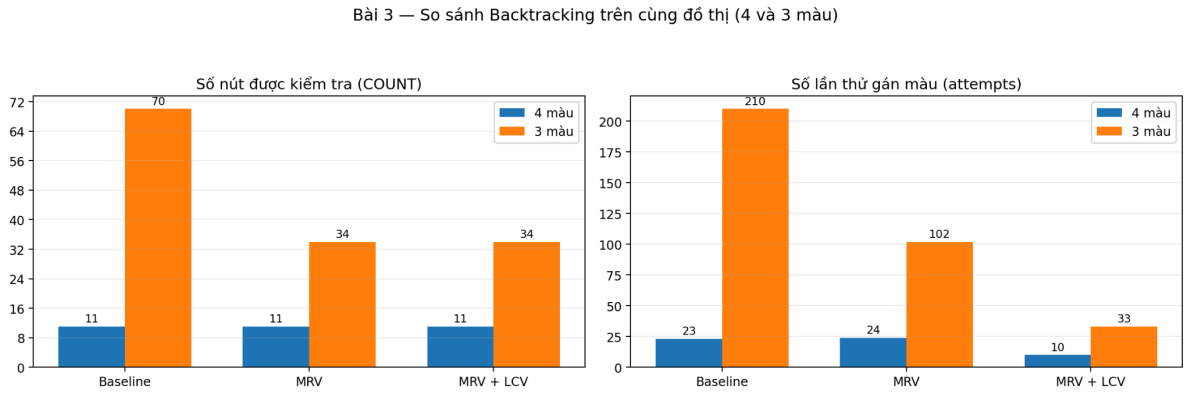
Với ba màu, MRV giảm COUNT từ 70 xuống 34 (khoảng 51,4%). MRV + LCV giảm attempts từ 210 xuống 33 (khoảng 84,3%). Tuy nhiên, ở cấu hình bốn màu, trung vị thời gian của MRV + LCV (0,1132 ms) vẫn cao hơn Baseline (0,0712 ms), do việc tính heuristic cũng có chi phí.

3.4.5. Trực quan hóa nghiệm tô màu và hiệu suất tìm kiếm



Hình 3.1. Ba nghiệm tô bốn màu trên cùng đồ thị Delaunay theo các chiến lược Baseline, MRV và MRV + LCV.

Trong Hình 3.1, các chiến lược cho những phép tô màu khác nhau nhưng đều thỏa mãn ràng buộc: mỗi cạnh nối hai đỉnh có màu khác nhau. Sự khác nhau của nghiệm không phản ánh trực tiếp hiệu quả; để so sánh công sức cần quan sát COUNT và attempts.



Hình 3.2. So sánh số nút đệ quy (COUNT) và số lần thử màu (attempts) với miền bốn màu và ba màu.

3.5. Đánh giá hiệu quả của MRV và LCV

3.5.1. Ảnh hưởng đến số nút được kiểm tra

Trong trường hợp bốn màu, cả ba cách chọn thứ tự đều đi thẳng tới nghiệm nên COUNT bằng 11. Với ba màu, MRV phát hiện các nhánh thiếu giá trị khả dụng sớm hơn và giảm hơn một nửa số lời gọi đệ quy. Bổ sung LCV không làm COUNT giảm thêm trên đồ thị thử nghiệm này; lợi ích phụ thuộc cấu trúc cạnh và thứ tự gán hiện có.

3.5.2. Ảnh hưởng đến số lần thử gán giá trị

Khi bốn màu, attempts của MRV (24) nhỉnh hơn Baseline (23), nhưng MRV + LCV chỉ cần 10 lần thử. Khi ba màu, attempts lần lượt là 210, 102 và 33. Cần lưu ý rằng LCV trong notebook chỉ duyệt màu hiện hợp lệ, trong khi Baseline và MRV còn thử cả các màu bị consistent() bác bỏ. Do đó phép so sánh không cô lập riêng tác động của việc sắp xếp LCV; muốn đánh giá LCV thuần túy phải thống nhất bước lọc màu ở tất cả chiến lược.

3.5.3. Sự đánh đổi giữa chi phí heuristic và thời gian thực thi

Giảm số nút hoặc số phép gán thử không bảo đảm giảm thời gian đo được. Việc tính số màu còn hợp lệ cho mỗi biến, kiểm tra ảnh hưởng của mỗi màu tới các đỉnh kề và sắp xếp theo LCV đều phát sinh thao tác bổ sung. Theo CSV, ở miền ba màu trung vị thời gian Baseline là 0,4831 ms, MRV 0,2984 ms và MRV + LCV 0,4171 ms; MRV nhanh nhất trong bộ đo này mặc dù MRV + LCV có ít attempts hơn. Các số đo đều dưới một mili giây và chịu ảnh hưởng môi trường thực thi, vì vậy không suy rộng thứ hạng này cho mọi đồ thị hoặc máy tính.

3.6. Nhận xét

Qua thực nghiệm, Backtracking Search giải quyết đúng bài toán tô màu đồ thị theo mô hình CSP. Trường hợp bốn màu cho thấy thuật toán có thể xây dựng một nghiệm đầy đủ, nhất quán và thuộc miền; trường hợp ba màu minh họa việc phát hiện vô nghiệm do tồn tại đồ thị con đầy đủ K_4. MRV giúp giảm đáng kể số nút cần kiểm tra trong trường hợp khó hơn; LCV kết hợp lọc màu giúp giảm mạnh số lần thử gán nhưng không nhất thiết rút ngắn thời gian.

Bài thực hành cho thấy cần đánh giá thuật toán bằng nhiều chỉ số độc lập: nghiệm, COUNT, attempts, backtracks, độ sâu và thời gian. Để có kết luận tổng quát hơn, có thể mở rộng khảo sát sang nhiều đồ thị Delaunay với số đỉnh và mật độ cạnh khác nhau. Forward Checking và Min-Conflicts không thuộc phạm vi triển khai của chương này mà được tiếp tục ở Chương 4.

CHƯƠNG 4: FORWARD CHECKING VÀ MIN-CONFLICTS

4.1. Giới thiệu các phương pháp cải thiện tìm kiếm CSP

4.1.1. Hạn chế của Backtracking cơ bản

Trong bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problem – CSP), Backtracking xây dựng nghiệm bằng cách lần lượt chọn biến và gán một giá trị thuộc miền cho biến đó. Nếu phép gán mới vi phạm ràng buộc, thuật toán thử giá trị khác hoặc quay lui về trạng thái trước. Đối với bài toán tô màu đồ thị, mỗi biến biểu diễn một đỉnh, miền giá trị là danh sách màu và ràng buộc yêu cầu hai đỉnh nối bằng một cạnh phải nhận hai màu khác nhau.

Backtracking có ưu điểm là có thể tìm nghiệm hoặc chứng minh vô nghiệm sau khi đã duyệt hết không gian tìm kiếm hữu hạn. Tuy nhiên, thuật toán có thể phải thử nhiều phép gán trước khi phát hiện một nhánh không thể hoàn thành. Ngay cả khi kết hợp Minimum Remaining Values (MRV) để ưu tiên biến có ít màu hợp lệ và Least Constraining Value (LCV) để sắp thứ tự thử màu, các ràng buộc của những biến chưa được gán vẫn chưa được lan truyền đầy đủ. Điều này làm tăng số lời gọi đệ quy, số lần thử màu và chi phí thực thi khi độ phức tạp của đồ thị tăng lên.

Trên đồ thị Delaunay cố định gồm 10 đỉnh và 21 cạnh từ các bài trước, Backtracking MRV + LCV cần COUNT = 34 để kết luận bài toán ba màu vô nghiệm. Kết quả này đặt ra nhu cầu loại bỏ sớm những lựa chọn chắc chắn không khả thi trước khi đi sâu vào cây tìm kiếm.

4.1.2. Phương pháp suy luận ràng buộc và tìm kiếm cục bộ

Bài thực hành khảo sát hai hướng cải thiện có nguyên lý khác nhau. Forward Checking (FC) kết hợp với Backtracking để suy luận trực tiếp trên miền của những biến chưa gán, qua đó cắt sớm các nhánh dẫn đến miền rỗng. Ngược lại, Min-Conflicts bắt đầu từ một phép gán đầy đủ, dù có thể chưa hợp lệ, rồi thay đổi màu tại các đỉnh nhằm giảm số cạnh vi phạm ràng buộc.

Forward Checking vẫn khai thác tính hệ thống của tìm kiếm quay lui; nếu thuật toán duyệt hết các nhánh hợp lệ mà không gặp giới hạn thì có thể kết luận vô nghiệm. Min-Conflicts là tìm kiếm cục bộ có yếu tố ngẫu nhiên, phù hợp để tìm nhanh nghiệm trong một số trường hợp, nhưng dừng tìm không đồng nghĩa với chứng minh vô nghiệm. Do đó, việc đánh giá cần tách riêng chất lượng nghiệm, số lần gọi đệ quy, số bước đổi màu và thời gian chạy.

4.2. Thuật toán Forward Checking

4.2.1. Nguyên lý loại bỏ giá trị không hợp lệ khỏi miền

Giả sử đồ thị $G = (V, E)$, trong đó $D(v)$ là tập màu còn lại của đỉnh v và A là phép gán một phần. Sau khi gán $A(v) = c$, mọi đỉnh u kề với v nhưng chưa được gán không thể nhận màu c. Forward Checking cập nhật miền còn lại theo quy tắc:

$D_{mới}(u) = D_{cũ}(u) \setminus \{c\}$, với $(u, v) \in E$ và u chưa được gán.

Nếu $D_{mới}(u) = \varnothing$, thuật toán phát hiện ngay nhánh hiện tại không thể mở rộng thành nghiệm và tiến hành quay lui. Khác với việc chỉ kiểm tra tính nhất quán của các đỉnh đã gán, Forward Checking sử dụng ràng buộc để dự báo tính khả thi của các lựa chọn ở các đỉnh chưa gán.

4.2.2. Cập nhật miền các đỉnh láng giềng sau khi gán màu

Trong notebook, hàm backtracking_forward_checking() tạo bản sao riêng remaining từ các miền màu ban đầu. Với mỗi phép gán A(v) = c, chương trình duyệt danh sách đỉnh kề adj[v], xóa c khỏi miền của các đỉnh chưa được gán và tăng bộ đếm pruned_values. Nếu phát hiện miền trống, cờ contradiction được kích hoạt, ngăn không cho thuật toán tiếp tục đi sâu vào nhánh đó.

Chỉ số pruned_values biểu thị tổng số lượt loại bỏ giá trị trong quá trình tìm kiếm, bao gồm cả các giá trị có thể bị loại thêm lần nữa sau khi hoàn tác và thử một nhánh khác. Đây không phải số màu bị mất vĩnh viễn. Mô hình CSP đầu vào luôn được giữ nguyên.

4.2.3. Cơ chế khôi phục miền khi quay lui

Mỗi lần xóa màu, chương trình lưu bộ ba (đỉnh láng giềng, vị trí ban đầu, màu đã xóa) vào danh sách removed. Sau khi kết thúc nhánh, tất cả thay đổi được hoàn tác theo thứ tự ngược lại, kể cả khi thuật toán đã phát hiện mâu thuẫn hoặc đạt giới hạn tìm kiếm.

Cơ chế này bảo đảm mỗi nhánh Backtracking bắt đầu từ trạng thái miền chính xác, không bị ảnh hưởng bởi những phép gán đã thất bại. Notebook kiểm tra hai bất biến domains_restored và việc phép gán tạm đã được xóa hoàn toàn. Các kiểm thử bổ sung sử dụng đồ thị tam giác với hai màu để xác nhận thuật toán phát hiện vô nghiệm và khôi phục đầy đủ miền sau khi quay lui.

4.2.4. Kết hợp Forward Checking với MRV và LCV

Biến chưa gán được lựa chọn theo MRV, tức ưu tiên đỉnh có ít màu còn lại nhất trong remaining. Khi các đỉnh có cùng số màu, chương trình sử dụng thứ tự biến ban đầu để phá hòa. Sau đó, LCV sắp xếp các màu ứng viên theo số miền láng giềng chưa gán sẽ bị ảnh hưởng; màu loại bỏ ít lựa chọn hơn được xét trước.

Việc phối hợp MRV, LCV và Forward Checking tạo thành một quy trình: chọn biến khó nhất, thử màu ít gây hạn chế nhất, lan truyền ràng buộc và quay lui khi cần thiết. Chương trình sử dụng COUNT để ghi số lời gọi đệ quy, attempts để ghi số lần thử gán màu, backtracks để ghi số lần quay lui và pruned_values để ghi công sức cắt miền. COUNT bao gồm cả lời gọi tại trạng thái gán đầy đủ.

4.3. Thuật toán Min-Conflicts

4.3.1. Nguyên lý tìm kiếm cục bộ trong CSP

Min-Conflicts chuyển việc giải CSP thành bài toán giảm số ràng buộc bị vi phạm. Đối với tô màu đồ thị, hàm mục tiêu là:

$f(A) = \Sigma[(u,v)\in E]\ I(A(u) = A(v))$,

trong đó I(P) bằng 1 khi mệnh đề P đúng và bằng 0 khi P sai. f(A) là số cạnh có hai đầu cùng màu; nghiệm hợp lệ phải thỏa mãn f(A) = 0. Thuật toán không xây dựng phép gán từng phần như Backtracking mà cải thiện trực tiếp một phép gán đầy đủ.

4.3.2. Khởi tạo phép gán đầy đủ

Hàm min_conflicts_strict() gán ngẫu nhiên một màu thuộc miền cho từng đỉnh bằng bộ sinh số giả ngẫu nhiên với seed cố định. Phép gán ban đầu luôn đầy đủ và mỗi màu đều thuộc miền, nhưng có thể có nhiều cạnh xung đột. Chương trình ghi nhận số xung đột hiện tại, đồng thời lưu best_assignment và best_conflicts để không đánh mất cấu hình tốt nhất đã gặp.

4.3.3. Lựa chọn biến đang vi phạm ràng buộc

Ở mỗi vòng lặp, thuật toán xác định những đỉnh nằm trên ít nhất một cạnh có hai đầu cùng màu, xáo trộn thứ tự các đỉnh này và lần lượt kiểm tra khả năng đổi màu. Chỉ các đỉnh đang tham gia xung đột mới được xem xét, nhờ đó tránh thử các thay đổi không liên quan trực tiếp đến những vi phạm hiện tại.

4.3.4. Lựa chọn giá trị nhằm giảm số ràng buộc vi phạm

Đối với mỗi đỉnh đang xung đột, chương trình thử các màu khác màu hiện tại và tính lại số xung đột trên những cạnh gắn với đỉnh đó. Màu mới chỉ được chấp nhận khi làm số xung đột giảm nghiêm ngặt:

$\Delta f = f(A_mới) - f(A_hiện\ tại) < 0.$

Nếu có nhiều màu đạt mức cải thiện tốt nhất cho đỉnh đang xét, thuật toán chọn ngẫu nhiên một màu trong số đó. Chương trình không chấp nhận bước làm tăng hoặc giữ nguyên số xung đột. Vì vậy, đây là biến thể Min-Conflicts strict-improvement trong notebook, không đồng nhất với mọi biến thể Min-Conflicts thông dụng có thể cho phép di chuyển trên vùng bằng phẳng (plateau). Kết quả uphill_moves = 0 được kiểm tra bằng mã nguồn.

4.3.5. Điều kiện dừng và cơ chế khởi tạo lại

Nếu đạt f(A) = 0, thuật toán trả trạng thái SOLVED và kiểm tra lại nghiệm bằng hàm is_csp_solution(). Nếu không có nước đổi màu nào làm giảm f(A), lần khởi tạo hiện tại dừng tại trạng thái STUCK; nếu đạt giới hạn bước của một lần khởi tạo, nó dừng do ngân sách vòng lặp. Sau đó thuật toán có thể khởi tạo lại từ một phép gán ngẫu nhiên khác.

Thiết lập thực nghiệm giới hạn tối đa tám lần khởi tạo (max_starts = 8, tức nhiều nhất bảy lần khởi động lại) và 200 bước cải thiện được chấp nhận cho mỗi lần khởi tạo (max_steps

= 200). Hai chỉ số steps và value_evaluations lần lượt ghi số bước đổi màu đã chấp nhận và số phép đánh giá màu ứng viên. Kết quả STUCK hoặc LIMIT không cung cấp bằng chứng rằng CSP vô nghiệm.

4.4. Thiết lập thực nghiệm

4.4.1. Sinh đồ thị Delaunay với các kích thước khác nhau

Bài thực hành sử dụng một đồ thị cố định từ Bài 1–3 và một tập gồm 25 đồ thị Delaunay độc lập. Đồ thị cố định có 10 đỉnh, 21 cạnh, seed 1111 và chứa đồ thị con K_4 gồm các đỉnh (2, 3, 4, 9). Tập đồ thị mở rộng được sinh với $n \in \{8, 10, 14, 18, 24\}$, mỗi kích thước gồm năm đồ thị ngẫu nhiên. Tọa độ được lấy trong miền $[0, 1]^2$, sau đó sử dụng phép tam giác hóa Delaunay để tạo cạnh.

4.4.2. Khảo sát miền ba màu và bốn màu

Trên mỗi đồ thị, chương trình tạo hai mô hình CSP dùng chung tập đỉnh và cạnh, chỉ thay đổi miền giá trị. Miền ba màu gồm red, blue, green; miền bốn màu bổ sung orange. Việc giữ nguyên cấu trúc đồ thị giúp đánh giá trực tiếp tác động của kích thước miền mà không bị nhiễu bởi thay đổi số cạnh. Đồ thị chứa K_4 không thể tô hợp lệ bằng ba màu, nhưng các đồ thị Delaunay khác có thể tồn tại nghiệm ba màu.

4.4.3. Thiết lập random seed và số lần thực nghiệm

Seed dùng để sinh đồ thị được xác định bằng công thức $20261020 + 100 \times n + trial$, trong đó trial được đánh số từ 0 đến 4 trong mã nguồn và từ 1 đến 5 trong CSV. Min-Conflicts sử dụng seed bằng seed đồ thị cộng 900001. Với 25 đồ thị, hai miền màu và ba phương pháp trên mỗi mô hình, tổng số lượt thực nghiệm là $25 \times 2 \times 3 = 150$. Cả hai thuật toán Backtracking trong phần khảo sát có node_limit = 10000.

4.4.4. Các chỉ số COUNT, số bước, tỷ lệ thành công và thời gian

Các thước đo phải được hiểu theo đúng định nghĩa triển khai. COUNT là số lần gọi hàm đệ quy của Backtracking, bao gồm cả nút nghiệm; attempts là số phép gán màu được thử. Với Min-Conflicts, steps là số lần đổi màu thực sự được chấp nhận, restarts là số lần khởi động lại đã dùng và value_evaluations là số lần đánh giá màu thay thế.

Tỷ lệ thành công được tính bằng tỷ số số đồ thị tìm được phép gán hợp lệ trên năm đồ thị mỗi cấu hình, nhân 100%. Thời gian được ghi nhận bằng perf_counter() và tổng hợp theo trung vị để hạn chế ảnh hưởng của dao động thời gian đo. Do các chỉ số COUNT và steps khác đơn vị, báo cáo trình bày thành các cột hoặc đồ thị riêng, không so sánh trực tiếp trị số của chúng.

4.5. Kết quả thực nghiệm và so sánh

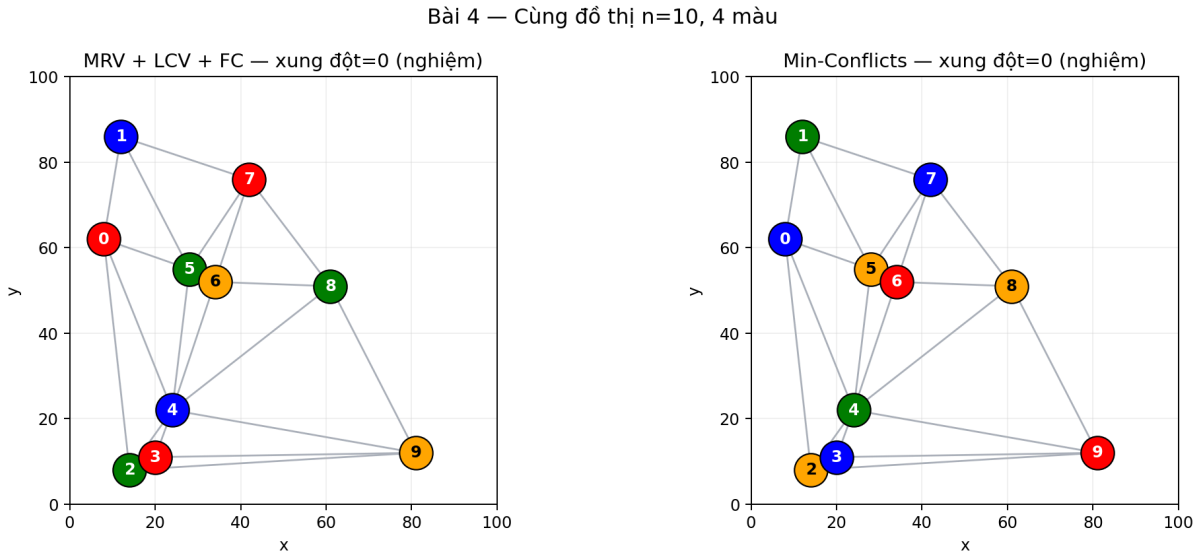
4.5.1. Kết quả trên đồ thị ban đầu

Bảng 4.1. Kết quả ba thuật toán trên đồ thị Delaunay cố định (n = 10, 21 cạnh)

n	BT 3 màu	FC 3 màu	MC 3 màu	BT 4 màu	FC 4 màu	MC 4 màu
4 màu	MRV + LCV	SOLVED	11	10	—	0,26615
4 màu	MRV + LCV + FC	SOLVED	11	10	—	0,07789
4 màu	Min-Conflicts	SOLVED	—	—	4	0,07372
3 màu	MRV + LCV	UNSAT	34	33	—	0,40169
3 màu	MRV + LCV + FC	UNSAT	28	33	—	0,19464
3 màu	Min-Conflicts	STUCK	—	—	18	0,34523

Với miền bốn màu, cả ba phương pháp đều tìm được nghiệm hợp lệ và không có cạnh vi phạm. Hai thuật toán Backtracking đều ghi COUNT = 11 và 10 lần thử màu. FC đã loại tổng cộng 20 lượt giá trị khỏi các miền láng giềng nhưng không giảm số nút DFS trong trường hợp dễ này. Min-Conflicts tìm được nghiệm sau bốn lần đổi màu được chấp nhận và 21 lượt đánh giá màu ứng viên.

Với miền ba màu, Backtracking MRV + LCV kết luận UNSAT sau 34 lời gọi đệ quy; khi thêm FC, số lời gọi giảm xuống 28, tương đương mức giảm khoảng 17,6%. pruned_values của FC đạt 90 lượt, cho thấy thuật toán phát hiện và loại bỏ các giá trị không còn hợp lệ qua nhiều nhánh. Min-Conflicts thực hiện 18 bước đổi màu, bảy lần restart và dừng với hai cạnh xung đột trong nghiệm tốt nhất. Trạng thái STUCK là kết quả giới hạn của tìm kiếm cục bộ, còn kết luận vô nghiệm ba màu ở đây được bảo đảm bởi K_4 và quá trình Backtracking đầy đủ.



Hình 4.1. Nghiệm tô bốn màu trên cùng đồ thị Delaunay với Forward Checking và Min-Conflicts.

4.5.2. Kết quả trên nhiều đồ thị ngẫu nhiên

Tập khảo sát gồm năm kích thước đồ thị với số cạnh trung bình lần lượt là 15,6; 21,6; 31,8; 43,8 và 60,6. Trên toàn bộ 150 lượt chạy, cả hai biến thể Backtracking đều hoàn

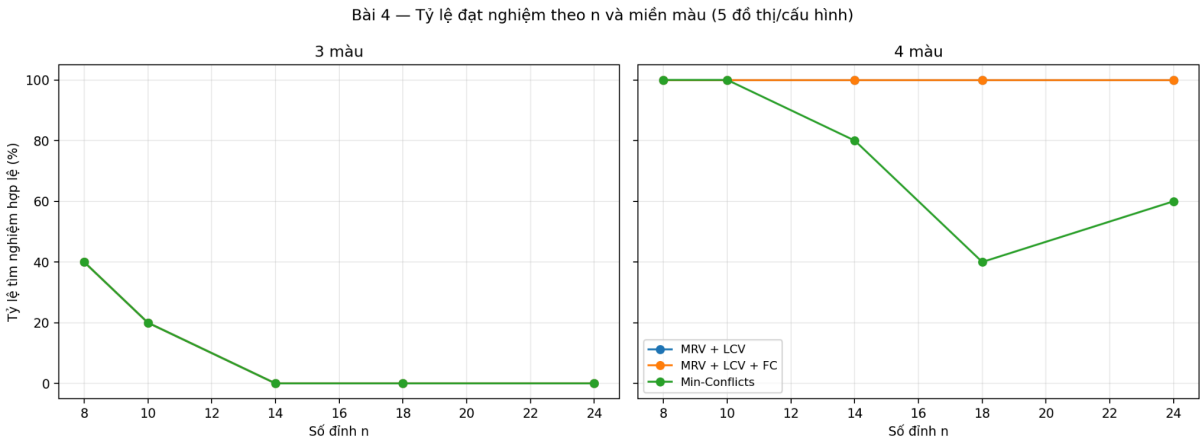
thành với trạng thái SOLVED hoặc UNSAT, không có lượt nào đạt giới hạn 10000 nút. Với miền ba màu, số đồ thị có nghiệm giảm dần từ 2/5 tại n = 8 xuống 1/5 tại n = 10, rồi 0/5 tại ba quy mô lớn hơn. Ngược lại, trong mẫu thí nghiệm này, Backtracking tìm được nghiệm bốn màu trên cả năm đồ thị của mọi kích thước.

4.5.3. So sánh tỷ lệ tìm được nghiệm hợp lệ

Bảng 4.2. Tỷ lệ tìm được nghiệm hợp lệ theo kích thước và miền màu (5 đồ thị/cấu hình)

n	BT 3 màu	FC 3 màu	MC 3 màu	BT 4 màu	FC 4 màu	MC 4 màu
8	40%	40%	40%	100%	100%	100%
10	20%	20%	20%	100%	100%	100%
14	0%	0%	0%	100%	100%	80%
18	0%	0%	0%	100%	100%	40%
24	0%	0%	0%	100%	100%	60%

Với ba màu, tỷ lệ nghiệm bằng nhau giữa ba phương pháp ở mọi kích thước được khảo sát, nhưng ý nghĩa trạng thái thất bại khác nhau. Backtracking đã chứng minh các đồ thị còn lại vô nghiệm, còn Min-Conflicts chỉ trả STUCK. Với bốn màu, hai thuật toán Backtracking đạt 100% trong bộ mẫu, trong khi Min-Conflicts strict-improvement giảm còn 80%, 40% và 60% tại n = 14, 18 và 24. Sự dao động của Min-Conflicts cho thấy kết quả phụ thuộc vào đồ thị, trạng thái khởi tạo và ngân sách tìm kiếm; không thể suy ra một quy luật suy giảm đơn điệu chỉ từ năm mẫu cho mỗi kích thước.



Hình 4.2. Tỷ lệ tìm được nghiệm hợp lệ theo kích thước đồ thị và miền màu (5 đồ thị cho mỗi cấu hình).

4.5.4. So sánh COUNT, số bước và thời gian thực thi

Ở miền ba màu, COUNT trung bình của Backtracking MRV + LCV và MRV + LCV + FC lần lượt là 19,2 so với 15,6 tại n = 8; 27,0 so với 22,2 tại n = 10; 40,0 so với 34,0 tại n = 14; 46,0 so với 40,0 tại n = 18; và 32,8 so với 26,8 tại n = 24. Forward Checking giảm COUNT ở cả năm quy mô ba màu được khảo sát. Các trị số không tăng đều theo n vì cấu trúc cạnh và khả năng phát hiện sớm mâu thuẫn khác nhau giữa các nhóm đồ thị.

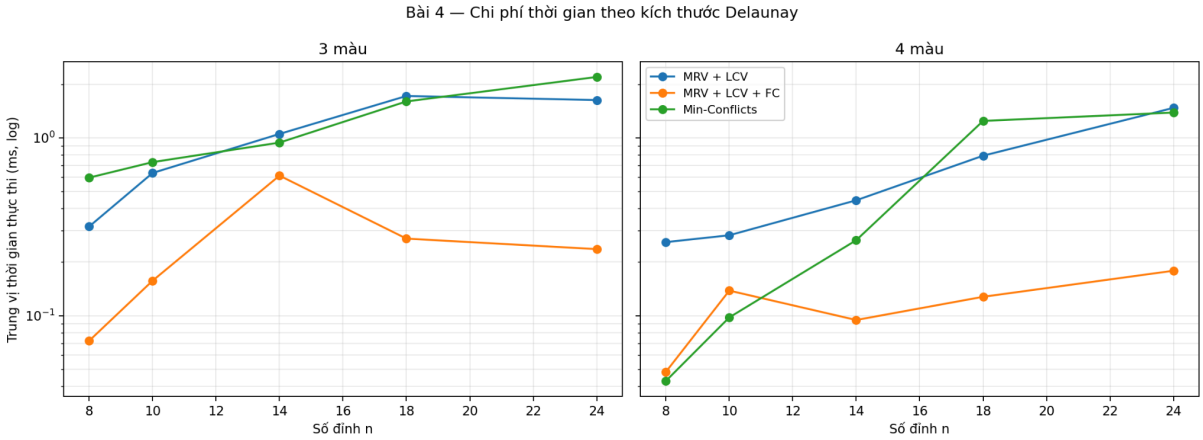
Bảng 4.3. Trung vị thời gian chạy (ms) trên năm đồ thị/cấu hình

n	BT 3 màu	FC 3 màu	MC 3 màu	BT 4 màu	FC 4 màu	MC 4 màu
8	0,31692	0,07215	0,59667	0,25896	0,04823	0,04288
10	0,63399	0,15712	0,72964	0,28300	0,13821	0,09768
14	1,05110	0,61267	0,93959	0,44511	0,09446	0,26539
18	1,71830	0,27104	1,60091	0,79291	0,12757	1,24400
24	1,62998	0,23654	2,19814	1,47282	0,17876	1,38648

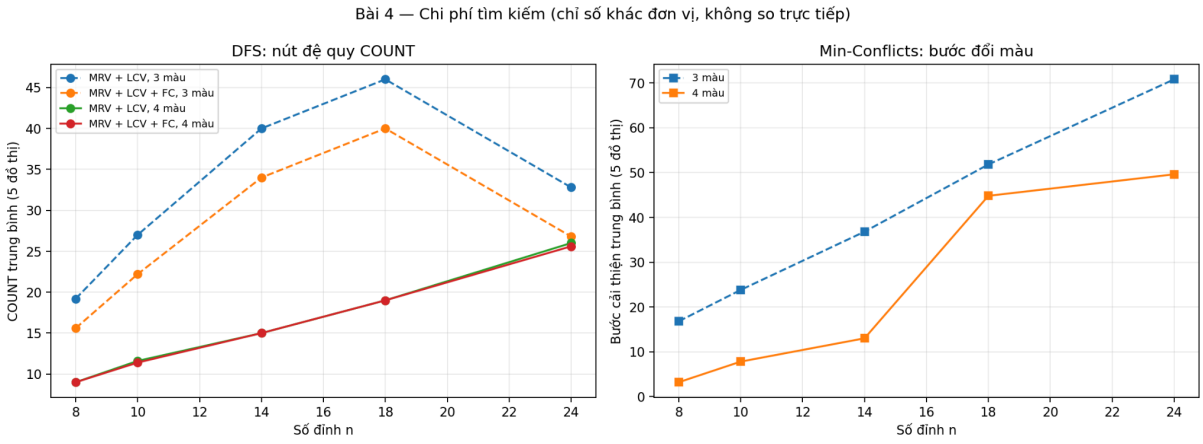
Trong bộ số liệu đã ghi nhận, FC thường có thời gian trung vị thấp hơn phiên bản không FC, dù bản thân FC phải thực hiện thêm thao tác duy trì miền. Chẳng hạn, ở n = 24 và miền bốn màu, trung vị giảm từ 1,47282 ms xuống 0,17876 ms. Tuy nhiên, các giá trị thời

gian chỉ thuộc về môi trường và cách cài đặt đang khảo sát; chúng không chứng minh rằng FC luôn nhanh hơn trên mọi CSP.

Biểu đồ COUNT và steps cần được đọc thành hai nhóm: COUNT biểu thị số lời gọi DFS, còn steps của Min-Conflicts phản ánh số đổi màu đã được chấp nhận, cộng dồn qua các lần khởi tạo khi cần thiết. Vì khác đơn vị, chỉ nên so sánh xu hướng trong từng nhóm hoặc kết hợp với chỉ số thời gian chạy và tỷ lệ nghiệm để đánh giá toàn diện.



Hình 4.3. Trung vị thời gian thực thi theo kích thước n trên thang đo logarit.



Hình 4.4. Công sức tìm kiếm với các thước đo khác đơn vị: COUNT của DFS và số bước đổi màu của Min-Conflicts.

4.5.5. Phân biệt trường hợp vô nghiệm và hết ngân sách tìm kiếm

Notebook quy định bốn trạng thái chính. SOLVED được ghi khi nghiệm đã vượt qua kiểm tra đầy đủ, đúng miền và nhất quán. UNSAT chỉ được sử dụng khi Backtracking kết thúc sau khi đã duyệt hết những nhánh cần xét mà không tìm được nghiệm. LIMIT biểu thị quá trình tìm kiếm chạm ngưỡng tính toán, nên chưa thể kết luận vô nghiệm. STUCK biểu thị Min-Conflicts không tìm được bước cải thiện tiếp theo trong các lần khởi tạo đã thử.

Trong 150 lượt thử, hai thuật toán Backtracking không đạt LIMIT, vì vậy các ca UNSAT trong bảng tổng hợp là kết luận của quá trình tìm kiếm đầy đủ đối với những đồ thị tương ứng. Min-Conflicts có những ca STUCK cả khi miền bốn màu tồn tại nghiệm, minh họa rõ việc thất bại của tìm kiếm cục bộ không đồng nghĩa bài toán không có lời giải.

4.6. Nhận xét

Kết quả thực nghiệm cho thấy Forward Checking là sự bổ sung hữu ích cho Backtracking: bằng cách loại màu không còn hợp lệ và phát hiện miền rỗng từ sớm, thuật toán giảm số nút đệ quy ở các trường hợp khó như bài toán ba màu. Cơ chế hoàn tác miền là điều kiện quan trọng để bảo đảm tính đúng đắn; notebook đã kiểm chứng việc khôi phục miền kể cả khi nhánh tìm kiếm thất bại. Tổng cộng 17 ca kiểm thử Bài 4 đều có trạng thái PASS, bao gồm kiểm tra tính hợp lệ của nghiệm, phát hiện vô nghiệm, bất biến của miền và dữ liệu 150 lượt chạy.

Min-Conflicts có ưu điểm là thao tác trên phép gán đầy đủ và tìm các thay đổi cục bộ làm giảm số cạnh vi phạm. Trên đồ thị 10 đỉnh bốn màu, phương pháp đạt nghiệm với bốn bước đổi màu; tuy nhiên, quy tắc chỉ chấp nhận cải thiện nghiêm ngặt khiến thuật toán có thể mắc kẹt ở cực tiểu cục bộ hoặc vùng không có bước giảm. Random Restart giúp mở rộng khả năng thử các điểm khởi tạo khác nhau nhưng không bảo đảm thành công trong ngân sách hữu hạn.

Khi đánh giá ba phương pháp cần tránh đồng nhất COUNT với steps và tránh kết luận về hiệu suất tổng quát chỉ từ thời gian chạy rất nhỏ. Các phép đo nên được đọc cùng tỷ lệ đạt nghiệm, số lần thử gán màu, số lần đánh giá hàm mục tiêu và trạng thái kết thúc. Xét trong phạm vi dữ liệu đã thực hiện, MRV + LCV + Forward Checking là lựa chọn đáng tin cậy khi cần xác định nghiệm hoặc chứng minh vô nghiệm; Min-Conflicts thích hợp để nghiên cứu khả năng tìm nghiệm bằng cải thiện cục bộ, với điều kiện chấp nhận rủi ro dừng sớm và phải kiểm tra lại nghiệm trả về.

CHƯƠNG 5: BIỂU DIỄN BÀI TOÁN N QUÂN HẬU

5.1. Giới thiệu bài toán n-Queens

5.1.1. Phát biểu bài toán và mục tiêu tìm kiếm

Bài toán n quân hậu yêu cầu đặt n quân hậu trên bàn cờ kích thước n × n sao cho không có hai quân hậu nào có thể tấn công lẫn nhau. Một cấu hình ban đầu có thể chứa xung đột; mục tiêu là tìm một cách bố trí thỏa mãn toàn bộ các điều kiện an toàn giữa các quân hậu. Đây là bài toán kinh điển dùng để minh họa việc biểu diễn trạng thái, ràng buộc và tìm kiếm nghiệm trong trí tuệ nhân tạo.

Trong notebook, chất lượng của một bàn cờ được biểu diễn bằng hàm h(board), là số cặp quân hậu tấn công nhau. Quá trình giải bài toán vì thế có thể được diễn đạt dưới dạng tối thiểu hóa h(board); nghiệm mong muốn đạt giá trị 0. Việc sử dụng hàm đếm xung đột tạo tiền đề cho các phương pháp tìm kiếm cục bộ ở những bài thực hành tiếp theo.

5.1.2. Quy tắc tấn công của quân hậu

Theo luật cờ vua, quân hậu có thể tấn công các ô nằm trên cùng hàng, cùng cột hoặc cùng đường chéo. Hai quân hậu sẽ xung đột nếu chúng cùng nằm trên một trong các hướng này và không bị xét đến yếu tố chặn quân khác. Với mô hình được sử dụng trong notebook, mỗi cột luôn chứa đúng một quân hậu, do đó không thể xuất hiện hai quân hậu cùng cột; chỉ cần kiểm tra hàng và hai hướng đường chéo.

Gọi i, j là hai chỉ số cột khác nhau và q_i, q□ là chỉ số hàng tương ứng, với i < j. Hai quân hậu tấn công nhau nếu $q_i = q□$ hoặc $|q_i - q□| = |i - j|$. Điều kiện thứ nhất biểu thị cùng

hàng; điều kiện thứ hai biểu thị nằm trên cùng đường chéo. Nếu cả hai điều kiện đều không xảy ra, cặp hậu đang xét không xung đột.

5.1.3. Mô hình hóa n-Queens dưới góc độ CSP

Dưới góc độ bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problem – CSP), ta định nghĩa tập biến $X = \{Q_0, Q_1, \ldots, Q_{n-1}\}$, mỗi biến Q_i mô tả hàng đặt quân hậu tại cột i. Miền giá trị của từng biến là $D_i = \{0, 1, \ldots, n - 1\}$. Tập ràng buộc C yêu cầu mọi cặp biến tại các cột khác nhau phải nhận giá trị hàng khác nhau và không nằm chung đường chéo.

$$Q_i \neq Q_j \quad \text{và} \quad |Q_i - Q_j| \neq |i - j|, \quad \text{với mọi } i < j$$

Một phép gán đầy đủ là việc xác định hàng cho cả n quân hậu, nhưng chỉ trở thành nghiệm CSP khi tất cả ràng buộc đều được thỏa mãn. Khi tiếp cận theo hướng tối ưu hóa của notebook, thay vì chỉ kiểm tra từng ràng buộc đúng hoặc sai, ta tổng hợp số vi phạm thành một giá trị nguyên không âm để định lượng mức độ không hợp lệ của trạng thái.

5.2. Biểu diễn trạng thái bàn cờ

5.2.1. Vector trạng thái gồm n phần tử

Notebook không lưu toàn bộ ma trận n × n như một trạng thái tìm kiếm. Thay vào đó, bàn cờ được biểu diễn bằng vector board gồm n phần tử nguyên, trong đó chỉ số phần tử thể hiện cột và giá trị phần tử thể hiện hàng của quân hậu. Quy ước đánh chỉ số bắt đầu từ 0 tương ứng với cách lưu trữ mảng của Python và NumPy.

$$\text{board} = [q_0, q_1, \ldots, q_{n-1}], \quad q_i \in \{0, 1, \ldots, n - 1\}$$

Chẳng hạn, với board = [1, 3, 0, 2], các quân hậu lần lượt ở vị trí (cột 0, hàng 1), (cột 1, hàng 3), (cột 2, hàng 0) và (cột 3, hàng 2). Vector giúp truy xuất vị trí quân hậu tại một

cột trực tiếp bằng board[i] và thuận tiện cho việc thay đổi giá trị hàng khi xét các trạng thái lân cận.

5.2.2. Quy ước mỗi cột có đúng một quân hậu

Mỗi phần tử của vector tương ứng với đúng một cột và lưu đúng một vị trí hàng, vì vậy quy tắc một hậu trên một cột được bảo đảm ngay từ cấu trúc dữ liệu. Điều này làm giảm số điều kiện cần kiểm tra, bởi ràng buộc không trùng cột không thể bị vi phạm trong tập trạng thái đã định nghĩa. Tuy nhiên, cấu trúc này không ngăn các quân hậu cùng hàng hoặc cùng đường chéo.

Điểm cần phân biệt là việc có đủ n quân hậu trên n cột không đồng nghĩa với việc bàn cờ hợp lệ. Ví dụ, board = [0, 0, 3, 0] thỏa điều kiện mỗi cột có một quân hậu nhưng xuất hiện ba cặp xung đột do các quân hậu ở cột 0, 1 và 3 cùng hàng 0.

5.2.3. Không gian trạng thái của bài toán

Với mỗi cột có n khả năng lựa chọn hàng, tổng số trạng thái có thể được mã hóa theo quy ước trên là n^n. Đây là số cấu hình khả dĩ trước khi áp dụng yêu cầu không tấn công nhau, không phải số nghiệm hợp lệ. Chẳng hạn, không gian của 4-Queens gồm $4^4 = 256$ trạng thái, còn 8-Queens gồm $8^8 = 16.777.216$ trạng thái.

Nếu bổ sung ràng buộc mỗi hàng chỉ chứa tối đa một hậu ngay trong cách biểu diễn, ta có thể giới hạn các trạng thái vào những hoán vị của n hàng, khi đó số cấu hình là n!. Tuy nhiên, notebook hiện tại không sử dụng biểu diễn hoán vị mà cho phép nhiều cột có cùng giá trị hàng; vì vậy kích thước không gian trạng thái cần ghi nhận là n^n.

5.3. Các hàm xử lý bài toán n-Queens

5.3.1. Hàm random_board() khởi tạo bàn cờ ngẫu nhiên

Hàm random_board(n) sử dụng np.random.randint(0, n, size=n) để tạo một mảng gồm n giá trị nguyên ngẫu nhiên từ 0 đến n − 1. Mỗi giá trị xác định hàng đặt quân hậu của cột tương ứng. Cơ chế này bảo đảm số phần tử và miền giá trị hợp lệ nhưng không kiểm soát số cặp xung đột, do đó bàn cờ ngẫu nhiên thường chỉ là trạng thái khởi tạo chứ chưa phải nghiệm.

Để thuận tiện tái lập kết quả, đoạn mã bổ sung dành cho Bài 5 đặt BAI5_SEED = 20261020 và khởi tạo lại bộ sinh số ngẫu nhiên trước khi sinh lần lượt bàn cờ 4-Queens và 8-Queens. Cần lưu ý các cell minh họa gốc phía trên sử dụng seed 1234; vì thế bàn cờ 4-Queens từ cell gốc có thể khác cấu hình 4-Queens được sử dụng trong các bảng CSV và hình xuất của Bài 5.

5.3.2. Hàm show_board() trực quan hóa bàn cờ

Hàm show_board(board) sử dụng Matplotlib để tạo nền bàn cờ với các ô sáng và tối xen kẽ, sau đó đặt ký hiệu quân hậu tại tọa độ (cột i, hàng board[i]). Hàm gọi conflicts(board) để in tổng số xung đột và dùng plt.show() hiển thị hình. Cách biểu diễn này giúp quan sát trực tiếp vị trí quân hậu, kiểm tra sự tương ứng giữa vector và bàn cờ cũng như phát hiện các hàng hoặc đường chéo có nguy cơ vi phạm ràng buộc.

Các tệp PNG đính kèm được sinh bằng hàm xuất hình riêng, dựa trên cùng những vector đã được kiểm chứng. So với hàm show_board() gốc, hình PNG có thêm tiêu đề, trục hàng/cột và giá trị h(board) để việc đưa vào báo cáo thuận tiện hơn; dữ liệu bố trí quân hậu không thay đổi.

5.3.3. Hàm conflicts() đếm các cặp quân hậu xung đột

Hàm conflicts(board) trong notebook không duyệt trực tiếp mọi cặp quân hậu. Thay vào đó, hàm tạo ba mảng đếm số quân hậu theo từng hàng và theo hai nhóm đường chéo. Khi duyệt cột i, hàng q_i = board[i], thuật toán tăng lần lượt bộ đếm tại chỉ số q_i, $i + q_i$ và $i - q_i + n$. Mỗi nhóm chứa k quân hậu góp $k(k - 1)/2$ cặp xung đột.

Sau khi thống kê số hậu ở mọi hàng và đường chéo, hàm áp dụng comb2(k) = k(k − 1) // 2 trên từng bộ đếm rồi cộng tất cả kết quả. Cách tính này tránh việc so sánh riêng từng cặp hậu, trong khi vẫn ghi nhận chính xác số cặp cùng hàng hoặc cùng đường chéo. Với ba mảng đếm có kích thước tỷ lệ theo n, thời gian xử lý của hàm là O(n), so với $O(n^2)$ đối với cách kiểm tra từng cặp.

5.4. Hàm mục tiêu và điều kiện nghiệm hợp lệ

5.4.1. Xung đột giữa các quân hậu cùng hàng

Hai quân hậu tại cột i và j xung đột theo hàng khi board[i] = board[j]. Nếu một hàng có k quân hậu, số cặp trong cùng hàng bằng tổ hợp chập hai C(k, 2). Gọi R_r là số quân hậu ở hàng r, tổng số cặp xung đột ngang bằng tổng $C(R_r, 2)$ trên tất cả các hàng. Ví dụ, khi có ba quân hậu nằm trên hàng 0, số cặp xung đột ngang là C(3, 2) = 3.

5.4.2. Xung đột giữa các quân hậu trên đường chéo

Trên bàn cờ có tọa độ cột i và hàng q_i, các ô thuộc cùng một đường chéo có tổng $i + q_i$ bằng nhau hoặc hiệu $i - q_i$ bằng nhau. Vì vậy, hai nhóm đường chéo được mã hóa bằng i + board[i] và i − board[i] + n. Hằng số n chỉ dịch chuyển chỉ số hiệu sang miền không âm, không làm thay đổi quan hệ trùng đường chéo. Mỗi đường chéo có k quân hậu cũng đóng góp C(k, 2) cặp vi phạm.

5.4.3. Hàm mục tiêu h(board)

Hàm mục tiêu của bài thực hành được định nghĩa là tổng số cặp quân hậu cùng hàng hoặc cùng đường chéo. Nếu R_r, $D\square^{(+)}$ và $D\square^{(-)}$ lần lượt là các bộ đếm quân hậu trên hàng r và hai họ đường chéo, hàm mục tiêu được viết như sau:

$h(board) = \Sigma_r\, C(R_r, 2) + \Sigma\square\, C(D\square^{(+)}, 2) + \Sigma\square\, C(D\square^{(-)}, 2)$

Tất cả các số hạng đều không âm. Mỗi cặp quân hậu ở hai cột khác nhau có thể trùng hàng hoặc nằm trên một đường chéo, và không bị đếm trùng giữa các loại xung đột. Giá trị h càng nhỏ phản ánh càng ít ràng buộc bị vi phạm; h = 0 biểu thị không còn cặp quân hậu tấn công nhau.

5.4.4. Điều kiện nghiệm hợp lệ h(board) = 0

Một trạng thái là nghiệm hợp lệ khi mọi quân hậu đều được đặt trên bàn cờ theo quy ước một hậu/cột và không tồn tại cặp quân hậu cùng hàng hoặc đường chéo. Theo cách định nghĩa và triển khai conflicts(), điều kiện đó tương đương h(board) = 0. Điều kiện này mang ý nghĩa kiểm tra ràng buộc, không phải điều kiện đánh giá đường đi; notebook không tìm đường từ trạng thái đầu đến trạng thái đích trong phần Bài 5.

board là nghiệm hợp lệ $\Leftrightarrow$ h(board) = 0

Phân biệt giữa giá trị hàm mục tiêu và trạng thái hợp lệ giúp đánh giá chính xác đầu ra của các thuật toán Hill Climbing hoặc Simulated Annealing ở những bài sau. Một thuật toán có thể trả về bàn cờ với h nhỏ nhưng lớn hơn 0; trạng thái đó là nghiệm gần đúng theo tiêu chí tối ưu hóa, chưa phải nghiệm CSP hợp lệ.

5.5. Kết quả thực nghiệm

5.5.1. Khảo sát các bàn cờ ngẫu nhiên với n = 4

Trong phần bổ sung của notebook, sau khi đặt seed 20261020, hàm random_board(4) sinh cấu hình [0, 0, 3, 0]. Ba quân hậu tại các cột 0, 1 và 3 cùng nằm trên hàng 0, tạo thành ba cặp xung đột (0, 1), (0, 3) và (1, 3). Quân hậu tại cột 2, hàng 3 không tạo thêm xung đột với ba quân còn lại, do đó kết quả h(board) = 3 phù hợp với hình minh họa và bảng kiểm tra.

Kết quả xác nhận điều kiện một hậu/cột không đủ để bảo đảm nghiệm. Hình 5.1 minh họa một trạng thái hợp lệ về cấu trúc vector nhưng chưa thỏa mãn các ràng buộc của bài toán.

5.5.2. Khảo sát các bàn cờ ngẫu nhiên với n = 8

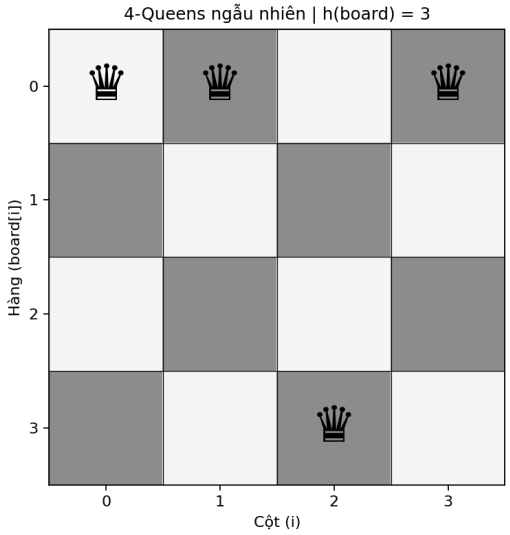
Với n = 8, cấu hình ngẫu nhiên được ghi nhận là [6, 2, 3, 7, 4, 1, 6, 4], có h(board) = 5. Các cặp cột chứa quân hậu xung đột lần lượt là (0, 5), (0, 6), (1, 2), (4, 6) và (4, 7). Trong đó, (0, 6) và (4, 7) là xung đột cùng hàng; ba cặp còn lại là xung đột trên đường chéo.

So với cấu hình bốn quân hậu, số lượng quân hậu tăng làm xuất hiện nhiều khả năng tương tác và xung đột hơn. Tuy nhiên, số xung đột thực tế phụ thuộc cách sắp xếp quân hậu, không chỉ phụ thuộc giá trị n. Bàn cờ 8-Queens ngẫu nhiên trong hình có năm cặp xung đột, trong khi cấu hình 8-Queens đối chứng không có cặp nào.

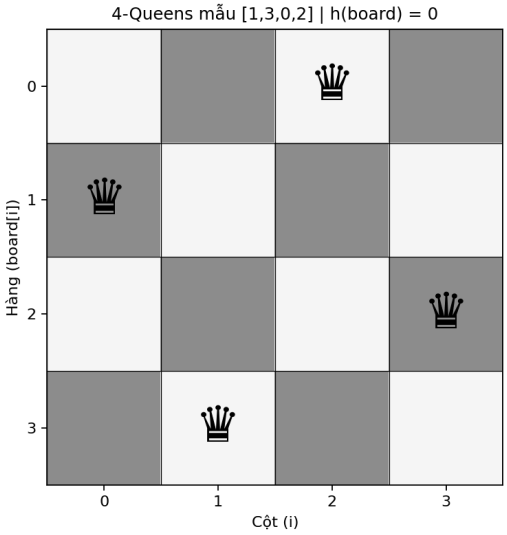
5.5.3. Kiểm chứng cấu hình mẫu [1, 3, 0, 2]

Cấu hình mẫu [1, 3, 0, 2] đặt bốn quân hậu ở bốn hàng khác nhau và không có hai quân hậu nào nằm trên cùng đường chéo. Kết quả của conflicts() bằng 0, khẳng định đây là nghiệm hợp lệ của bài toán 4-Queens. Để mở rộng việc đối chiếu, notebook sử dụng thêm cấu hình 8-Queens tham chiếu [0, 4, 7, 5, 2, 6, 1, 3], cũng đạt h(board) = 0.

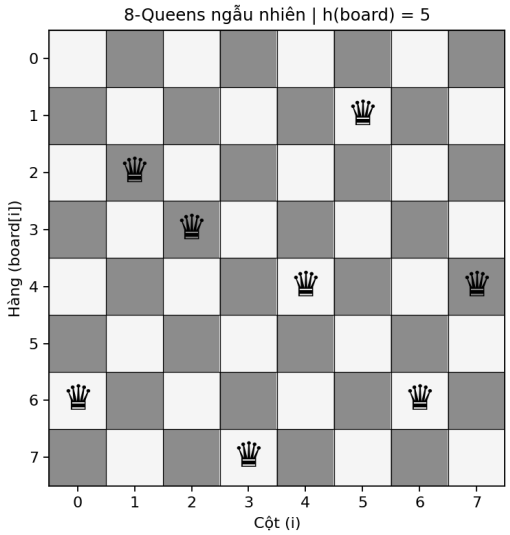
Bốn hình dưới đây thể hiện trực tiếp sự khác biệt giữa trạng thái ngẫu nhiên chưa hợp lệ và trạng thái đối chứng hợp lệ. Các tọa độ ghi trên hình tuân theo quy ước chỉ số hàng và cột bắt đầu từ 0.



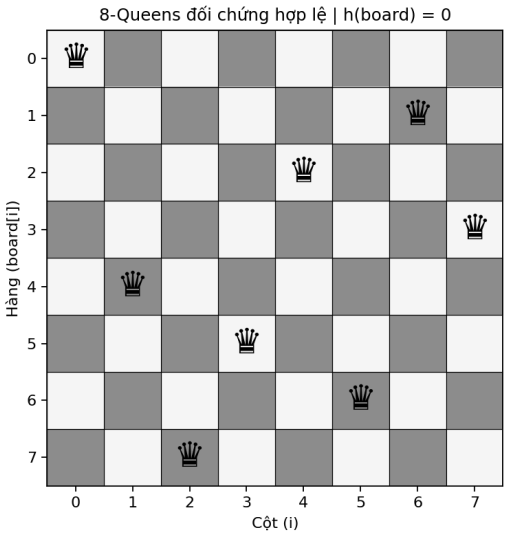
Hình 5.1. Bàn cờ 4-Queens ngẫu nhiên với ba cặp xung đột.



Hình 5.2. Cấu hình 4-Queens mẫu [1, 3, 0, 2] không xung đột.



Hình 5.3. Bàn cờ 8-Queens ngẫu nhiên với năm cặp xung đột.



Hình 5.4. Cấu hình 8-Queens đối chứng có h(board) = 0.

Bảng 5.1. Tổng hợp bốn cấu hình được khảo sát trong Bài 5

Trường hợp	n	Vector board	h(board)	Hợp lệ
Ngẫu nhiên 4-Queens	4	[0, 0, 3, 0]	3	Không
Mẫu 4-Queens	4	[1, 3, 0, 2]	0	Có
Ngẫu nhiên 8-Queens	8	[6, 2, 3, 7, 4, 1, 6, 4]	5	Không
Đối chứng 8-Queens	8	[0, 4, 7, 5, 2, 6, 1, 3]	0	Có

5.5.4. Đối chiếu hàm conflicts() với phương pháp kiểm tra từng cặp hậu

Nhằm kiểm chứng độ chính xác của hàm conflicts() gốc, phần bổ sung định nghĩa hàm attack_pairs(board) dùng cách kiểm tra từng cặp cột i < j. Một cặp được đưa vào danh sách khi có cùng hàng hoặc khoảng chênh lệch hàng bằng khoảng chênh lệch cột. Kết quả so sánh được đánh giá bằng hai chỉ số: chênh lệch số cặp xung đột và sự bất đồng về điều kiện h = 0.

Bộ kiểm thử duyệt đầy đủ $4^4 = 256$ trạng thái với n = 4 và lấy mẫu ngẫu nhiên 500 trạng thái với n = 8 bằng bộ sinh số độc lập. Cả hai phép thử đều không phát hiện sai lệch giữa số cặp của attack_pairs() và giá trị conflicts(), đồng thời không có trường hợp hai cách kiểm tra bất đồng về trạng thái có xung đột hay không.

Bảng 5.2. Kết quả kiểm chứng độc lập hàm đếm xung đột

Bộ kiểm thử	Số trạng thái	Sai lệch số cặp	Sai lệch điều kiện h = 0	Kết quả
4-Queens (duyệt toàn bộ)	256	0	0	PASS
8-Queens (lấy mẫu)	500	0	0	PASS

Việc kiểm thử toàn bộ trạng thái 4-Queens là bằng chứng thực nghiệm đầy đủ cho kích thước n = 4. Đối với 8-Queens, 500 trạng thái chỉ là mẫu thử, không bao phủ toàn bộ 16.777.216 trạng thái có thể mã hóa. Mệnh đề h(board) = 0 khi và chỉ khi không có cặp

hậu tấn công nhau được bảo đảm bởi định nghĩa toán học của h và cách cộng số cặp theo từng nhóm, chứ không chỉ bởi kết quả lấy mẫu.

Ở cấu hình 4-Queens ngẫu nhiên, danh sách xung đột thu được là [(0, 1), (0, 3), (1, 3)]; ở cấu hình 8-Queens ngẫu nhiên là [(0, 5), (0, 6), (1, 2), (4, 6), (4, 7)]. Hai cấu hình đối chứng đều trả về danh sách rỗng. Các dữ liệu này được lưu trong Bai5_Queens_Conflict_Pairs.csv, còn kết quả kiểm thử nằm trong Bai5_Queens_Validation.csv.

5.6. Nhận xét

Qua quá trình thực hành, cách biểu diễn trạng thái bằng vector n phần tử cho thấy ưu điểm về tính gọn nhẹ, dễ truy xuất và dễ thao tác. Nhờ gắn mỗi chỉ số cột với đúng một giá trị hàng, bài toán tự động loại bỏ xung đột cùng cột ngay trong cấu trúc dữ liệu. Tuy nhiên, các xung đột hàng và đường chéo vẫn phải được kiểm tra trước khi kết luận một cấu hình là nghiệm.

Hàm conflicts() sử dụng các bộ đếm theo hàng và đường chéo cùng công thức tổ hợp C(k, 2), cho phép tính số cặp tấn công nhau với chi phí theo n tuyến tính trong cách cài đặt này. Kết quả đối chiếu với phương pháp duyệt từng cặp trên 256 trạng thái 4-Queens và 500 mẫu 8-Queens đều thống nhất. Hai cấu hình ngẫu nhiên lần lượt có h = 3 và h = 5, trong khi các cấu hình đối chứng có h = 0, minh họa rõ sự khác biệt giữa trạng thái ban đầu và nghiệm hợp lệ.

Kích thước không gian trạng thái n^n tăng nhanh khi n lớn hơn, đặt ra nhu cầu sử dụng các phương pháp tìm kiếm hoặc tối ưu hóa thay vì liệt kê toàn bộ cấu hình. Hàm h(board) là cơ sở để đánh giá và lựa chọn những trạng thái cải thiện trong các thuật toán Hill Climbing, Random Restart và Simulated Annealing. Riêng Bài 5 chỉ thực hiện mô hình hóa và kiểm chứng; các thuật toán này sẽ được triển khai và đánh giá ở các bài tiếp theo, không được xem là đã hoàn thành trong notebook hiện tại.

CHƯƠNG 6: CÁC BIẾN THỂ HILL CLIMBING

6.1. Cơ sở lý thuyết về Hill Climbing

6.1.1. Nguyên lý tối ưu hóa cục bộ

Hill Climbing là phương pháp tìm kiếm cục bộ trong đó thuật toán duy trì một trạng thái hiện tại và cải thiện dần trạng thái này thông qua các phép di chuyển sang trạng thái lân cận. Thay vì xây dựng một cây tìm kiếm đầy đủ từ trạng thái bắt đầu, thuật toán chỉ xét những phương án có thể đạt tới bằng một phép thay đổi nhỏ. Cách tiếp cận này phù hợp với các bài toán tối ưu có không gian trạng thái lớn, chẳng hạn bài toán n quân hậu (n-Queens).

Trong Bài 6, mục tiêu là giảm số cặp quân hậu có thể tấn công nhau. Ba biến thể được cài đặt từ cùng cơ sở dữ liệu và toán tử dịch chuyển: Steepest-ascent Hill Climbing (Task 1), Inefficient Stochastic Hill Climbing (Task 2) và Efficient Stochastic Hill Climbing, còn gọi là First-choice (Task 3). Mặc dù tên gọi Steepest-ascent thường gắn với cực đại hóa, chương trình sử dụng mục tiêu cực tiểu hóa số xung đột, do đó mỗi bước "leo đồi" tương ứng với việc làm giảm giá trị hàm mục tiêu.

6.1.2. Hàm mục tiêu và không gian trạng thái lân cận

Trạng thái bàn cờ được biểu diễn bằng vector board gồm n phần tử, trong đó board[c] là chỉ số hàng của quân hậu ở cột c. Theo quy ước mỗi cột luôn có đúng một quân hậu, các xung đột có thể xuất hiện trên cùng hàng hoặc đường chéo. Hàm conflicts(board) đếm tổng số cặp quân hậu tấn công nhau; một lời giải hợp lệ phải có giá trị hàm mục tiêu bằng 0.

h(board) = conflicts(board) = số cặp quân hậu tấn công nhau

Toán tử lân cận thay đổi vị trí của đúng một quân hậu sang một hàng khác trong cùng cột, không làm thay đổi những quân hậu còn lại. Mỗi cột có n − 1 vị trí mới và có n cột, vì vậy số trạng thái lân cận của một cấu hình là:

$M = n(n - 1)$

Với n = 4, mỗi trạng thái có 12 lân cận; với n = 8 có 56 lân cận. Cả ba thuật toán chỉ chấp nhận phép di chuyển khi h(mới) < h(hiện tại), bảo đảm dãy giá trị hàm mục tiêu giảm nghiêm ngặt qua mỗi bước đã chấp nhận. Chính sách này được giữ cố định để việc so sánh phản ánh sự khác biệt ở cách lựa chọn lân cận.

6.1.3. Hiện tượng Local Optimum và Plateau

Cực tiểu cục bộ (local minimum) là một trạng thái mà không có lân cận nào đạt giá trị h nhỏ hơn hiện tại, dù giá trị hiện tại vẫn có thể lớn hơn 0. Khi đó, một thuật toán chỉ chấp nhận bước cải thiện sẽ dừng mặc dù chưa tìm được nghiệm hợp lệ. Plateau là vùng gồm các trạng thái có cùng giá trị hàm mục tiêu. Nếu chính sách loại bỏ cả các bước đi ngang, thuật toán có thể không thoát khỏi plateau ngay cả khi tồn tại đường đi đến nghiệm tốt hơn thông qua các trạng thái có cùng giá trị.

Trong notebook, nhãn LOCAL_MINIMUM chỉ được gán cho phương pháp đã khảo sát toàn bộ lân cận và không tìm thấy bước cải thiện; một điểm dừng vẫn có thể có lân cận cùng giá trị h. Đối với First-choice, việc không gặp bước cải thiện qua nhiều lượt lấy mẫu không đủ để kết luận toàn bộ lân cận đều không cải thiện, vì vậy chương trình sử dụng nhãn TRY_LIMIT.

6.2. Triển khai các thuật toán Hill Climbing

6.2.1. Steepest-ascent Hill Climbing – Task 1

Hàm steepest_ascent(board, max_steps=1000) bắt đầu bằng cách sao chép và kiểm tra vector đầu vào, tính h cho trạng thái ban đầu, sau đó duyệt lần lượt toàn bộ $M = n(n - 1)$ ứng viên trong mỗi vòng. Hàm iter_local_neighbors() sinh các lân cận theo thứ tự cố định từ cột nhỏ đến lớn và từ hàng nhỏ đến lớn. Thuật toán lưu trạng thái có giá trị h nhỏ nhất trong số các ứng viên cải thiện, rồi chỉ thực hiện một phép di chuyển sau khi đã đánh giá hết lân cận.

Nếu không có ứng viên nào làm giảm h, chương trình trả trạng thái LOCAL_MINIMUM. Nếu đạt $h = 0$, trạng thái là SOLVED. Nếu số bước cải thiện đạt max_steps trước khi hoàn tất, trạng thái là STEP_LIMIT. Cách triển khai này có ưu điểm lựa chọn được cải thiện lớn nhất trong một bước, nhưng phải chi trả chi phí đánh giá toàn bộ lân cận ở từng vòng, kể cả vòng cuối dùng để xác định không còn hướng cải thiện.

6.2.2. Inefficient Stochastic Hill Climbing – Task 2

Hàm inefficient_stochastic(board, seed=0, max_steps=1000) cũng duyệt đủ toàn bộ lân cận trong mỗi vòng. Điểm khác biệt so với Steepest-ascent là hàm tạo danh sách improving chứa tất cả ứng viên có h nhỏ hơn hiện tại, sau đó chọn ngẫu nhiên một phần tử bằng bộ sinh số ngẫu nhiên có seed. Mọi ứng viên cải thiện có cơ hội được lựa chọn như nhau, không bắt buộc ưu tiên lân cận giảm h mạnh nhất.

Phương pháp này có thể đi theo những quỹ đạo khác nhau từ cùng một trạng thái đầu, do thứ tự các bước cải thiện phụ thuộc lựa chọn ngẫu nhiên. Tuy nhiên, thao tác liệt kê toàn bộ ứng viên khiến số lần đánh giá conflicts() vẫn tỉ lệ với M ở mỗi vòng. Trạng thái dừng là SOLVED, LOCAL_MINIMUM hoặc STEP_LIMIT, tùy kết quả.

6.2.3. Efficient Stochastic Hill Climbing – Task 3

Hàm efficient_stochastic(board, seed=0, max_steps=1000, max_no_improvement=None, max_evaluations=None) thực hiện chiến lược First-choice. Thay vì quét toàn bộ M lân cận, thuật toán sinh ngẫu nhiên từng ứng viên bằng random_local_neighbor() và chấp nhận ngay ứng viên đầu tiên có h nhỏ hơn hiện tại. Nếu ứng viên không cải thiện, thuật toán tiếp tục lấy mẫu. Việc lấy mẫu được thực hiện có hoàn lại, vì vậy một ứng viên có thể xuất hiện nhiều lần trong cùng một giai đoạn thử.

Notebook đặt ngưỡng số lần thử thất bại liên tiếp mặc định theo một chặn xác suất bỏ sót một nước đi cải thiện cố định là 5%. Khi vượt ngưỡng, thuật toán trả về TRY_LIMIT. Chương trình còn hỗ trợ giới hạn số lượt gọi hàm mục tiêu (EVAL_LIMIT), bên cạnh STEP_LIMIT và SOLVED. Cách lựa chọn lân cận này giảm nhu cầu duyệt và lưu toàn bộ tập ứng viên, nhưng kết quả phụ thuộc thứ tự mẫu và ngân sách thử.

6.3. Cơ chế lựa chọn trạng thái và điều kiện dừng

6.3.1. Lựa chọn trạng thái cải thiện tốt nhất

Trong Steepest-ascent, tại trạng thái s, thuật toán tìm một ứng viên s* có h(s*) nhỏ nhất trong tập lân cận N(s). Nếu h(s*) < h(s), thuật toán cập nhật s ← s*. Nếu không tồn tại ứng viên thỏa điều kiện này, quá trình dừng. Tập lân cận được khảo sát đầy đủ, do đó trạng thái LOCAL_MINIMUM được xác định có cơ sở trực tiếp từ phép đánh giá.

s* = arg min { h(s′) : s′ thuộc N(s) }

6.3.2. Lựa chọn ngẫu nhiên trong tập trạng thái cải thiện

Inefficient Stochastic xây dựng tập I(s) gồm tất cả lân cận cải thiện và chọn ngẫu nhiên một ứng viên trong tập này. Mỗi bước được chấp nhận chắc chắn giảm h, nhưng không

nhất thiết đạt mức giảm tốt nhất. Việc đi theo các ứng viên cải thiện khác nhau có thể làm thay đổi điểm dừng cuối, dù cùng một đầu vào và cùng hàm mục tiêu.

$I(s) = \{ s' \text{ thuộc } N(s) : h(s') < h(s) \}$

6.3.3. Sinh và đánh giá từng trạng thái lân cận ngẫu nhiên

First-choice lựa chọn trực tiếp một cột và một hàng mới hợp lệ để sinh ứng viên, chỉ gọi conflicts() cho ứng viên vừa tạo. Khi gặp bước cải thiện, thuật toán cập nhật trạng thái và đặt lại bộ đếm số lần thất bại liên tiếp. Khi chưa gặp cải thiện, bộ đếm tăng lên. Do lấy mẫu có hoàn lại, số lần thử không tương đương số trạng thái lân cận khác nhau đã được kiểm tra.

Theo cấu hình của notebook, với $M = n(n - 1)$ và ngưỡng xác suất 0,05, số lượt thử thất bại liên tiếp mặc định được xác định như sau:

$L = \text{ceil}[\ln(0{,}05) / \ln(1 - 1/M)]$

Công thức trên chặn xác suất không chọn trúng một ứng viên cụ thể qua L lượt lấy mẫu độc lập. Đây không phải chứng minh rằng không còn bất kỳ trạng thái cải thiện nào, cũng không phải bảo đảm thành công với xác suất 95%.

6.3.4. Điều kiện SOLVED, LOCAL_MINIMUM, TRY_LIMIT và STEP_LIMIT

Trạng thái	Ý nghĩa	Có chứng minh không còn bước tốt hơn?
SOLVED	Đã đạt h = 0; nghiệm n-Queens hợp lệ.	Không cần
LOCAL_MINIMUM	Đã quét toàn bộ lân cận, không có h mới nhỏ hơn.	Có, tại trạng thái đang xét
TRY_LIMIT	Hết số lần lấy mẫu liên tiếp không cải thiện.	Không
STEP_LIMIT	Đã hết số bước cải thiện được phép.	Không
EVAL_LIMIT	First-choice đã dùng hết ngân sách đánh giá.	Không

Bảng 6.1. Ý nghĩa các trạng thái kết thúc được sử dụng trong notebook

Việc phân biệt các trạng thái trên cần thiết khi đánh giá kết quả. Đặc biệt, TRY_LIMIT là giới hạn thao tác lấy mẫu chứ không phải bằng chứng toán học của cực tiểu cục bộ. EVAL_LIMIT có mặt trong phần cài đặt mở rộng của First-choice, dù không xuất hiện trong bảng thống kê 180 lượt chạy hiện tại.

6.4. Thiết lập và kết quả thực nghiệm

6.4.1. Thiết lập random seed và số lần chạy

Các thí nghiệm kế thừa hàm random_board(), conflicts() và show_board() của Bài 5, đồng thời dùng seed cơ sở 20261020. Phần minh họa sử dụng cùng các trạng thái khởi tạo n = 4 và n = 8 đã được xây dựng ở Chương 5. Để khảo sát ổn định hơn, notebook sinh 30 bàn cờ cho mỗi kích thước; với mỗi bàn cờ, cả ba thuật toán được thực hiện trên cùng trạng thái đầu. Như vậy, bảng thống kê có 30 × 2 × 3 = 180 lượt chạy độc lập theo thuật toán và cấu hình.

Các chỉ số được ghi gồm initial_h, final_h, steps, evaluations, elapsed_ms, status và solved. Chỉ số steps là số nước đi có giảm h đã được chấp nhận; evaluations là tổng số lần gọi conflicts() trong chính thuật toán, bao gồm lượt tính h ban đầu. Thời gian elapsed_ms được lấy bằng perf_counter(), vì vậy phụ thuộc môi trường thực thi và không đại diện cho độ phức tạp lý thuyết. Phần kiểm thử riêng của notebook ghi nhận sáu trường hợp PASS trên n = 4 và n = 8; yêu cầu tối thiểu 100 lượt của Task 6 sẽ thuộc Bài 8, không được tính là đã hoàn thành tại đây.

6.4.2. Kết quả thực nghiệm trên n = 4

Ở lượt chạy minh họa chung của n = 4, trạng thái đầu có h = 3. Steepest-ascent và Stochastic (all) đều tìm được nghiệm h = 0 sau hai bước chấp nhận, với 25 lần gọi conflicts() cho mỗi phương pháp. First-choice dừng ở h = 2 sau một bước cải thiện, với 38

lần đánh giá và trạng thái TRY_LIMIT. Điều này minh họa rằng kết quả một lượt chạy riêng lẻ có thể khác tỷ lệ thành công trung bình khi lặp lại nhiều lần.

Trong 30 lượt ngẫu nhiên với n = 4, Steepest-ascent đạt 12/30 lượt thành công (40,00%), Stochastic (all) đạt 11/30 (36,67%) và First-choice đạt 14/30 (46,67%). Số xung đột cuối trung bình lần lượt là 0,70; 0,73 và 0,67. First-choice có tỷ lệ thành công cao nhất trong mẫu khảo sát này, nhưng không thể suy rộng thành kết luận tổng quát vì kết quả phụ thuộc các trạng thái đầu và lựa chọn ngẫu nhiên.

6.4.3. Kết quả thực nghiệm trên n = 8

Với n = 8, cùng trạng thái khởi tạo có h = 5 trong lượt minh họa, cả ba thuật toán đều giảm được số xung đột xuống h = 1 nhưng không tìm được nghiệm h = 0. Steepest-ascent dừng tại LOCAL_MINIMUM sau ba bước (225 lần đánh giá); Stochastic (all) cũng dừng tại LOCAL_MINIMUM sau bốn bước (281 lần đánh giá); First-choice dừng tại TRY_LIMIT sau bốn bước (325 lần đánh giá). Hai thuật toán quét toàn bộ lân cận có bằng chứng không còn phép di chuyển cải thiện tại điểm dừng, trong khi First-choice không có bằng chứng này chỉ từ điều kiện thử hết ngân sách.

Khi mở rộng lên 30 lượt chạy ngẫu nhiên, cả ba thuật toán đều chỉ giải thành công 5/30 trường hợp, tương ứng 16,67%. Mean final_h của First-choice là 1,167; Steepest-ascent là 1,300; Stochastic (all) là 1,267. Tỷ lệ thành công giảm rõ rệt so với n = 4, cho thấy việc chỉ chấp nhận các bước giảm h khiến phương pháp dễ bị giới hạn ở những trạng thái chưa phải nghiệm.

6.4.4. Bảng so sánh tỷ lệ thành công và số xung đột cuối

n	Thuật toán	Thành công	h cuối TB	Bước TB	Đánh giá TB	Thời gian TB (ms)
4	Steepest-ascent	40,00%	0,700	1,500	26,200	0,117
4	Stochastic (all)	36,67%	0,733	1,967	32,200	0,174

4	First-choice	46,67%	0,667	2,033	29,233	0,224
8	Steepest-ascent	16,67%	1,300	3,167	225,000	1,855
8	Stochastic (all)	16,67%	1,267	5,233	340,733	2,715
8	First-choice	16,67%	1,167	4,967	206,767	2,470

Bảng 6.2. Kết quả tổng hợp từ 180 lượt chạy Bài 6 (30 lượt/thuật toán/kích thước)

Các tỷ lệ trong Bảng 6.2 được tính bằng số lượt đạt h = 0 chia cho 30. Đối với n = 4, First-choice thành công nhiều hơn Steepest-ascent hai lượt; đối với n = 8, ba phương pháp có số lượt thành công bằng nhau. Không có bằng chứng từ dữ liệu hiện có rằng một thuật toán chiếm ưu thế về mọi tiêu chí hoặc mọi kích thước đầu vào.

6.4.5. So sánh số bước và số lần đánh giá hàm conflicts()

Khi n = 8, Steepest-ascent thực hiện trung bình 3,167 bước cải thiện và 225 lần đánh giá; Stochastic (all) thực hiện 5,233 bước và 340,733 lần đánh giá; First-choice thực hiện 4,967 bước và 206,767 lần đánh giá. Như vậy, Stochastic (all) phải đánh giá nhiều ứng viên hơn Steepest-ascent do đi qua nhiều bước hơn, còn First-choice không phải quét toàn bộ lân cận ở mỗi bước.

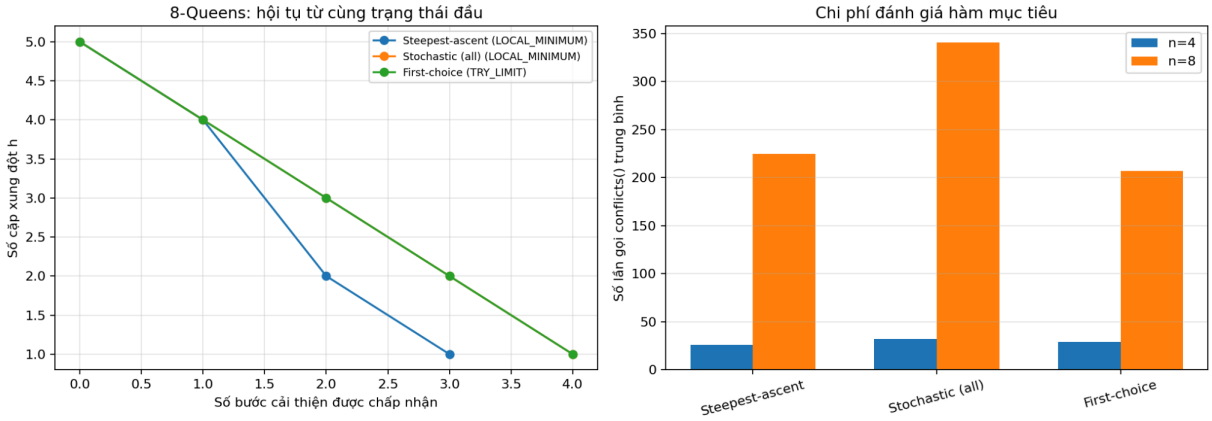
Tuy nhiên, số lần đánh giá ít hơn không đồng nghĩa thời gian luôn thấp hơn. Ở n = 8, First-choice có số lần đánh giá trung bình thấp hơn Steepest-ascent nhưng thời gian trung bình cao hơn (2,470 ms so với 1,855 ms). Chi phí sinh số ngẫu nhiên, tạo bản sao ứng viên và điều kiện kiểm tra trong từng vòng cũng góp phần tạo nên thời gian thực thi. Các số đo này cần được diễn giải trong phạm vi môi trường benchmark của notebook.

6.5. Phân tích hiệu quả các thuật toán

6.5.1. Trực quan hóa quá trình hội tụ

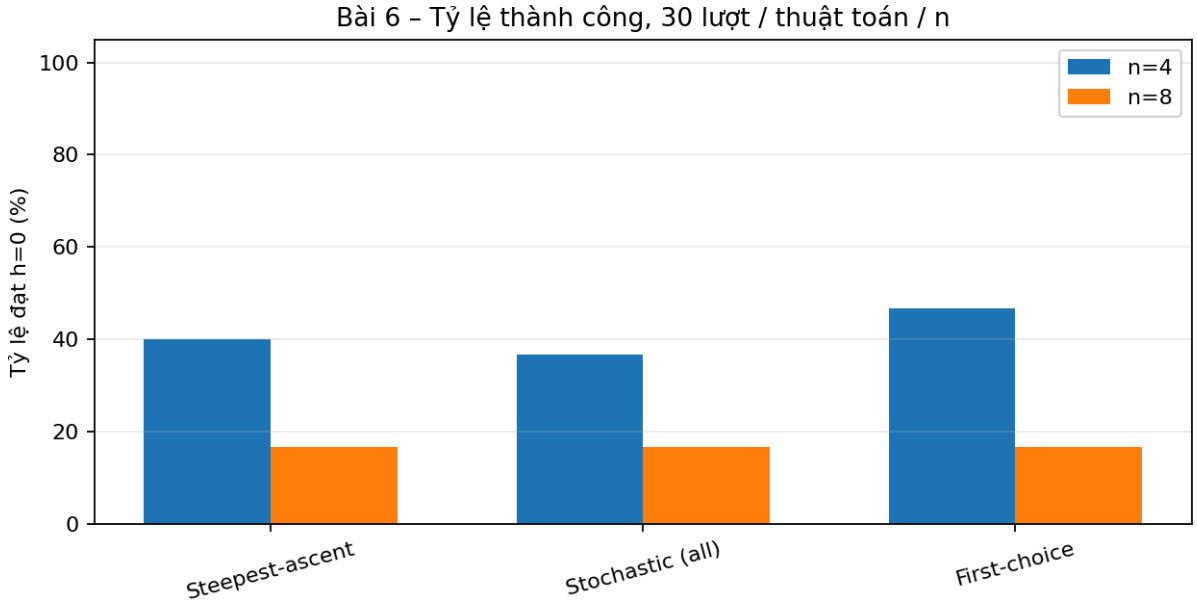
Hình 6.1 biểu diễn đồng thời sự thay đổi số cặp hậu xung đột theo số bước được chấp nhận và số lần gọi conflicts() trung bình theo phương pháp. Ở cùng trạng thái khởi tạo n =

8, ba đường cong đều bắt đầu tại h = 5 và kết thúc tại h = 1. Steepest-ascent giảm từ 5 xuống 1 sau ba bước, trong khi Stochastic (all) và First-choice cần bốn bước; nhưng không thuật toán nào đạt h = 0 ở lượt minh họa này.



Hình 6.1. Hội tụ trên cùng bàn cờ 8-Queens và chi phí đánh giá conflicts() trung bình.

Hình 6.2 trình bày tỷ lệ đạt nghiệm h = 0 qua 30 lượt chạy cho mỗi phương pháp và mỗi kích thước. Sự chênh lệch giữa n = 4 và n = 8 thể hiện rõ ràng, nhưng kích thước mẫu 30 lượt chưa đủ để khẳng định khác biệt nhỏ giữa các thuật toán là ổn định về mặt thống kê.



Hình 6.2. Tỷ lệ thành công của ba biến thể Hill Climbing ở n = 4 và n = 8.

6.5.2. Sự đánh đổi giữa chất lượng nghiệm và chi phí tính toán

Steepest-ascent ưu tiên mức cải thiện tốt nhất tại mỗi bước và nhờ đó thường có ít bước cải thiện hơn. Đổi lại, phương pháp phải đánh giá M ứng viên trên mỗi lượt mở rộng. Stochastic (all) cũng đánh giá toàn bộ lân cận nhưng lựa chọn ngẫu nhiên trong tập tốt hơn, có thể dẫn đến quỹ đạo dài hơn và nhiều lần đánh giá hơn. First-choice không lập danh sách đầy đủ mà thử từng ứng viên ngẫu nhiên; điều này có thể tiết kiệm số lượt đánh giá, nhưng đôi khi tăng thời gian do chi phí thao tác ngẫu nhiên và khả năng lặp lại những ứng viên đã thử.

Dữ liệu thực nghiệm cho thấy Steepest-ascent có thời gian trung bình thấp nhất ở cả n = 4 (0,117 ms) và n = 8 (1,855 ms). Tuy nhiên, ở n = 4, First-choice lại có tỷ lệ thành công cao nhất trong 30 lượt khảo sát. Do đó, nên đánh giá thuật toán trên nhiều tiêu chí đồng thời thay vì chỉ dựa vào số bước hoặc thời gian chạy.

6.5.3. Nguyên nhân dừng sớm tại cực tiểu cục bộ

Nguyên nhân trực tiếp của hiện tượng dừng sớm là chính sách chỉ chấp nhận h(mới) < h(hiện tại). Một trạng thái có h > 0 vẫn có thể không có lân cận nào làm giảm h. Trong bảng kiểm thử Bài 6, cả hai thuật toán quét toàn bộ lân cận đều dừng tại LOCAL_MINIMUM đối với một số bàn cờ; các điểm dừng này không có lân cận cải thiện trong phép kiểm tra độc lập. Một số trạng thái dừng vẫn có lân cận cùng giá trị h, minh họa tác động của plateau.

Riêng First-choice trả về TRY_LIMIT khi không gặp bước cải thiện trong ngân sách lấy mẫu. Trong các trường hợp kiểm thử lưu CSV, phép kiểm tra độc lập tại trạng thái dừng ghi nhận không có lân cận cải thiện, nhưng điều đó là thông tin xác minh bổ sung; bản thân nhãn TRY_LIMIT không chứng minh tính cực tiểu. Để khắc phục khả năng mắc kẹt,

các bài thực hành sau có thể áp dụng Random Restart hoặc Simulated Annealing thay vì chỉ duy trì quy tắc cải thiện nghiêm ngặt.

6.6. Nhận xét

Qua quá trình triển khai và thực nghiệm, cả ba biến thể Hill Climbing đều giải quyết bài toán n-Queens bằng cùng một biểu diễn vector, cùng hàm mục tiêu và cùng toán tử di chuyển một quân hậu trong cột. Khác biệt cốt lõi nằm ở cách tìm ứng viên cho bước tiếp theo: Steepest-ascent chọn ứng viên tốt nhất sau khi đánh giá đầy đủ; Inefficient Stochastic chọn ngẫu nhiên trong tập các ứng viên cải thiện đã được liệt kê; còn Efficient Stochastic chấp nhận ngay ứng viên ngẫu nhiên đầu tiên có cải thiện.

Bảng thống kê 180 lượt chạy cho thấy hiệu suất và chất lượng nghiệm phụ thuộc kích thước bài toán và đặc điểm khởi tạo. Ở n = 4, tỷ lệ thành công của ba phương pháp nằm trong khoảng 36,67–46,67%; ở n = 8, cả ba chỉ đạt 16,67%. Số lần đánh giá conflicts() không phản ánh trực tiếp thời gian thực thi, bởi mỗi thuật toán có chi phí phụ trợ khác nhau. Số bước cải thiện được chấp nhận cũng phải phân biệt với tổng số ứng viên được sinh hoặc đã đánh giá.

Bài thực hành đồng thời nhấn mạnh giới hạn của tối ưu hóa cục bộ với điều kiện giảm h nghiêm ngặt: thuật toán dễ dừng ở trạng thái chưa thỏa mãn mục tiêu khi không còn lân cận tốt hơn, hoặc khi phương pháp lấy mẫu đạt ngân sách cho phép. Việc ghi nhận rõ trạng thái SOLVED, LOCAL_MINIMUM, TRY_LIMIT và STEP_LIMIT giúp tránh nhầm lẫn giữa tìm được nghiệm, kiểm chứng cực tiểu cục bộ và việc dừng vì giới hạn tài nguyên. Những kết quả này tạo cơ sở thực nghiệm để tiếp tục khảo sát Random Restart và Simulated Annealing trong Chương 7.

CHƯƠNG 7: RANDOM RESTART VÀ SIMULATED ANNEALING

7.1. Hạn chế của Hill Climbing truyền thống

7.1.1. Sự phụ thuộc vào trạng thái khởi tạo

Trong bài toán n-Queens, một trạng thái được biểu diễn bằng vector board gồm n phần tử, trong đó board[i] biểu thị hàng đặt quân hậu ở cột i. Hàm mục tiêu h(board) đếm số cặp quân hậu tấn công nhau theo hàng hoặc đường chéo; bài toán yêu cầu tìm một cấu hình có h(board) = 0. Toán tử lân cận của các bài thực hành trước là chuyển một quân hậu sang hàng khác trong cùng cột, bảo đảm mỗi cột luôn có đúng một quân hậu. Một trạng thái có $n(n-1)$ lân cận ứng với cách sinh này.

Ba thuật toán Hill Climbing ở Bài 6 đều chỉ chấp nhận bước di chuyển cải thiện nghiêm ngặt, tức h(new) < h(current). Do đó, khi khởi tạo ở các vị trí khác nhau, thuật toán có thể đi theo những chuỗi bước cải thiện khác nhau và hội tụ tới các cấu hình cuối khác nhau. Một bàn cờ có số xung đột ban đầu thấp chưa chắc dẫn đến nghiệm hợp lệ nhanh hơn, vì hướng đi tiếp theo còn bị chi phối bởi cấu trúc lân cận và quy tắc chọn bước.

7.1.2. Khả năng mắc kẹt tại cực tiểu cục bộ

Cực tiểu cục bộ là trạng thái có giá trị hàm mục tiêu không lớn hơn mọi lân cận một bước, nhưng vẫn chưa đạt mục tiêu toàn cục h = 0. Nếu thuật toán đã xét hết các lân cận mà không tìm được giá trị nhỏ hơn, các biến thể Steepest-ascent và Stochastic (all) kết thúc với trạng thái LOCAL_MINIMUM. Trường hợp nhiều lân cận có giá trị bằng nhau tạo thành plateau; nếu chỉ cho phép giảm h nghiêm ngặt, thuật toán cũng không thể di chuyển ngang để thoát khỏi vùng này.

Với First-choice, việc không phát hiện bước cải thiện sau một số lần lấy mẫu được ghi là TRY_LIMIT. Trạng thái này chỉ thể hiện đã dùng hết ngân sách thử ngẫu nhiên, không đủ để chứng minh mọi lân cận đều không thể cải thiện. Vì thế, cần phân biệt dừng do kiểm tra đầy đủ lân cận với dừng do giới hạn mẫu.

7.1.3. Động cơ sử dụng Random Restart và Simulated Annealing

Random Restart và Simulated Annealing được sử dụng để giảm sự phụ thuộc vào một đường tìm kiếm cục bộ duy nhất. Random Restart thay đổi điểm xuất phát: nếu một lượt Hill Climbing dừng trước khi đạt nghiệm, thuật toán khởi tạo bàn cờ mới rồi tìm kiếm lại, đồng thời duy trì nghiệm tốt nhất từ các lượt trước. Ngược lại, Simulated Annealing không nhất thiết khởi động lại; nó cho phép chấp nhận một số bước làm tăng số xung đột để có khả năng vượt qua cực tiểu cục bộ. Hai cách tiếp cận đều mở rộng khả năng khám phá nhưng phải đánh đổi bằng thêm số lần đánh giá hàm mục tiêu và thời gian thực thi.

7.2. Thuật toán Random Restart

7.2.1. Nguyên lý khởi tạo lại trạng thái ngẫu nhiên

Random Restart tổ chức nhiều lượt Hill Climbing liên tiếp. Lượt đầu sử dụng chính bàn cờ đầu vào; các lượt sau sinh một vector ngẫu nhiên có n phần tử, mỗi phần tử thuộc tập hàng từ 0 đến n − 1. Mỗi lượt chạy độc lập về trạng thái bắt đầu, nhưng thuật toán bao ngoài lưu cấu hình tốt nhất từng tìm thấy. Quy trình kết thúc ngay nếu bất kỳ lượt nào đạt h = 0; nếu không, trả về cấu hình tốt nhất sau khi hết ngân sách.

Nếu xác suất một lượt độc lập tìm được nghiệm là p và có R lần khởi động lại sau lượt đầu, xác suất thành công lý tưởng là 1 − (1 − p)^(R + 1). Biểu thức này chỉ có giá trị minh họa khi giả định xác suất mỗi lượt ổn định và các lượt độc lập; nó không phải cam kết bảo đảm nghiệm cho mọi dữ liệu.

7.2.2. Triển khai Random Restart cho ba biến thể Hill Climbing

Notebook triển khai hàm random_restart_hill_climbing(board, method, seed, max_restarts, max_steps, first_choice_failure_limit). Tham số method lựa chọn một trong ba thuật toán ở Bài 6: steepest đánh giá toàn bộ lân cận và lấy bước tốt nhất; stochastic đánh giá toàn bộ lân cận rồi chọn ngẫu nhiên một bước cải thiện; first_choice lấy mẫu từng lân cận ngẫu nhiên và chấp nhận bước cải thiện đầu tiên. Hàm sử dụng bộ sinh ngẫu nhiên cục bộ numpy.random.default_rng(seed), nhờ đó kết quả nghiệm có thể được tái lập với seed và dữ liệu đầu vào tương ứng.

Các lượt chạy không sửa đổi bàn cờ mà người dùng truyền vào. Sau mỗi lượt, chương trình lấy result['final_h'], đối chiếu với best_h và cập nhật best_board nếu tìm được cấu hình tốt hơn. Cách tổ chức này giữ nguyên triển khai ba thuật toán gốc, chỉ bổ sung cơ chế điều phối nhiều điểm xuất phát.

7.2.3. Giới hạn số lần khởi động lại

Thí nghiệm thiết lập max_restarts = 100. Theo đúng cách cài đặt trong notebook, đây là tối đa 100 lần khởi tạo lại sau lần chạy đầu tiên, tương đương tối đa 101 lượt Hill Climbing. Hai đại lượng được lưu riêng: runs là tổng lượt đã thực hiện, còn restarts_used = runs − 1 là số lần khởi động lại thực tế. Mỗi lượt Hill Climbing có giới hạn max_steps = 1000, nhưng có thể dừng sớm khi gặp nghiệm hoặc điều kiện dừng riêng.

Nếu đã dùng hết số lượt mà chưa có h = 0, thuật toán trả trạng thái RESTART_LIMIT. Điều này phản ánh giới hạn tài nguyên của phép thử chứ không chứng minh bài toán vô nghiệm. Trong dữ liệu thí nghiệm n = 4 và n = 8 được cung cấp, ba biến thể Random Restart đều tìm được nghiệm trong số lượt cho phép.

7.2.4. Lưu trữ nghiệm tốt nhất và thống kê chi phí tìm kiếm

Kết quả Random Restart bao gồm final_h, board, solved, status, runs, restarts_used, steps, evaluations, elapsed_ms và lịch sử từng lượt restart_trace. Giá trị final_h là số xung đột của nghiệm tốt nhất, không đơn thuần là giá trị của lần khởi động cuối. steps cộng dồn các bước cải thiện được chấp nhận ở tất cả lượt; evaluations cộng dồn toàn bộ lần gọi conflicts() trong các lượt Hill Climbing, bao gồm việc đánh giá bàn cờ khởi tạo của mỗi lượt.

File Bai7_Restart_Trace.csv lưu cụ thể initial_h, final_h, best_h_so_far, evaluations, steps và trạng thái kết thúc theo từng lượt. Vì vậy có thể kiểm chứng rằng best_h_so_far không tăng trong quá trình khởi động lại, đồng thời quan sát mức chi phí phải trả để có một nghiệm hợp lệ.

7.3. Thuật toán Simulated Annealing

7.3.1. Nguyên lý hoạt động của Simulated Annealing

Simulated Annealing (SA) mô phỏng quá trình làm nguội: khi nhiệt độ cao, thuật toán có khả năng chấp nhận cả một số bước xấu để khám phá nhiều khu vực của không gian trạng thái; khi nhiệt độ giảm, xác suất này nhỏ đi và hành vi tìm kiếm dần thiên về các bước không làm tăng hàm mục tiêu. Trong notebook, mỗi vòng lặp tạo ngẫu nhiên một lân cận bằng đúng toán tử đổi hàng của một quân hậu trong cùng cột.

Khác với Hill Climbing, giá trị xung đột tại trạng thái hiện tại của SA có thể tăng hoặc giảm qua các vòng. Vì vậy, chương trình duy trì song song current_board, current_h và best_board, best_h. Giá trị best_h chỉ giảm hoặc giữ nguyên, và cấu hình tốt nhất này được trả về để đánh giá kết quả.

7.3.2. Quy tắc chấp nhận trạng thái tốt hơn và xấu hơn

Đặt $\Delta h = h(\text{candidate}) - h(\text{current})$, trong đó candidate là trạng thái lân cận được sinh ngẫu nhiên. Khi $\Delta h < 0$, trạng thái mới có ít xung đột hơn và được chấp nhận. Khi $\Delta h = 0$, triển khai trong notebook vẫn chấp nhận bước đi ngang; đây là một điểm khác biệt so với các biến thể Hill Climbing ở Bài 6. Khi $\Delta h > 0$, trạng thái làm tăng số xung đột chỉ được chấp nhận theo xác suất phụ thuộc nhiệt độ.

Quy tắc chấp nhận này cho phép SA di chuyển qua plateau và đôi khi vượt ra khỏi vùng cực tiểu cục bộ. Tuy nhiên, chấp nhận một bước xấu không đồng nghĩa thuật toán chắc chắn đạt nghiệm; hiệu quả còn phụ thuộc lịch nhiệt, số vòng lặp và yếu tố ngẫu nhiên.

7.3.3. Xác suất chấp nhận bước xấu phụ thuộc nhiệt độ

Với $\Delta h > 0$ và nhiệt độ $T > 0$, xác suất chấp nhận là $p = \exp(-\Delta h/T)$. Nhiệt độ T càng lớn thì xác suất chấp nhận một bước có mức tăng xung đột xác định càng cao. Với một bước xấu có $\Delta h = +1$ và $T = 3$, xác suất ban đầu bằng $\exp(-1/3) \approx 0{,}7165$, tức khoảng 71,65%.

Ở mỗi lần thử, thuật toán sinh một số ngẫu nhiên trong đoạn [0, 1) và chấp nhận bước xấu nếu số này nhỏ hơn p. Khi $T = 0$, notebook từ chối mọi bước $\Delta h > 0$ để tránh phép chia cho 0, nhưng vẫn chấp nhận các bước $\Delta h \leq 0$. Số lượt chấp nhận bước làm tăng h được ghi trong chỉ số accepted_worse.

7.3.4. Lịch làm nguội Exponential

Lịch hàm mũ được xác định bởi $T(t) = T_0 \times \alpha^t$, với $0 < \alpha < 1$. Notebook sử dụng $T_0 = 3$ và $\alpha = 0{,}995$, bắt đầu từ $t = 0$. Nhiệt độ giảm liên tục theo cấp số nhân; sau 500 vòng lặp, T còn xấp xỉ 0,245, thấp hơn nhiều so với nhiệt độ ban đầu. Với $\Delta h = +1$, xác suất chấp nhận khi đó chỉ còn khoảng 1,7%.

Đường biểu diễn cho thấy lịch Exponential giảm nhanh trong giai đoạn đầu và tiến dần về 0 nhưng không chạm 0 tại một thời điểm hữu hạn. Do đó, về cuối ngân sách, thuật toán gần như không chấp nhận bước xấu có độ tăng xung đột đáng kể.

7.3.5. Lịch làm nguội Linear

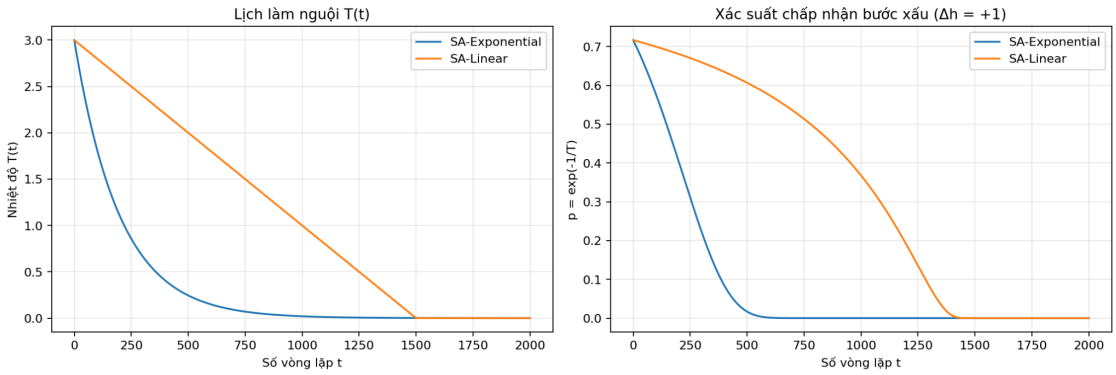
Lịch tuyến tính được xác định bởi $T(t) = \max(0, T_0 - \eta t)$, với $\eta > 0$. Trong notebook, $T_0 = 3$ và $\eta = 0{,}002$; vì vậy nhiệt độ bằng 2 tại vòng 500, bằng 1 tại vòng 1000, và về 0 từ vòng 1500. Với $\Delta h = +1$, xác suất chấp nhận tại vòng 500 xấp xỉ $\exp(-1/2) = 60{,}65\%$, cao hơn nhiều lịch Exponential ở cùng mốc thời gian.

Việc giữ nhiệt độ tương đối cao trong thời gian dài khiến SA-Linear có thể duy trì khả năng khám phá mạnh hơn nhưng cũng phát sinh nhiều bước đi làm tăng xung đột và làm chậm quá trình ổn định nghiệm. Khi nhiệt độ đạt 0, thuật toán không nhận bước xấu nữa nhưng vẫn tiếp tục thử lân cận cho tới khi đạt nghiệm hoặc hết ngân sách.

7.3.6. Điều kiện dừng và lưu nghiệm tốt nhất

Hàm simulated_annealing_queens() có ngân sách max_iterations = 2000 lượt thử lân cận. Thuật toán kết thúc ngay khi best_h = 0, gán trạng thái SOLVED. Nếu chưa tìm được nghiệm trong số lượt thử quy định, trạng thái là ITERATION_LIMIT, đồng thời vẫn trả về bàn cờ tốt nhất từng gặp. Kết quả còn lưu history_current, history_best và temperature_history để phân tích quá trình hội tụ.

Notebook quy ước evaluations = attempted + 1: có một lần gọi conflicts() để đánh giá trạng thái đầu và thêm một lần cho mỗi lân cận được thử. Chỉ số steps của SA là số bước được chấp nhận, kể cả các bước không cải thiện; do đó không thể so sánh trực tiếp với steps của Random Restart, vốn cộng các bước Hill Climbing cải thiện nghiêm ngặt.



Hình 7.1. So sánh nhiệt độ và xác suất chấp nhận bước xấu Δh = +1 của lịch Exponential và Linear với $T_0 = 3$.

7.4. Thiết lập thực nghiệm

7.4.1. Điều kiện chạy trên n = 4 và n = 8

Thí nghiệm sử dụng hai kích thước bàn cờ n = 4 và n = 8. Trên mỗi kích thước, năm cấu hình được so sánh gồm RR-Steepest, RR-Stochastic, RR-First-choice, SA-Exponential và SA-Linear. Mọi thuật toán trong cùng lượt khảo sát nhận cùng một bàn cờ khởi tạo, giúp hạn chế sai lệch do độ khó của trạng thái đầu. Tất cả đều dùng hàm conflicts() và cùng quy tắc sinh lân cận là di chuyển một quân hậu tới hàng khác trong cùng cột.

Ngoài lượt minh họa trên hai bàn cờ của Bài 5, notebook chạy 30 lượt ngẫu nhiên cho từng cấu hình và từng kích thước, tạo tổng cộng 2 × 5 × 30 = 300 lượt chạy. Đây là tập thống kê của Bài 7; yêu cầu ít nhất 100 lượt cho Task 6 sẽ được thực hiện riêng trong Bài 8, không được gộp hoặc ghi nhầm là đã hoàn thành tại chương này.

7.4.2. Thiết lập random seed và ngân sách tìm kiếm

Notebook sử dụng seed gốc 20261020. Các bàn cờ khảo sát được sinh từ một bộ sinh ngẫu nhiên cục bộ; từng lượt thuật toán cũng nhận seed được xác định từ kích thước, chỉ số lần chạy và loại thuật toán. Random Restart cho phép tối đa 100 lần khởi động lại sau

lần chạy đầu và tối đa 1000 bước cho mỗi lượt Hill Climbing. Simulated Annealing được giới hạn tối đa 2000 lượt thử lân cận, với $T_0 = 3$, $\alpha = 0{,}995$ và $\eta = 0{,}002$.

Các phương pháp không có cùng một ngân sách đánh giá hàm mục tiêu: một bên có thể lặp lại nhiều lượt Hill Climbing, bên kia thử tối đa 2000 lân cận. Vì vậy, kết quả cần được diễn giải dựa trên cả tỷ lệ thành công, tổng lần gọi hàm mục tiêu và thời gian, thay vì chỉ so sánh một chỉ số đơn lẻ.

7.4.3. Các chỉ số đánh giá thuật toán

Chương trình ghi lại initial_h, final_h, solved, status, steps, evaluations, elapsed_ms, runs, restarts_used và accepted_worse. Trong đó, final_h là h của nghiệm tốt nhất thuật toán trả về; solved nhận giá trị đúng khi final_h = 0; evaluations là số lần gọi conflicts() bên trong thuật toán; elapsed_ms là thời gian chạy tính bằng mili giây theo môi trường thực thi notebook. runs và restarts_used chỉ có ý nghĩa khởi động lại đối với RR; accepted_worse chỉ áp dụng thực chất cho SA.

Kết quả chi tiết được lưu trong Bai7_Trial_Results.csv, thống kê trung bình và tỷ lệ thành công trong Bai7_Summary_Results.csv, dữ liệu demo trong Bai7_Demo_Results.csv và các tệp lịch sử phục vụ trực quan hóa. Chỉ số thời gian có thể biến động khi chạy trên máy khác; mọi số trung bình dưới đây là kết quả ghi nhận từ bộ file được cung cấp, không phải thời gian dự đoán lý thuyết.

7.5. Kết quả thực nghiệm

7.5.1. Kết quả Random Restart trên n = 4 và n = 8

Kết quả tổng hợp cho thấy cả ba biến thể Random Restart đều đạt tỷ lệ tìm được nghiệm h = 0 là 100% trên n = 4 và n = 8 trong 30 lượt thử cho mỗi cấu hình. Với n = 4, số lần khởi động lại trung bình lần lượt là 2,03 (RR-Steepest), 1,90 (RR-Stochastic) và 1,37

(RR-First-choice). Với n = 8, các giá trị tương ứng tăng lên 5,70; 5,50 và 5,93. Điều này cho thấy khi kích thước bàn cờ tăng, chương trình thường cần nhiều điểm xuất phát hơn để tìm một nghiệm hoàn toàn không xung đột.

Bảng 7.1. Tổng hợp kết quả thực nghiệm trên 4-Queens (30 lượt cho mỗi thuật toán)

Phương pháp	Thành công	h cuối TB	Gọi conflicts() TB	Thời gian TB (ms)	Restart TB
RR-Steepest	30/30 (100%)	0,000	77,03	0,494	2,03
RR-Stochastic	30/30 (100%)	0,000	85,30	0,575	1,90
RR-First-choice	30/30 (100%)	0,000	72,73	0,730	1,37
SA-Exponential	30/30 (100%)	0,000	81,70	0,719	0
SA-Linear	30/30 (100%)	0,000	70,83	0,597	0

Bảng 7.2. Tổng hợp kết quả thực nghiệm trên 8-Queens (30 lượt cho mỗi thuật toán)

Phương pháp	Thành công	h cuối TB	Gọi conflicts() TB	Thời gian TB (ms)	Restart TB
RR-Steepest	30/30 (100%)	0,000	1556,03	12,433	5,70
RR-Stochastic	30/30 (100%)	0,000	2084,10	17,632	5,50
RR-First-choice	30/30 (100%)	0,000	1426,40	16,428	5,93
SA-Exponential	26/30 (86,67%)	0,133	1002,90	12,694	0
SA-Linear	20/30 (66,67%)	0,333	1717,27	21,241	0

7.5.2. Kết quả Simulated Annealing với hai lịch làm nguội

Ở n = 4, cả SA-Exponential và SA-Linear đều thành công 30/30 lượt. Ở n = 8, Exponential thành công 26/30 lượt và Linear thành công 20/30 lượt; các lượt chưa thành công dừng ở ITERATION_LIMIT khi hết 2000 lần thử. Giá trị h tốt nhất trung bình của hai cấu hình trên n = 8 tương ứng là 0,133 và 0,333. Trong tập thí nghiệm này, lịch Exponential đạt chất lượng nghiệm và tỷ lệ thành công cao hơn, nhưng chỉ có thể kết luận cho những seed, tham số nhiệt và ngân sách đã khảo sát.

Trên cùng bàn cờ minh họa 8-Queens với initial_h = 5, SA-Exponential đạt nghiệm h = 0 sau 443 lần đánh giá conflicts() và 214 bước được chấp nhận; SA-Linear kết thúc sau 2001 lần đánh giá, trả về nghiệm tốt nhất h = 1 và 821 bước được chấp nhận. Ví dụ này cho thấy một lịch duy trì nhiệt độ cao lâu hơn có thể khám phá nhiều trạng thái nhưng vẫn không bảo đảm đạt nghiệm trong ngân sách hữu hạn.

7.5.3. So sánh tỷ lệ đạt nghiệm h = 0

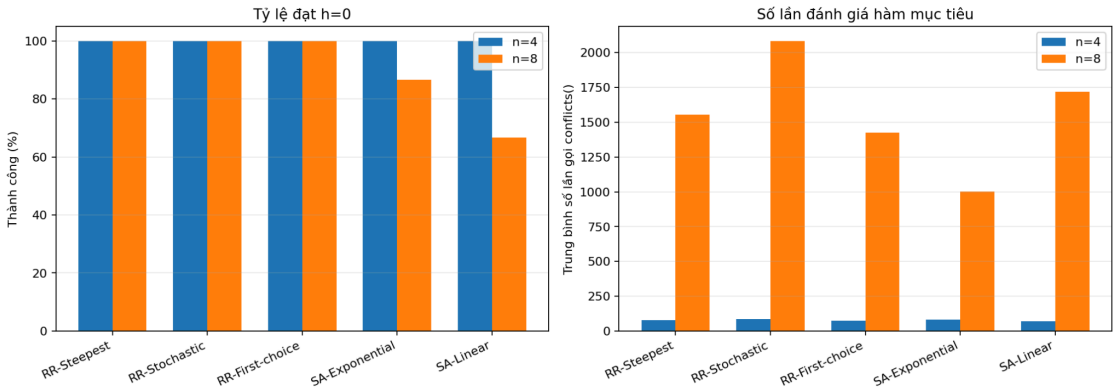
Tất cả năm cấu hình đều đạt 100% trên bài toán 4-Queens. Khi tăng lên n = 8, tỷ lệ thành công của ba biến thể Random Restart vẫn là 100%, trong khi SA-Exponential đạt 86,67% và SA-Linear đạt 66,67%. Chênh lệch giữa hai lịch SA là 20 điểm phần trăm (6/30 lượt), thể hiện hiệu quả quan sát được của việc làm nguội theo hàm mũ trong cấu hình hiện tại.

Cần lưu ý kết quả Random Restart sử dụng tới 101 lượt Hill Climbing, còn SA chỉ có một quá trình với giới hạn 2000 lần thử lân cận. Do ngân sách vận hành không giống nhau, biểu đồ thành công không thể được hiểu là bằng chứng Random Restart luôn có hiệu suất cao hơn nếu bị ràng buộc cùng số lần gọi hàm mục tiêu hoặc cùng thời gian.

7.5.4. So sánh số lần đánh giá và thời gian thực thi

Tại n = 8, RR-First-choice có số lần gọi conflicts() trung bình thấp nhất trong ba phương pháp Random Restart: 1426,40 lần; RR-Steepest là 1556,03 và RR-Stochastic là 2084,10 lần. Tuy nhiên, thời gian trung bình RR-Steepest (12,433 ms) lại thấp hơn RR-First-choice (16,428 ms), cho thấy số lần gọi hàm mục tiêu không phản ánh toàn bộ chi phí quản lý lân cận, lấy mẫu và các thao tác Python.

Đối với Simulated Annealing, SA-Exponential sử dụng trung bình 1002,90 lần gọi hàm mục tiêu và 12,694 ms, trong khi SA-Linear sử dụng 1717,27 lần và 21,241 ms. Sự khác biệt gắn với việc Exponential đạt nghiệm sớm hơn trong nhiều lượt và được phép dừng ngay, còn Linear có nhiều lượt kéo dài tới giới hạn. Những thời gian trên là phép đo quan sát trong notebook và không phải đánh giá benchmark độc lập đa môi trường.

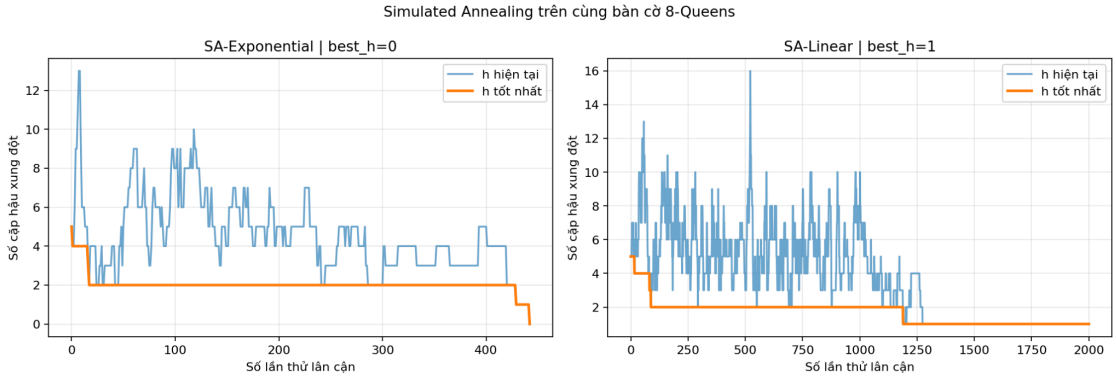


Hình 7.2. Tỷ lệ đạt h = 0 và số lần đánh giá `conflicts()` trung bình của năm cấu hình trên n = 4 và n = 8.

7.5.5. Trực quan hóa quá trình hội tụ của Simulated Annealing

Biểu đồ hội tụ trên cùng bàn cờ 8-Queens thể hiện hai đường riêng biệt: h hiện tại và h tốt nhất. Đường h hiện tại dao động do SA cho phép chấp nhận cả bước làm tăng số xung đột, đặc biệt trong giai đoạn nhiệt độ còn cao. Trong khi đó, đường h tốt nhất giảm theo từng bậc hoặc giữ nguyên, xác nhận cơ chế ghi nhớ nghiệm tốt nhất trong suốt quá trình.

Ở lịch Exponential, đường nghiệm tốt nhất lần lượt giảm và cuối cùng chạm h = 0 ở khoảng 442 lần thử lân cận của lượt minh họa. Với Linear, mặc dù phát sinh nhiều dao động trong quá trình, nghiệm tốt nhất chỉ đạt h = 1 vào giai đoạn sau của phép thử rồi giữ nguyên tới giới hạn 2000 lượt. Đây là minh chứng trực quan cho sự khác biệt giữa khả năng khám phá không gian tìm kiếm và hiệu quả hội tụ thực tế.



Hình 7.3. Diễn biến h hiện tại và h tốt nhất của hai lịch Exponential và Linear trong một lượt minh họa 8-Queens.

7.6. Nhận xét

Bài thực hành cho thấy Random Restart và Simulated Annealing xử lý giới hạn của Hill Climbing theo hai hướng khác nhau. Random Restart giữ nguyên quy tắc chỉ chấp nhận bước cải thiện trong mỗi lượt, nhưng sử dụng nhiều trạng thái khởi tạo để có cơ hội vượt qua sự phụ thuộc vào một cực tiểu cục bộ cụ thể. Simulated Annealing duy trì một chuỗi tìm kiếm linh hoạt hơn, cho phép bước xấu theo xác suất nhiệt độ và ghi nhớ nghiệm tốt nhất để tránh đánh mất kết quả đã tìm được.

Trong 300 lượt chạy của Bài 7, Random Restart đạt 100% thành công cho cả n = 4 và n = 8 ở ba biến thể. Simulated Annealing cũng thành công hoàn toàn trên n = 4, nhưng tại n = 8 cho kết quả khác nhau giữa Exponential (86,67%) và Linear (66,67%). Việc Exponential đạt tỷ lệ cao hơn trong thí nghiệm này không đồng nghĩa rằng lịch nhiệt hàm mũ luôn tốt hơn lịch tuyến tính; cần khảo sát thêm các tham số, seed, quy mô lớn hơn và ngân sách tương đương để rút ra kết luận tổng quát.

Một điểm phương pháp luận quan trọng là không đồng nhất số bước được chấp nhận của Random Restart với số bước được chấp nhận của SA. Các bước trong Hill Climbing luôn làm giảm h, còn SA có thể bao gồm cả bước ngang hoặc bước xấu. Vì vậy, khi đánh giá công sức tìm kiếm, số lần gọi conflicts() và thời gian thực thi là hai chỉ số bổ sung cần được xem cùng tỷ lệ đạt h = 0 và giá trị h cuối. Đồng thời, trạng thái RESTART_LIMIT hay ITERATION_LIMIT chỉ biểu thị hết ngân sách, không khẳng định bài toán không có nghiệm.

Từ những kết quả này có thể nhận thấy việc bổ sung cơ chế thoát khỏi tình trạng dừng sớm cải thiện khả năng tìm nghiệm n-Queens so với chỉ áp dụng Hill Climbing từ một trạng thái xuất phát. Tuy nhiên, mỗi phương pháp có chi phí và độ nhạy tham số riêng.

Kết quả Chương 7 tạo cơ sở để tiếp tục đánh giá ít nhất 100 lần chạy và khảo sát khả năng mở rộng trong Bài 8, nơi các so sánh được thực hiện với phạm vi thống kê rộng hơn.

CHƯƠNG 8: ĐÁNH GIÁ TỔNG HỢP VÀ BÀI NÂNG CAO

8.1. Mục tiêu và phạm vi thực nghiệm tổng hợp

8.1.1. Các thuật toán được lựa chọn để so sánh

Sau khi mô hình hóa bài toán n-Queens và hoàn thiện các phương pháp tìm kiếm cục bộ ở những bài trước, phần thực hành này tiến hành đánh giá tổng hợp để xác định mức độ phù hợp của từng phương pháp trong các điều kiện thực nghiệm khác nhau. Một trạng thái được biểu diễn bằng vector board gồm n phần tử, trong đó board[i] là hàng đặt quân hậu tại cột i. Hàm mục tiêu h(board) = conflicts(board) trả về số cặp quân hậu có thể tấn công nhau; nhiệm vụ là tìm cấu hình có h = 0.

Task 6 đối chiếu tám cấu hình: Steepest-ascent, Stochastic (all), First-choice, ba phiên bản tương ứng có Random Restart (RR-Steepest, RR-Stochastic, RR-First-choice), cùng Simulated Annealing với lịch giảm nhiệt Exponential và Linear. Ba biến thể Hill Climbing chỉ nhận bước đi làm giảm số xung đột. Random Restart khởi chạy lại Hill Climbing từ nhiều bàn cờ khác nhau, còn Simulated Annealing có thể chấp nhận bước xấu hơn với xác suất tùy thuộc vào nhiệt độ.

8.1.2. Thiết lập dữ liệu và random seed

Notebook sử dụng seed gốc SEED_B8 = 20261020, tạo số ngẫu nhiên bằng numpy.random.default_rng. Bộ dữ liệu Task 6 gồm 100 bàn cờ khởi tạo phân biệt cho mỗi giá trị n bằng 4 và 8. Mỗi thuật toán nhận đúng cùng một bàn cờ đầu vào ở cùng cặp (n, trial); seed riêng của từng thuật toán được tách ra nhằm tái lập các quyết định ngẫu nhiên.

Với cách tổ chức này, khác biệt giữa các phương pháp không xuất phát từ việc mỗi phương pháp được thử trên một bộ trạng thái đầu khác nhau.

Hai khảo sát bổ sung được thực hiện tách biệt. Khảo sát khả năng mở rộng sử dụng n thuộc tập {4, 8, 12, 16, 20} với 25 lượt cho mỗi cấu hình. Phần nâng cao sử dụng 100 bàn cờ khởi tạo phân biệt ở mỗi n bằng 8 và 12, so sánh năm toán tử lân cận. Không gộp số liệu ba khảo sát khi so sánh tỷ lệ thành công hay thời gian thực thi.

8.1.3. Số lần chạy và ngân sách thực nghiệm

Trong Task 6, mỗi thuật toán có 100 lượt trên mỗi kích thước bàn cờ, tương ứng 2 × 8 × 100 = 1.600 lượt chạy. Một lần chạy của Hill Climbing đơn lượt giới hạn tối đa 1.000 bước cải thiện. Với Random Restart, giới hạn là tối đa 100 lần khởi động lại sau lượt chạy ban đầu, tức tối đa 101 lượt Hill Climbing. Với Simulated Annealing, mỗi lần chạy được phép tối đa 2.000 lần thử trạng thái lân cận. Hai lịch nhiệt có cùng T0 = 3; lịch Exponential dùng alpha = 0,995, còn lịch Linear dùng eta = 0,002.

Trong khảo sát khả năng mở rộng, mỗi cấu hình được chạy 25 lượt, tổng cộng 5 × 3 × 25 = 375 lượt. Simulated Annealing trong khảo sát này chỉ có ngân sách 500 lần thử, khác với 2.000 lần thử của Task 6. Thực nghiệm nâng cao bao gồm 2 × 5 × 100 = 1.000 lượt; giới hạn tối đa 400 lần thử và 120 lần liên tiếp không cải thiện với n = 8, tương ứng 700 và 250 với n = 12. Việc ghi rõ ngân sách là cần thiết vì tỷ lệ tìm nghiệm phụ thuộc không chỉ vào thuật toán mà còn vào số cơ hội tìm kiếm.

8.1.4. Các chỉ số đánh giá hiệu suất

Một lượt được xem là thành công khi nghiệm trả về có h = 0. Tỷ lệ thành công được tính bằng số lượt thành công chia cho tổng số lượt, nhân 100%. Chất lượng nghiệm còn được đánh giá qua số xung đột cuối trung bình (mean_final_h): chỉ số càng thấp thì cấu hình

thu được càng ít xung đột. Với Random Restart và Simulated Annealing, final_h là giá trị tốt nhất từng tìm thấy, không nhất thiết là trạng thái cuối của toàn bộ quá trình.

Chi phí thực thi được đo bằng số lần gọi conflicts() trong thuật toán (evaluations), số bước (steps) và thời gian đo bằng perf_counter quy đổi sang mili giây. Tuy nhiên, steps của Hill Climbing phản ánh các bước cải thiện được chấp nhận, Random Restart cộng số bước của nhiều lượt khởi chạy, còn Simulated Annealing đếm cả bước được chấp nhận dù có thể làm tăng h. Vì vậy, không nên so sánh trực tiếp số bước giữa các phương pháp nếu chưa chuẩn hóa ý nghĩa. Số lần đánh giá hàm mục tiêu và thời gian chạy được dùng làm chỉ số bổ sung để giải thích chi phí.

8.2. Kết quả thực nghiệm tổng hợp trên n-Queens

8.2.1. Kết quả Task 6 trên n = 4

Ở kích thước n = 4, số xung đột khởi tạo trung bình trên 100 bàn cờ là 3,27. Ba thuật toán Hill Climbing đơn lượt có tỷ lệ giải thành công từ 27% đến 38%. Steepest-ascent đạt 38%, Stochastic (all) đạt 34% và First-choice đạt 27%. Điều này cho thấy ngay ở bài toán nhỏ, không phải mọi điểm xuất phát đều có đường đi giảm h liên tục đến nghiệm hợp lệ.

Các phương pháp Random Restart và cả hai lịch Simulated Annealing đều tìm được nghiệm h = 0 trong toàn bộ 100 lượt. Về thời gian trung bình, RR-Steepest cần khoảng 0,514 ms, RR-Stochastic 2,152 ms và RR-First-choice 3,100 ms; SA-Exponential và SA-Linear lần lượt cần 1,199 ms và 1,366 ms. Trong khi đó, Steepest-ascent đơn lượt chỉ mất khoảng 0,182 ms nhưng chỉ thành công 38 lượt. Đây là biểu hiện rõ ràng của đánh đổi giữa chi phí thấp cho một lượt tìm kiếm và khả năng đạt được nghiệm.

8.2.2. Kết quả Task 6 trên n = 8

Khi tăng kích thước lên n = 8, số xung đột khởi tạo trung bình tăng thành 8,09. Tỷ lệ thành công của Hill Climbing đơn lượt giảm mạnh: Steepest-ascent 12%, Stochastic (all) 11% và First-choice 11%. Ba phương pháp này vẫn thường cải thiện đáng kể nghiệm so với ban đầu, nhưng cuối cùng dừng ở cấu hình còn khoảng 1,31–1,32 cặp xung đột trung bình.

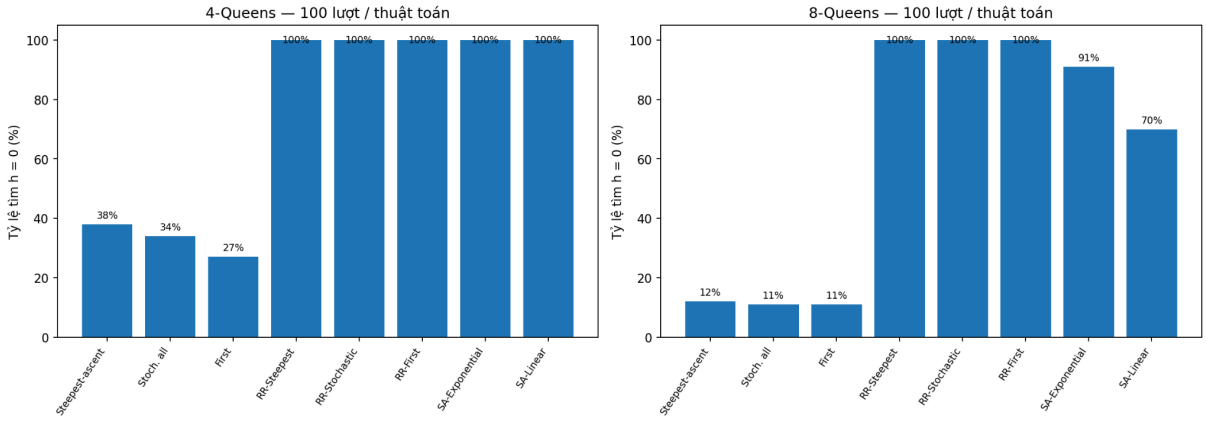
Ba biến thể Random Restart tiếp tục đạt 100% trong tập 100 lượt được khảo sát. Tuy nhiên, trung bình RR-Steepest cần 1.840,43 lần đánh giá hàm mục tiêu; RR-Stochastic cần 2.237,35 và RR-First-choice cần 1.904,01. Trong khi đó, SA-Exponential đạt tỷ lệ 91% với 890,71 lần đánh giá trung bình và 13,025 ms; SA-Linear đạt 70% với 1.606,21 lần đánh giá và 22,354 ms. Các số liệu cho thấy SA-Exponential sử dụng ít lần đánh giá hơn RR-Steepest nhưng cũng có tỷ lệ thành công thấp hơn trong thí nghiệm này.

8.2.3. Bảng so sánh tỷ lệ thành công và h cuối trung bình

Bảng 8.1 tổng hợp hai chỉ số phản ánh khả năng giải được bài toán và chất lượng nghiệm sau khi thuật toán kết thúc. Giá trị h cuối trung bình bằng 0 chứng tỏ toàn bộ các nghiệm trả về trong tập thử đều không còn xung đột, còn tỷ lệ thành công dưới 100% không có nghĩa mọi lượt thất bại đều giữ nguyên trạng thái ban đầu.

Bảng 8.1. Tỷ lệ đạt nghiệm và giá trị h cuối trung bình trong Task 6

Thuật toán	n = 4: thành công (%)	n = 4: h cuối TB	n = 8: thành công (%)	n = 8: h cuối TB
Steepest-ascent	38	0,71	12	1,31
Stochastic (all)	34	0,81	11	1,31
First-choice	27	0,94	11	1,32
RR-Steepest	100	0,00	100	0,00
RR-Stochastic	100	0,00	100	0,00
RR-First-choice	100	0,00	100	0,00
SA-Exponential	100	0,00	91	0,09
SA-Linear	100	0,00	70	0,30

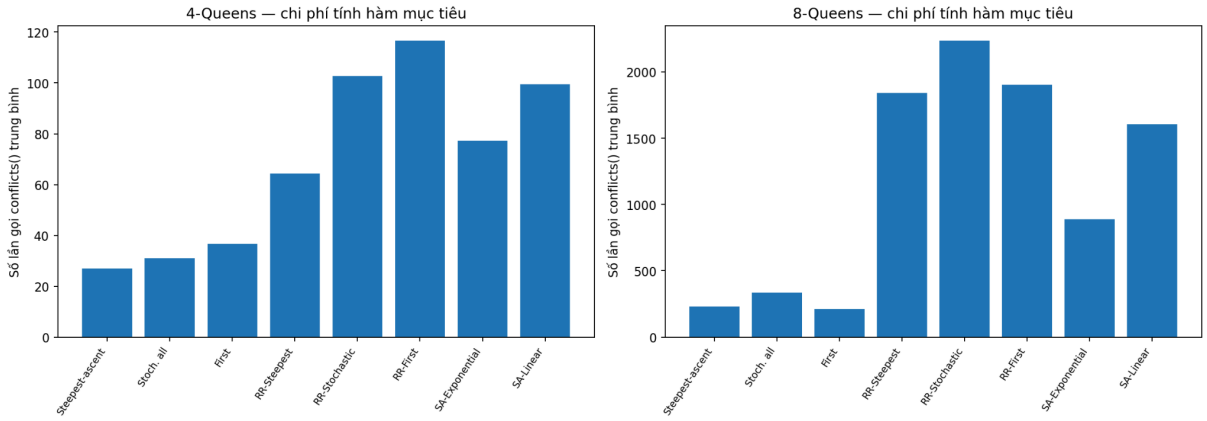


Hình 8.1. So sánh tỷ lệ đạt nghiệm h = 0 của tám cấu hình thuật toán với 100 lượt chạy mỗi cấu hình.

8.2.4. So sánh số bước và số lần đánh giá hàm mục tiêu

Dữ liệu số lần đánh giá phản ánh mức độ công việc thực tế mà mỗi phương pháp thực hiện. Ở n = 4, Steepest-ascent cần 26,92 lần đánh giá trung bình, Stochastic (all) 31,12 và First-choice 36,72. Các giá trị tương ứng ở n = 8 lần lượt tăng lên 230,04; 334,20 và 212,23. Dù cùng dùng hàm mục tiêu và toán tử thay đổi vị trí một quân hậu, cách lựa chọn lân cận làm số lần gọi conflicts() khác nhau.

Với n = 8, Random Restart phát sinh số lần đánh giá cao hơn nhiều, từ khoảng 1.840 đến 2.237 lần trung bình, do lặp lại các lượt tìm kiếm từ điểm xuất phát mới. Hai lịch SA cần 890,71 và 1.606,21 lần đánh giá. Điều này lý giải vì sao không thể xem tỷ lệ thành công 100% của Random Restart là kết quả không có chi phí. Ngoài ra, số bước trung bình của SA-Linear đạt 806,31, cao hơn SA-Exponential (238,17), nhưng các bước này không tương đương với bước giảm xung đột của Hill Climbing.



Hình 8.2. So sánh chi phí đánh giá hàm mục tiêu trên n = 4 và n = 8.

8.2.5. So sánh thời gian thực thi giữa các thuật toán

Bảng 8.2 trình bày thời gian trung bình và chi phí đánh giá từ dữ liệu Task 6. Đơn vị ms biểu thị mili giây mỗi lượt chạy, tính cả toàn bộ các lần restart nếu thuật toán dùng Random Restart.

Bảng 8.2. Thời gian và số lần đánh giá trung bình trong Task 6

Thuật toán	n=4: thời gian (ms)	n=4: evaluations	n=8: thời gian (ms)	n=8: evaluations
Steepest-ascent	0,182	26,92	1,917	230,04
Stochastic (all)	0,223	31,12	2,998	334,20
First-choice	0,485	36,72	2,781	212,23
RR-Steepest	0,514	64,39	16,321	1.840,43
RR-Stochastic	2,152	102,75	21,946	2.237,35
RR-First-choice	3,100	116,72	25,779	1.904,01
SA-Exponential	1,199	77,21	13,025	890,71
SA-Linear	1,366	99,49	22,354	1.606,21

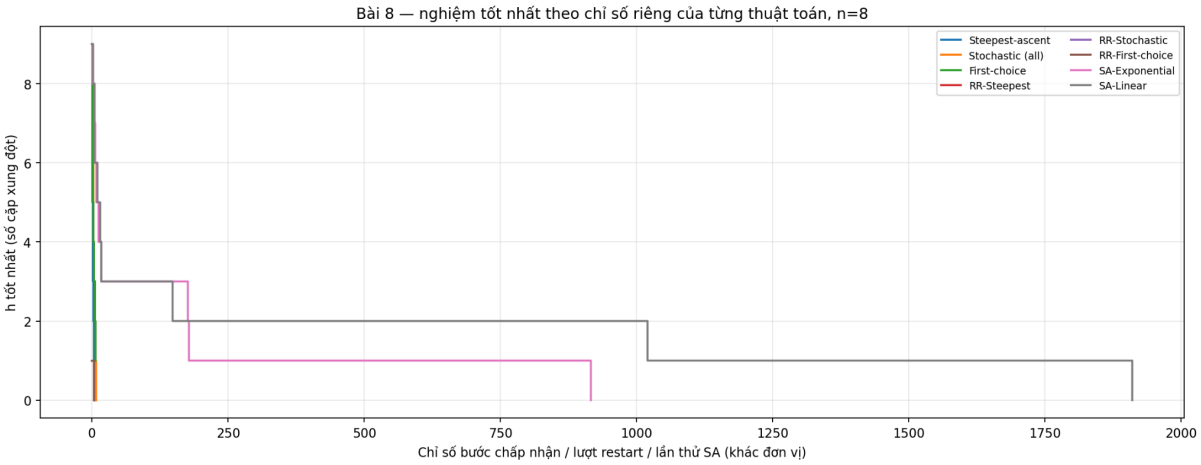
Ở n = 8, Steepest-ascent đơn lượt là thuật toán có thời gian trung bình thấp nhất trong bảng, nhưng chỉ giải thành công 12% số lượt. RR-Steepest có thời gian 16,321 ms và đạt 100% thành công, trong khi SA-Exponential nhanh hơn theo trung bình thời gian (13,025 ms) nhưng chỉ đạt 91% thành công. Do phụ thuộc vào phần cứng, môi trường Python và

nhiễu khi đo các tác vụ rất ngắn, thời gian ms ở đây mang ý nghĩa quan sát trên lần benchmark cụ thể, không nên xem là hằng số cố định.

8.3. Phân tích quá trình hội tụ

8.3.1. Trực quan hóa đường hội tụ trên cùng trạng thái khởi tạo

Nhằm quan sát hành vi tìm kiếm, notebook sử dụng một bàn cờ n = 8 chung cho cả tám thuật toán và vẽ lịch sử giá trị h tốt nhất. Trên biểu đồ, trục hoành được tính theo quy ước riêng: Hill Climbing dùng số bước cải thiện đã chấp nhận, Random Restart dùng lượt khởi động và Simulated Annealing dùng lần thử lân cận. Bởi vậy, chiều dài các đường trên trục hoành không cho phép so sánh trực tiếp tốc độ hoặc chi phí giữa các thuật toán.



Hình 8.3. Giá trị h tốt nhất theo chỉ số tiến trình riêng của từng thuật toán trên cùng bàn cờ n = 8.

8.3.2. Khả năng cải thiện trong giai đoạn đầu

Đường hội tụ cho thấy ba biến thể Hill Climbing thường làm giảm h nhanh trong vài bước được chấp nhận đầu tiên. Nguyên nhân là khi trạng thái còn nhiều cặp xung đột, không gian lân cận thường chứa một số nước di chuyển đem lại cải thiện rõ rệt. Steepest-ascent chọn nước giảm h tốt nhất sau khi khảo sát toàn bộ lân cận nên thường tiến gần đến một

cực tiểu cục bộ với ít bước cải thiện được chấp nhận. Tuy nhiên, mỗi bước như vậy lại có thể tốn nhiều lần đánh giá hàm mục tiêu.

Stochastic (all) và First-choice có thể cần nhiều bước hơn vì không nhất thiết chọn ứng viên làm giảm h nhiều nhất. Số bước ít hoặc đường cong giảm nhanh trên hình không đồng nghĩa tổng thời gian ngắn, bởi hai yếu tố quan trọng khác là số láng giềng đã thử và chi phí đánh giá chúng.

8.3.3. Hiện tượng Plateau và Local Optimum

Khi thuật toán không còn tìm được láng giềng có h nhỏ hơn, Hill Climbing có thể dừng dù h vẫn lớn hơn 0. Đây là hiện tượng mắc kẹt tại cực tiểu cục bộ xét theo tập các nước đi được phép. Với Steepest-ascent và Stochastic (all), trạng thái LOCAL_MINIMUM được ghi nhận khi đã kiểm tra tập lân cận nhưng không tìm thấy cải thiện; còn First-choice có thể trả về TRY_LIMIT vì chỉ lấy mẫu ngẫu nhiên một số ứng viên hữu hạn. Hai trạng thái dừng này không hoàn toàn tương đương.

Plateau là vùng có nhiều trạng thái cùng giá trị h. Khi chỉ chấp nhận các bước giảm nghiêm ngặt, thuật toán không thể đi ngang qua vùng bằng phẳng ngay cả khi một chuỗi di chuyển bằng điểm có thể mở ra hướng giảm h trong tương lai. Trên hình hội tụ, một đoạn đường h tốt nhất nằm ngang cũng có thể xuất phát từ việc chưa tìm thấy nghiệm tốt hơn trong khi thuật toán vẫn đang thử các bước; do đó, không phải mọi đoạn nằm ngang đều chứng minh đã mắc kẹt ở plateau của trạng thái hiện tại.

8.3.4. Ảnh hưởng của Random Restart và Simulated Annealing đến hội tụ

Random Restart khắc phục sự phụ thuộc vào một điểm xuất phát bằng cách tạo nhiều bàn cờ ngẫu nhiên mới. Giá trị h tốt nhất qua các lượt restart không tăng, vì thuật toán giữ lại

nghiệm tốt nhất từng gặp. Cách này mở rộng khả năng thăm dò sang nhiều vùng của không gian trạng thái, song làm tăng chi phí nếu phải thử nhiều lần trước khi đạt h = 0.

Simulated Annealing duy trì hai giá trị khác nhau: h của trạng thái hiện tại và h tốt nhất đã tìm thấy. Trạng thái hiện tại có thể trở nên xấu hơn do cơ chế chấp nhận bước tăng xung đột, trong khi h tốt nhất vẫn không tăng. Trong hình 8.3, đường được trình bày là h tốt nhất nên chỉ thể hiện các mốc cải thiện, không cho thấy đầy đủ những dao động lên xuống của trạng thái hiện tại. Lịch nhiệt và ngân sách lần thử ảnh hưởng mạnh đến khả năng vượt qua các vùng tìm kiếm kém thuận lợi.

8.4. Khảo sát khả năng mở rộng

8.4.1. Thiết lập thực nghiệm với n = 4, 8, 12, 16, 20

Để kiểm tra xu hướng thời gian khi bài toán lớn hơn, phần thực hành chọn ba đại diện gồm Steepest-ascent, First-choice và SA-Exponential. Mỗi thuật toán được chạy 25 lần tại từng kích thước n thuộc {4, 8, 12, 16, 20}. Các phương pháp dùng chung bàn cờ theo từng cặp (n, trial), nhưng đây là bộ khởi tạo riêng của khảo sát khả năng mở rộng. Đặc biệt, ngân sách SA trong phần này chỉ giới hạn 500 lần thử một lượt, nên tỷ lệ thành công không thể so sánh trực tiếp với kết quả SA sử dụng 2.000 lần thử ở Task 6.

8.4.2. Kết quả thời gian thực thi theo kích thước bàn cờ

Bảng 8.3 cho thấy trung vị thời gian chạy ở từng kích thước. Trung vị được ưu tiên sử dụng trong đồ thị nhằm hạn chế ảnh hưởng của những lượt đo có thời gian bất thường so với đa số lần chạy.

Bảng 8.3. Trung vị thời gian thực thi trong khảo sát khả năng mở rộng (ms/lượt)

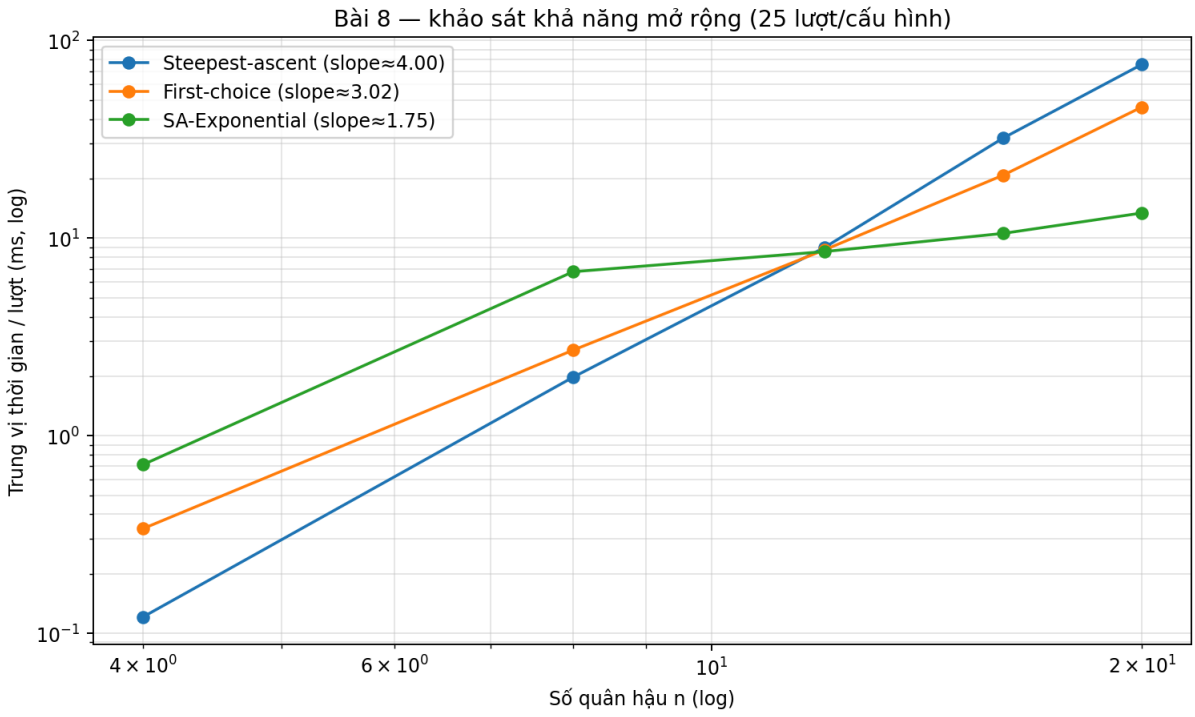
n	Steepest-ascent	First-choice	SA-Exponential
4	0,121	0,338	0,713
8	1,977	2,710	6,756

12	8,978	8,726	8,548
16	32,106	20,831	10,563
20	75,604	46,021	13,392

Khi n tăng từ 4 lên 20, thời gian trung vị của Steepest-ascent tăng mạnh và đến n = 20 là cao nhất trong ba thuật toán. First-choice cũng tăng theo kích thước nhưng có thời gian nhỏ hơn Steepest-ascent ở những kích thước lớn trong bảng. SA-Exponential tăng chậm hơn về thời gian ở phần cuối dải n, một phần vì số lần thử được giới hạn cố định và thuật toán thường sử dụng hết ngân sách 500 lần thử tại n lớn.

8.4.3. Biểu đồ log-log thời gian chạy

Đồ thị log-log biểu diễn đồng thời logarit của n và logarit thời gian trung vị nhằm trực quan hóa xu hướng tăng theo quy mô. Đường thẳng gần đúng được xác định bằng hồi quy tuyến tính trên các cặp (log n, log time). Trong phạm vi các điểm đo, độ dốc thực nghiệm khoảng 4,00 đối với Steepest-ascent, 3,02 đối với First-choice và 1,75 đối với SA-Exponential.



Hình 8.4. Khảo sát khả năng mở rộng với 25 lượt cho mỗi thuật toán và mỗi kích thước n.

8.4.4. Phân tích độ dốc thực nghiệm và chi phí cài đặt

Độ dốc lớn hơn cho thấy thời gian trong dải đo tăng nhanh hơn khi n tăng. Tuy nhiên, không thể đồng nhất hệ số dốc 4,00 hoặc 3,02 với một chứng minh rằng độ phức tạp lý thuyết chính xác lần lượt là $O(n^4)$ hay $O(n^3)$. Các giá trị này chịu ảnh hưởng của số bước tìm kiếm thực tế, kích thước tập lân cận, cách cài đặt hàm conflicts(), chi phí Python và giới hạn ngân sách.

Với toán tử đổi hàng của một quân hậu, một trạng thái có $n(n-1)$ lân cận khác nhau. Steepest-ascent duyệt hết tập lân cận mỗi vòng nên chi phí cho một bước có thể tăng đáng kể khi n lớn, đặc biệt nếu hàm đánh giá mỗi ứng viên còn cần xử lý theo n. First-choice chỉ lấy mẫu đến khi gặp cải thiện nhưng số lần thất bại trước khi tìm được bước tốt cũng có thể tăng. SA dùng một láng giềng ngẫu nhiên mỗi lần thử, vì vậy khi số lần thử tối đa bị cố định, mức tăng thời gian có thể thấp hơn, song chất lượng nghiệm không được bảo đảm duy trì.

8.4.5. Đánh giá khả năng mở rộng của các thuật toán

Bảng dữ liệu mở rộng cho thấy SA-Exponential chỉ có tỷ lệ thành công 16% ở n = 8 và không giải được lượt nào ở n = 12, 16 và 20 trong ngân sách 500 lần thử. Trong khi đó, Steepest-ascent đạt 8% tại n = 8, 12% tại n = 16 và 4% tại n = 20. Các tỷ lệ nhỏ biến động giữa kích thước là điều có thể xảy ra với mẫu 25 lượt, không nên diễn giải là phương pháp đột nhiên tốt hơn khi bài toán lớn hơn.

Do đó, SA có đường thời gian tăng chậm hơn trong thí nghiệm này chưa đồng nghĩa SA có khả năng mở rộng tốt nhất xét theo tỷ lệ tìm ra nghiệm hợp lệ. Đánh giá khả năng mở rộng cần đi kèm chất lượng nghiệm, tỷ lệ thành công và một ngân sách tìm kiếm được ghi rõ.

8.5. Phần thực nghiệm nâng cao về toán tử lân cận

8.5.1. Các toán tử di chuyển được khảo sát

Phần Advanced Task giữ nguyên cách biểu diễn mỗi cột một quân hậu và sử dụng nguyên tắc First-choice: sinh một ứng viên, tính số xung đột, chỉ chấp nhận nếu h giảm. Có năm toán tử được so sánh. random-row chuyển một quân hậu sang hàng khác bất kỳ trong cùng cột, là cách di chuyển cơ sở. single-step chỉ đưa hậu lên hoặc xuống một hàng nếu còn nằm trong bàn cờ. column-swap hoán đổi vị trí hàng của hai quân hậu ở hai cột. dual-queen thay đổi hàng của hai quân hậu đồng thời. adaptive ưu tiên chọn quân đang chịu nhiều xung đột nhất; theo notebook, 75% lần thử chỉ đổi hàng quân được chọn và 25% lần thử thay đổi thêm một quân khác.

8.5.2. Ảnh hưởng của toán tử đến không gian trạng thái lân cận

Toán tử tác động trực tiếp đến số ứng viên có thể tiếp cận trong một bước. Với random-row, số lân cận là $n(n-1)$, tương ứng 56 khi $n = 8$ và 132 khi $n = 12$. single-step có số nước đi khả dụng từ n đến 2n tùy vị trí quân hậu ở biên hay bên trong. column-swap có tối đa $C(n,2)$ cách chọn hai cột, tương ứng 28 và 66; một số cách hoán đổi hai quân ở cùng hàng không làm thay đổi trạng thái.

Toán tử dual-queen có số cách chọn hai cột rồi đổi hàng của cả hai về mặt cấu hình là $C(n,2)(n-1)^2$, tương ứng 1.372 khi $n = 8$ và 7.986 khi $n = 12$; đây là số cách phát sinh ứng viên, không nhất thiết là số trạng thái đích phân biệt. Với adaptive, kích thước tập ứng viên hiệu dụng phụ thuộc vào trạng thái hiện tại, việc chọn quân xung đột nhất và nhánh di chuyển một hay hai quân.

Một hạn chế đáng chú ý của column-swap là bảo toàn đa tập các hàng đang có quân hậu. Nếu cấu hình ban đầu có ít nhất hai quân hậu cùng hàng, chuỗi hoán đổi các cột không thể

loại bỏ sự trùng hàng đó để đạt nghiệm h = 0. Tính chất bất biến này giải thích vì sao toán tử có thể không giải được một phần lớn bài toán từ các trạng thái khởi tạo ngẫu nhiên.

8.5.3. Thực nghiệm trên n = 8 và n = 12

Mỗi toán tử được chạy trên 100 bàn cờ phân biệt ở n = 8 và 100 bàn cờ phân biệt ở n = 12. Mỗi phép so sánh trên cùng (n, trial) dùng chung bàn cờ đầu, song nguồn ngẫu nhiên trong quá trình lựa chọn nước đi được tách riêng. Với n = 8, mỗi lượt có tối đa 400 lần thử và dừng sau 120 lần liên tiếp không có cải thiện; với n = 12, giới hạn tương ứng là 700 và 250. Do chỉ nhận bước cải thiện, các thuật toán có thể dừng tại cực tiểu cục bộ hoặc khi cạn ngân sách.

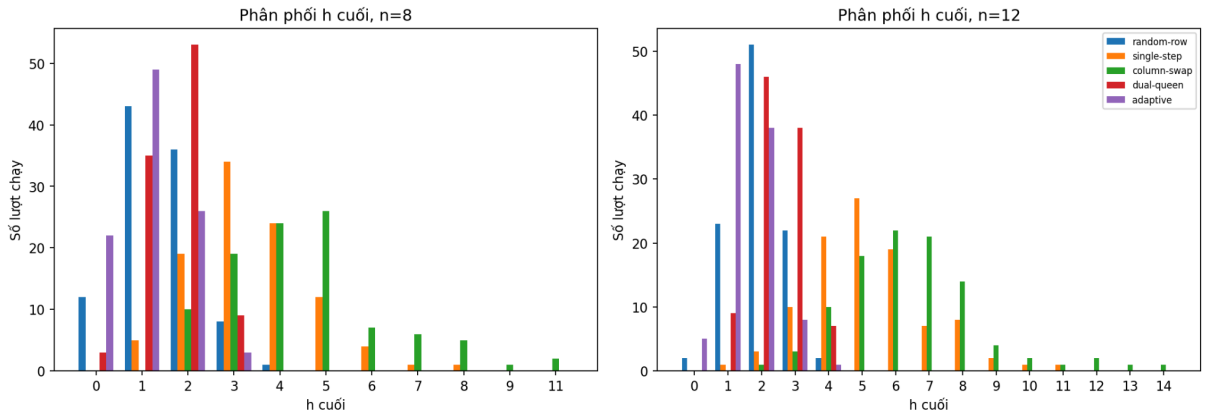
8.5.4. So sánh tỷ lệ thành công và h cuối trung bình

Bảng 8.4 tổng hợp kết quả 100 lượt chạy của từng toán tử ở mỗi kích thước. Tỷ lệ thành công được tính bằng số lượt đạt h = 0, còn h cuối trung bình phản ánh chất lượng nghiệm ngay cả khi chưa giải được hoàn toàn.

Bảng 8.4. Kết quả so sánh năm toán tử lân cận (100 lượt/cấu hình)

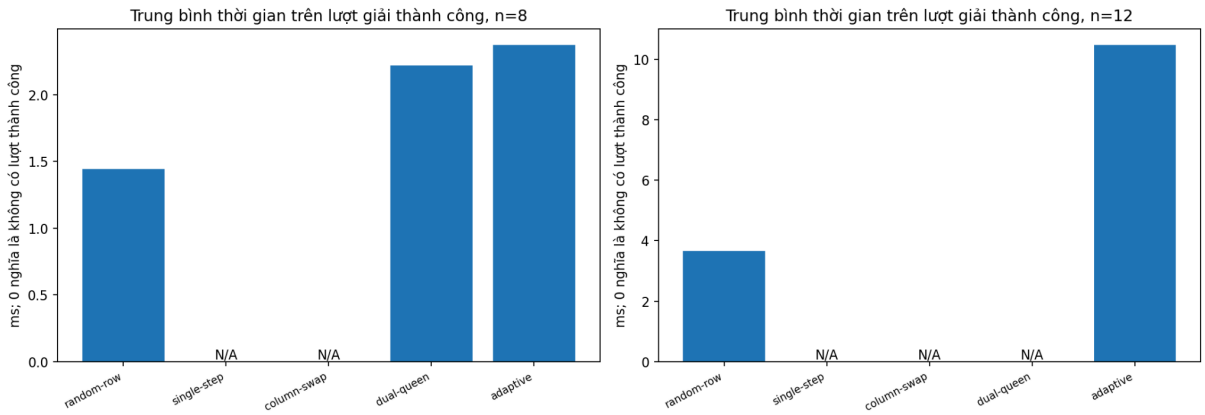
Toán tử	n=8: thành công (%)	n=8: h cuối TB	n=12: thành công (%)	n=12: h cuối TB
random-row	12	1,43	2	1,99
single-step	0	3,40	0	5,22
column-swap	0	4,58	0	6,50
dual-queen	3	1,68	0	2,43
adaptive	22	1,10	5	1,52

Toán tử adaptive cho kết quả tốt nhất trong tập khảo sát này, đạt 22% khi n = 8 và 5% khi n = 12; đồng thời có h cuối trung bình thấp nhất ở cả hai kích thước. random-row đứng tiếp theo với 12% và 2%. single-step và column-swap không có lượt đạt h = 0 trong bộ dữ liệu, nhưng 0% ở đây chỉ có nghĩa không thành công trong ngân sách và tập khởi tạo đã thử, không phải chứng minh tất cả trạng thái đều bất khả giải.



Hình 8.5. Phân phối số xung đột cuối sau 100 lượt chạy của mỗi toán tử.

Số liệu thời gian khi chỉ xét các lượt thành công cũng cần được đọc thận trọng. Ở n = 8, thời gian thành công trung bình của random-row, dual-queen và adaptive lần lượt khoảng 1,44; 2,22 và 2,37 ms. Với n = 12, chỉ random-row và adaptive có lượt thành công, lần lượt trung bình khoảng 3,66 và 10,47 ms. Những cột được đánh dấu N/A trên hình không có dữ liệu vì không tồn tại lượt thành công để tính trung bình; chúng không đại diện cho thời gian bằng 0.

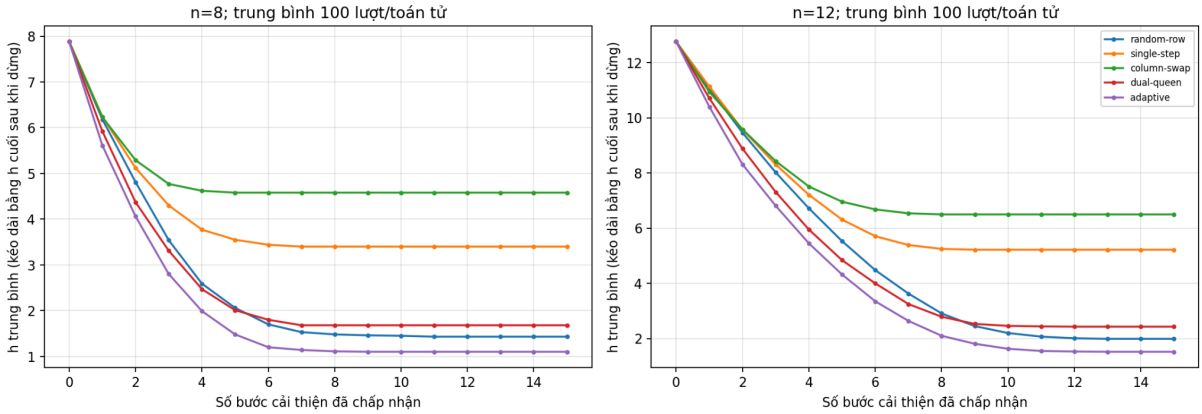


Hình 8.6. Thời gian trung bình có điều kiện trên các lượt thành công của từng toán tử.

8.5.5. Trực quan hóa quá trình hội tụ của các toán tử

Đường hội tụ trung bình từ 100 lượt cho thấy năm toán tử có tốc độ và giới hạn giảm h khác nhau. adaptive thường giảm số xung đột nhanh trong giai đoạn đầu và có giá trị h

cuối trung bình thấp nhất. dual-queen và random-row cũng giảm mạnh trong vài bước ban đầu, nhưng sau đó mức cải thiện chậm lại. single-step và đặc biệt column-swap sớm đi vào các đoạn gần như nằm ngang ở mức h còn cao.



Hình 8.7. H trung bình theo số bước cải thiện đã chấp nhận đối với năm toán tử lân cận.

Để vẽ các đường ở cùng chiều dài, notebook giữ nguyên giá trị h cuối của lượt đã dừng tại những chỉ số bước về sau. Đây chỉ là quy ước hiển thị nhằm tính trung bình trên cùng trục ngang; không có nghĩa thuật toán tiếp tục di chuyển sau khi đã dừng. Vì vậy, các đoạn nằm ngang phải được diễn giải cùng với điều kiện dừng và phân phối h cuối.

8.6. Đánh giá tổng hợp các phương pháp tìm kiếm

8.6.1. Tính hợp lệ và chất lượng nghiệm

Trong tất cả các thực nghiệm, tiêu chuẩn giải thành công là không còn cặp quân hậu nào cùng hàng hoặc đường chéo, tương đương h(board) = 0. Một cấu hình vẫn thỏa mãn cách biểu diễn mỗi cột một quân hậu nhưng có h > 0 chỉ là trạng thái hợp lệ về mặt biểu diễn, chưa phải nghiệm của n-Queens. Bên cạnh tỷ lệ đạt h = 0, h cuối trung bình cho biết khả năng giảm xung đột của phương pháp ở các lượt chưa tìm được nghiệm hoàn chỉnh.

8.6.2. Tác động của Local Optimum, Plateau và Ridge

Local Optimum xuất hiện khi không có trạng thái lân cận nào tốt hơn, dù lời giải toàn cục có thể nằm ở vùng khác trong không gian tìm kiếm. Plateau gồm những trạng thái có cùng giá trị h; Ridge có thể đòi hỏi chuỗi bước chuyển không thuận lợi theo toán tử hiện tại. Do các biến thể Hill Climbing trong notebook chỉ chấp nhận h giảm nghiêm ngặt, chúng không có cơ chế đi qua bước bằng điểm hoặc bước tăng xung đột. Kết quả tỷ lệ thành công thấp của những phương pháp này ở n = 8 phù hợp với hạn chế đó, mặc dù không thể chỉ từ số liệu tổng hợp mà xác định chính xác mỗi lượt đã vướng loại cấu trúc địa hình nào.

8.6.3. Hiệu quả của các biến thể Hill Climbing

Steepest-ascent đánh giá toàn bộ các lân cận ở mỗi trạng thái, chọn ứng viên cải thiện tốt nhất, nên số bước cải thiện có thể ít nhưng chi phí cho từng bước đáng kể. Stochastic (all) cũng phải duyệt hết các ứng viên trước khi chọn ngẫu nhiên một bước cải thiện, do đó không nhất thiết ít lần đánh giá hơn Steepest-ascent. First-choice chỉ thử từng ứng viên ngẫu nhiên, có thể giảm số công việc khi nhanh chóng tìm được cải thiện, nhưng cũng có thể tốn nhiều lần thử liên tiếp trước khi dừng theo hạn mức. Task 6 chứng minh không tồn tại một biến thể luôn dẫn đầu đồng thời về tỷ lệ thành công, số bước và thời gian.

8.6.4. Vai trò của Random Restart và Simulated Annealing

Random Restart giải quyết hạn chế do điểm xuất phát bằng cách đưa thuật toán đến các vùng khác nhau của không gian trạng thái; trong Task 6, cả ba biến thể đạt 100% ở n = 4 và n = 8. Simulated Annealing mở rộng khả năng thoát khỏi điểm kẹt bằng cách tạm chấp nhận bước xấu với xác suất phụ thuộc nhiệt độ. Trong cùng cấu hình Task 6, SA-Exponential có tỷ lệ thành công tốt hơn SA-Linear ở n = 8, song kết quả này phụ thuộc lịch nhiệt đã chọn chứ không khẳng định lịch Exponential luôn ưu việt trên mọi tham số hay bài toán.

8.6.5. Sự đánh đổi giữa tỷ lệ thành công và chi phí tính toán

Tỷ lệ thành công cao thường đạt được khi thuật toán có nhiều cơ hội thăm dò hơn hoặc cho phép chấp nhận những di chuyển không cải thiện tức thời. Điều này có thể tăng số lần gọi hàm mục tiêu hoặc kéo dài thời gian. Task 6 cho thấy RR-Steepest giải thành công 100% ở n = 8 nhưng dùng nhiều đánh giá hơn SA-Exponential, trong khi SA-Exponential chỉ đạt 91%. Trong Advanced Task, adaptive đạt tỷ lệ thành công và h cuối tốt hơn random-row, song phải tính thêm mức xung đột riêng của từng quân hậu để định hướng lựa chọn. Vì vậy, tiêu chí phù hợp phụ thuộc mục tiêu thực hành: cần nghiệm chắc chắn hơn trong ngân sách lớn, hay cần giảm nhanh h với tài nguyên hạn chế.

8.6.6. Hạn chế của bộ thực nghiệm và điều kiện áp dụng

Kết quả đo được thực hiện trên số lượng lượt chạy hữu hạn và các thiết lập cụ thể của notebook. Ngân sách giữa ba họ phương pháp không bằng nhau; bản thân Task 6, khảo sát khả năng mở rộng và Advanced Task cũng sử dụng các giới hạn khác nhau. Chi phí CPU, phiên bản thư viện và cách triển khai Python ảnh hưởng đến số đo thời gian. Ngoài ra, các đồ thị hội tụ ghép những trục chỉ số có ý nghĩa khác nhau; biểu đồ thời gian chỉ xét lượt thành công trong phần nâng cao không phản ánh thời gian trung bình của toàn bộ lượt chạy. Các kết luận vì vậy nên giới hạn trong bộ dữ liệu đã khảo sát và không được xem là chứng minh độ phức tạp hay bảo đảm nghiệm tổng quát.

8.7. Kết luận bài thực hành tổng hợp

Thông qua việc hoàn thiện Task 6 và Advanced Task, bài thực hành đã xây dựng một quy trình đánh giá thuật toán tìm kiếm cục bộ dựa trên cùng trạng thái khởi tạo, seed có thể tái lập, các chỉ số đo lường rõ ràng và bảng/biểu đồ đối chiếu. Kết quả cho thấy ba biến thể Hill Climbing có thể cải thiện nghiệm nhanh nhưng dễ dừng trước khi đạt h = 0; Random Restart nâng cao đáng kể tỷ lệ thành công bằng cách thử nhiều điểm xuất phát; Simulated

Annealing bổ sung khả năng thăm dò linh hoạt thông qua nhiệt độ và cơ chế chấp nhận bước xấu.

Khảo sát khả năng mở rộng cho thấy thời gian chạy gia tăng khi số quân hậu lớn hơn, nhưng mức tăng còn phụ thuộc vào ngân sách lần thử và cách cài đặt. Phần nâng cao khẳng định việc lựa chọn toán tử lân cận là một thành phần quan trọng của bài toán tối ưu: toán tử quá hạn chế có thể khó vượt qua điểm kẹt, trong khi toán tử có định hướng tốt hơn có thể cải thiện chất lượng nghiệm với chi phí xử lý bổ sung. Nhìn chung, hiệu quả của tìm kiếm cục bộ cần được đánh giá đồng thời bằng chất lượng nghiệm, tỷ lệ thành công, số lần đánh giá và thời gian thực thi thay vì dựa trên một chỉ số duy nhất.